

Helix Constitution – Universal CLAUDE.md

Field	Value									
Revision	2024 05 14									
Created	- -									
Last modified	- - T : : Z									
Status	active									

Status
summary

Added \$. . - Device-independent host-side rendere
user-facing UI surface MUST be proven by DEVICE-INDEPENDENT ho
PNG ON THE HOST (no device/emulator/running app; Compose→Robor
stack) for EVERY screen×state×{light,dark} theme, dual-validat
text+labels+control bounds" (NO overlap" / label-over-label / c
property-assertion UI tests are FORBIDDEN as the PROOF a UI is
stayed GREEN while the operator opened a broken giant-button s
golden-good/golden-bad analyzer \$. . (); device
COMPLEMENTS not replaces \$. . /. /. /.
exported-doc visual validation. Propagation gate CM-COVENANT-
paired \$. mutation. Classification: universal (\$. .
- -): Universal Semgrep static-analysis mandat
commit/PATH wiring, docs_chain semgrep_status context, and CM-
\$. . mirror - Universal Media Validation & Verifi
acceptance (OCR/transcription/text parsing vs SPECIFY-phase pa
pinpoint on FAIL, post-recording+real-time triggers, paired \$
Propagation & Hook System: post_update_hook.sh detects/regist
constitutional pull. Added \$. . mirror - Universa
(\$. . /.), iterates to zero-finding GO (\$. .
\$. . mirror - OpenDesign UI design system mandate
interfaces MUST use OpenDesign as the mandatory UI design-and-
dependency and use its design tokens/themes for color palette
\$. . for missing patterns; every UI component ships
NOT overlap or overlay labels; all UI changes covered by stand
(\$. .). Prior round: Added \$. . mirror -
project MUST use Podman in rootless mode (or equivalent rootle
sudo, or any escalation to root is FORBIDDEN unless the target
\$. . ; the vasic-digital/containers submodule (\$
docker/podman commands outside pkg/boot / pkg/compose / pkg/health
integration tests boot infra on-demand via the Submodule. Clas
- Vision-verified recording + HelixQA bridge mandate (User man
processed through a vision/OCR pipeline that reads on-screen c
provide a bridge feeding captured frames to HelixQA's test int
against specified expected patterns; the bridge MUST capture t
(\$. . ()), compare against SPECIFY-phase patter
for re-record per \$. . (L). Honest boundary: vision
analysis nor \$. . runtime-signature; FLAG_SECURE
(\$. .). Prior round: Added \$. . mirror -
video-recording confirmation (User mandate - -

```

(Status.md + $ . . Status_Summary.md) enumerating EV
cli_agents $ . . ), NO feature left out, reconciled vs
Implementation/Wiring($ . . )/Real-use/Tests-coverage
$ . . )/Video-confirmation; every user-visible confirm
responses→real results, no frozen frame $ . . , no f
$ . . /. SKIP; video-analysis remediation loop $
$ . . fingerprint; four-format export HTML+PDF+DOCX
PROPAGATION + recommended gates CM-FEATURE-STATUS-COMPLETE + CM
round: Added $ . . mirror – Crashlytics-recorded-d
(User mandate - - ): every project with Fireba
(fatal crashes, ANRs, performance traces, AND non-fatals), sys
and cover every closure with a permanent $ . . reg
issue id/URL + the validation/verification test paths; regular
console-resolved Crashlytics issue with no falsifiable regress
$ . . (review/dedup finds it; $ . . fixes
project's $ .0 + $ .AC. Composes
$ . . /. /. /. /. /. /. /. /. /.
$ . . Propagation gate CM-COVENANT-114-152-PROPAGATION + recom
($ . . ). Prior round: Added $ . . mirror -
. . ): every release tag AND version name on th
(e.g. myproject-1.0.0-dev-0.0.1 ); prefix resolution order ( )
documented in tracked .env.example $ . . ) else "( )" L
+ all owned submodules in one release = greppable across every
$ . . /. /. /. /. /. /. . Propa
CM-RELEASE-PREFIX-NAMING . Classification: universal ($ . . .
item integrity + testing-diary family (User mandate . . .
$ . . /. /. /. /. /. /. . i
valid status+type+stable id on ALL three surfaces; (D ) comp
items carry WHY + enumerated unblock CHOICES (tightens $ . .
regular never-missed bidirectional DB docs tracker sync, $
idempotent external-tracker push (statuses/types/assignee-from
$ . . ). $ . . per-workable-item TESTING DI
item_history/Reopens: append-only test_diary table with PASS-
format per-item exports $ . . -fingerprinted $ . .
minimal-LLM deterministic tooling % test-covered $
per-display brightness/auto-dim/font-size landed ONLY in the
COVENANT-114-148-PROPAGATION + CM-COVENANT-114-149-PROPAGATION + pa
agent respawn-until-complete + no-work-loss registry mandate
through its full lifecycle so a crash NEVER loses/forgets/corru

```

(status CLOSED SET {dispatched|in-flight|crashed|respawned|complete work owed?"; (b) mechanical CRASH-DETECTION (transient rate-l autonomously per \$. . . with ALREADY-DEFINED backof (\$. backup) in a per-agent worktree; (d) the \$. . . entry is non-complete . Forensic FACT this session: subag re-dispatched by pure conductor vigilance with NO mechanical \$. . /. /. /. /. /. /. /. /. Classification: universal (\$. .). Prior round: Added (User mandate - -): every device the project reachable transport – USB / wireless ADB / SSH / serial / netw continuously monitored, any drop handled, never silently aban recorder KNOWS a tracked device is absent (its loop guards on data, no offline event, no resume, no escalation) → silent cor system MUST automatically (a) DETECT + log an honest offline e (b) WAIT using the project's ALREADY-DEFINED reconnection tim recovery path already uses, (c) RE-ATTACH + log an honest onl device-recovery path (per \$. . feature class device sanctioned, authorization-gated recovery entry point ONLY, nev autonomously without that authorization (\$. . /. blocked-escalation honestly). Composes \$. . /. / Added \$. . mirror – Real-user-journey mandate for verbatim "All video streaming apps ... require to choose some t interaction to play exact show with proper content and subtitl that's it mostly!"): any full-automation test asserting a vide through the app's OWN UI – launch → BROWSE the actual catalog chosen content plays with its correct subtitles on the intende VIEW / synthetic shortcut (those validate ROUTING only, not r non-introspectable streaming UIs use the \$. . CV/ content confirmation via \$ " . . liveness + \$. . operator_attended SKIP-with-reason per \$. . /. ONLY blanking) – NEVER a faked routing-only PASS. Composes \$. . universal (\$. .). Prior round: Added \$. . exception (User mandate - - , verbatim "ALL 'ch ANY governed repository – no exception (source/fixes/tests/gat agent-output/refactors/one-liners) – MUST pass an INDEPENDENT \$. . (no "just a doc edit"/"just a one-liner"/"se bearing (reviewer structurally separated from author per \$ (\$. /. , conclusions captured-evidence \$. . /. NOT replace \$. . /. , it is the FIRST of MULTIPL

facts/captured-evidence + the existing codebase (blast radius)
 validate+verify every covering test genuinely exercises the wo
 any finding (defect / error-prone change / safety risk / will-
 fixed+polished+improved+test-covered (four-layer \$. . ()
 iterates until no blocking findings; one of MULTIPLE STRONG LA
 pass (author-side) + \$. . + \$. . . Comp
 \$. . /. /. /. /. /. /. /. /. /. /.
 round: Added \$. . mirror – Dead/unwired-code inve
 ⇒ dead ⇒ delete” is a guess (\$. .), never a finding –
 function/type/file/module/package/asset/config/build-target) t
 pickaxe, blame on the deleted call-site) and capture as FACT v
 real or a hidden reference (reflection/DI/build-tags/codegen/p
 captured PROOF it is genuinely unneeded AND MUST be its own se
 operator-confirmation for end-user capabilities; tracked via \$
 instead wire it in properly (restore a mistakenly-deleted call
 unwired tests (\$. . /. /.); ALWAYS be extra
 Classification: universal (\$. .). Prior round: Added
 - “ ” -): every reported issue / fix / claimed com
 (\$. . /. /.) before fixed/implemented/completo
 forbidden (\$. . /.”); when UNSURE how to validate, the ag
 discover/build an evidence-producing method – declaring “untes
 itself a \$. . violation; forensic case study (FAC
 research yielded the CV/OCR/liveness/sink-probe oracle stack
 Prior round: Added \$. . mirror – No-silent-remova
 - -): no application/component/service/package
 removed from the existing codebase/System without FIRST intera
 keep-or-remove decision; silent removal is a release blocker;
 without asking, operator reversed both; tracked DROP path ask-
 Classification: universal (\$. .). Prior round: Added
 . . -dev live-defect remediation, - -): \$
 against the known-good tag FIRST – the tagged-good version is
 polarity-switch (one test reproduces the defect on the pre-fix
 \$. . “ ” real-time conductor autonomous-test-framework
 snapshot the orchestrator tails live; every verdict event carr
 introspectable UIs (drive + assert via pixels when the access
 discovery-pressure to confirm known-issue-set completeness (f
 coverage), \$. . single-resource-owner partitionin
 resource – refines \$. . /.), \$. .
 mechanism; never fake-pass, never revert the fix), \$. .

Field	Value
	completion so partial/stale artifacts never land – build-output \$. . mirrors continue from earlier Revisions.
Issues	none
Issues summary	—
Fixed	\$. . mirror
Fixed summary	\$. . lands in lockstep with the Constitution.md \$
Continuation	—

Table of contents

- [How inheritance works](#)
- [MANDATORY DEVELOPMENT PRINCIPLES](#)
- [MANDATORY ANTI-BLUFF COVENANT – END-USER QUALITY GUARANTEE](#)
 - [§ . . – FAIL-bluffs are equally forbidden](#)
 - [§ . . – Recorded-evidence requirement](#)
 - [§ . . – Per-environment-topology test dispatch](#)
 - [§ . . – Test-interrupt-on-discovery + retest-from-clean-baseline](#)
 - [§ . . – Captured-evidence quality analysis](#)
 - [§ . . – No-guessing mandate](#)
 - [§ . . – Demotion-evidence rule](#)
 - [§ . . – Deep-web-research-before-implementation](#)
 - [§ . . – Batch-source-fixes-before-rebuild](#)
 - [§ . . – Credentials-handling mandate](#)
 - [§ . . .A – Pre-store credential leak audit \(User mandate, – – \)](#)

- § . . – File-layout discipline
- § . . – Auto-generated docs sync
- § . . – Out-of-band sink-side captured-evidence
- § . . – Test playback cleanup
- § . . – Item-status tracking
- § . . – Item-type tracking
- § . . – Universal-vs-project classification of new rules (User mandate, – –)
- § . . – Subagent-driven-by-default mandate (User mandate, – –)
- § . . – Script documentation mandate (User mandate, – –)
- § . . – Fixed-document column-alignment mandate (User mandate, – –)
- § . . – Operator-blocked status + self-resolution exhaustion (User mandate, – –)
- § . . – Document-sync commit discipline (User mandate, – –)
- § . . – Build-resource stats tracking mandate (User mandate, – –)
- § . . – Full-Automation-Coverage Mandate (User mandate, – –)
- § . . – Constitution-Submodule Update Workflow Mandate (User mandate, – –)
- § . . – Submodule-Dependency-Manifest Mandate (User mandate, – –)
- § . . – Post-Constitution-Pull Validation Mandate (User mandate, – –)
- § . . – .gitignore + No-Versioned-Build-Artifacts Mandate (User mandate, – –)

- 11 4 28
 § . . – Lowercase-Snake_Case-Naming Mandate
 (User mandate, - -)
2024 05 15
- 11 4 27
 § . . – Submodules-As-Equal-Codebase +
 Decoupling + Dependency-Layout Mandate (User mandate,
2024 05 15
 - -)
11 4 33
- 11 4 34
 § . . – No-Fakes-Beyond-Unit-Tests + %-
 Test-Type-Coverage Mandate (User mandate,
2026 05 15
 - -)
2026 05 15
- 11 4 35
 § . . – Type-aware closure-status vocabulary
 (User mandate, - -)
2026 05 15
- 11 4 36
 § . . – Reopened-source attribution mandate
 (User mandate, - -)
2026 05 15
- 11 4 37
 § . . – Canonical-root inheritance clarity
 (User mandate, - -)
2026 05 15
- 11 4 38
 § . . – Mandatory install_upstreams on clone/
 add Mandate (User mandate, - -)
2026 05 17
- 11 4 40
 § . . – Fetch-before-edit mandate (User
 mandate, - -)
2026 05 17
- 11 4 41
 § . . – Installable-Asset Evidence Mandate
 (User mandate, - -)
2026 05 17
- 11 4 42
 § . . – Full-suite retest before release tag
 mandate (User mandate, - -)
2026 05 18
- 11 4 43
 § . . – Pre-Force-Push Merge-First Mandate
 (User mandate, - -)
2026 05 18
- 11 4 44
 § . . – Iteration-discipline mandate (User
 mandate, - -)
2026 05 18
- 11 4 45
 § . . – TDD-Fix-Discipline mandate (User
 mandate, - -)
2026 05 18
- 11 4 46
 § . . – Document revision header mandate (User
 mandate, - -)
2026 05 18
- § . . – Integration-status-doc maintenance
 mandate (User mandate, - -)
- § . . – Validate-recent-work-before-post-
 flash-tests mandate (User mandate, - -)

- § 1.1.1 – Firebase data review mandate (User mandate, 2026-05-18)
- MANDATORY HOST-SESSION SAFETY^{1.2 1.0} (Constitution § 1.1.1)
 - Forbidden – directly OR indirectly
 - Required safeguards for heavy scripts
 - § 1.1.1.1 Memory-Budget Ceiling – 100% MAXIMUM
 - § 1.1.1.2 Continuation document maintenance
- MANDATORY ABSOLUTE DATA SAFETY – ZERO RISK (Constitution § 1.1.2)
- MANDATORY COMMIT & PUSH CONSTRAINTS
- MANDATORY TESTING CONSTRAINTS
- Code conventions
- When in doubt

This is the **base CLAUDE.md** imported by every project that includes the Helix Constitution submodule. Project-level **CLAUDE.md**^{1.2.26 0.5 1.4} may extend or tighten any rule by adding an explicit `Override: <section>` block – but MUST NOT weaken them.

Last revision: 2026-05-14 – 1.2.26

How inheritance works

A consuming project's root **CLAUDE.md** MUST start with a clearly-marked inheritance pointer:

```
## INHERITED FROM constitution/CLAUDE.md
```

All rules in ``constitution/CLAUDE.md`` (and the ``constitution/Constitution.md`` it references) apply unconditionally. The project-specific rules below extend them.

Claude Code supports the `@path/to/file import` syntax natively, so a consuming project can also write `@constitution/CLAUDE.md` at the top of its own `CLAUDE.md` and Claude Code will resolve it recursively. For agents that do not support `@imports`, the pointer-block pattern above ensures the inheritance is at least readable.

MANDATORY DEVELOPMENT PRINCIPLES

NO BLUFF. Every change ships with positive-evidence validation on the real target environment. A test that passes without exercising the user-visible behaviour is a critical defect.

(Constitution § . + § . – these references point to `constitution/Constitution.md`.)

CRITICAL: All code changes **MUST** follow these principles **WITHOUT EXCEPTION:**

- **Solutions MUST NOT be error-prone.** Every fix must be robust, not introduce new failure modes. If a fix solves problem A but creates problem B, it is NOT acceptable. Test the fix against ALL existing functionality before committing.
- **No blocking operations inside synchronized / shared-lock regions.** Long computations, network calls, or sleeps inside `synchronized` / `Lock`-held regions cause deadlocks or timeouts in other threads.
- **Always consider concurrent callers.** Any method called from multiple threads must be safe for rapid consecutive calls.
- **Test the fix, not just the symptom.** Verify the fix works AND doesn't break anything else.
- **Anti-bluff is mandatory** (Constitution § .) – every runtime test **MUST** source the project anti-bluff helper, call at least one explicit user-visible action before any **PASS**, capture state delta, and assert positive evidence.

- . **Real captured evidence for audio/video** (Constitution § . + § . .) – features producing audio output validated via captured audio; features producing video output validated via captured frames + OCR or pixel-diff.

2021 24 28

MANDATORY ANTI-BLUFF COVENANT – END-USER QUALITY GUARANTEE

Forensic anchor – verbatim user mandate (- -):

“We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

This is the historical origin of the project’s anti-bluff covenant. Every test, every Challenge, every gate, every mutation pair exists to make the failure mode (PASS on broken-for-end-user feature) mechanically impossible.

Operative rule: the bar for shipping is **not** “tests pass” but **“users can use the feature.”** Every PASS MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, “absence-of-error” PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests AND Challenges (HelixQA, integration suites, smoke tests, acceptance suites) are bound equally – a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does.

Canonical authority: constitution/Constitution.md § .
and its sub-sections § . . through § . . .

Non-compliance is a release blocker regardless of context.

§ . . – FAIL-bluffs are equally forbidden

A test that crashes for a script-internal reason (undefined variable under `set -u`, regex error, malformed assertion, missing argument) and produces a FAIL exit code is just as misleading as a PASS-bluff. Both let real defects ship undetected. Every test MUST fail ONLY for genuine product defects – script-bug failures must be fixed at the source layer^{1 1 4 2} (helper library, shared lib, test source), not patched in individual call sites.

§ . . – Recorded-evidence^{1 1 4} requirement

A test that emits PASS without captured visual or audio evidence of the user-visible feature actually working on the screen the user would see is a § . . PASS-bluff. Every PASS^{1 1 4 3} for a user-visible feature MUST be cross-checked against captured recording + action timeline.

§ . . – Per-environment-topology test dispatch

Tests that depend on environment topology MUST detect topology at test entry and dispatch the topology-appropriate variant. SKIP-with-reason is the correct fallback when the required topology is absent; PASS-by-default is forbidden.

§ . . – Test-interrupt-on-discovery + retest-from-clean-baseline

The moment any defect is re-discovered, re-produced, or newly identified during a test cycle, the cycle **MUST** stop. Then: systematic debugging → fix at root cause → four-layer test coverage (pre-build / post-build / runtime / meta-test paired mutation) → full rebuild → re-deploy on every target → full retest from beginning.

§ . . – Captured-evidence quality analysis

Audio: presence (RMS amplitude), channel count, sample rate + bit depth, glitch census, coexistence-artifact census. Video: presence (non-zero frame count), routing target, frame health (drops / jitter / freeze), obstruction census (OCR for hostile overlays), resolution + codec. Every check is required for every PASS.

§ . . – No-guessing mandate

Forensic anchor – verbatim user mandate (– –):

“‘LIKELY’ is guessing, we **MUST NOT** have guessing, since it can be or may not be! No bluffing and uncertainty is allowed at any cost! We **MUST** always know exactly precisely what is happening exactly, in any context, under any conditions, everywhere!”

Forbidden vocabulary in tests / gates / status reports / closure narratives / commit messages when describing causes: likely , probably , maybe , might , possibly , presumably , seems , appears to , guess , seemingly , apparently , perhaps , supposedly , conjectured , and synonyms.

Either prove the cause with captured forensic evidence and state it as fact, OR explicitly mark `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:` with a tracked-task ID for follow-up.

§ . . – Demotion-evidence rule

Demotion from a FAIL classification to a lower-severity classification requires positive evidence captured under the SAME CONDITIONS (same target, same build, same cycle position, same load profile) that originally exposed the defect. “I cannot reproduce in isolation” is a hypothesis, not a finding.

§ . . – Deep-web-research-before-implementation

Before designing a non-trivial fix, perform deep web research: official docs, vendor technical guides, open-source codebases, coding tutorials, issue trackers. Every non-trivial fix’s commit message (or accompanying entry) MUST cite at least one external source OR the literal “NO external solution found – original work”.

§ . . – Batch-source-fixes-before-rebuild

All source-side fixes that DO NOT require runtime validation to design MUST be landed BEFORE the next artifact rebuild. The anti-pattern of “fix A → rebuild → flash → cycle → fix B → rebuild → ...” serializes operator time onto rebuild latency.

§ . . – Credentials-handling mandate

Forensic anchor – verbatim user mandate (- -):

“Credentials or any secret and sensitive data MUST NOT leak!”

Credentials MUST NEVER be tracked in git. `.env` / `.env.*` / `*.env` patterns + `scripts/testing/secrets/*` (with `.example` + `README.md` exception) git-ignored project-wide. Test scripts MUST NEVER print or log credentials. Per-service file separation limits blast radius. `chmod 600` on credential files, `chmod 700` on parent directory. Rotation on suspected leak.

\$. . .A – Pre-store credential leak audit (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“Us these for all future testing (full automation testing) and make sure they are not leaking anywhere or get git versioned!”

When an operator provides credentials, API tokens, signing keys, or any other secret material to be stored in the project’s gitignored configuration, the storing agent MUST FIRST execute a repo-wide audit for prior leaks of THOSE specific values BEFORE storing: ()

`git ls-files | xargs grep -l <value>` for tree-leaks, ()
`git log -S<value> --all --source --remotes` for history-leaks,
() surface findings to operator BEFORE storing (operator may rotate, accept-as-compromise, or abort), () on finding open a \$ /\$ sixth-law-incidents record + redact tracked files in-place to `<redacted-per-§11.4.10>` + record OPERATOR ACTION REQUIRED for rotation per \$. . . sub-clause

, () extend pre-push hook credential-pattern grep to catch the escaped class in the same commit. See Constitution § . . .A for the full mandate.

§ . . – File-layout discipline

Project files organised by purpose, not historical accident. Source under canonical project roots. Tests under canonical test directories. Logs and forensic artifacts under operator-controlled directories – never scattered at repo root, never tracked unless they are reference assets.

§ . . – Auto-generated docs sync

Every auto-generated document MUST be regenerated in the same commit as any edit to its source. All output formats (.md + .html + .pdf) MUST stay in sync at all times.

§ . . – Out-of-band sink-side captured-evidence

Whenever a downstream consumer (HDMI sink, cloud monitor, downstream service) provides a network-accessible introspection API that reports what was actually received, the test suite MUST consume that report as captured-evidence. The on-source-side view alone is insufficient.

§ . . – Test playback cleanup

Every test MUST leave the target in a quiescent state. Cleanup mandatory on every exit path (trap '<cleanup>' EXIT). The orchestrator MUST run a post-test sanity check and FAIL the just-completed test if it left orphan state.

§ . . – Item-status tracking

Every active item in the project Issues file carries a `**Status:**` line within five lines of its heading. Six-state vocabulary: `Queued` , `In progress` , `Ready for testing` , `In testing` , `Reopened` , `Fixed` (→ `Fixed.md`) . Status updated as the item progresses. All three Issues / Issues_Summary / Fixed file types kept in sync (Markdown + HTML + PDF).

§ . . – Item-type tracking

Every active item in the project Issues file carries a `**Type:**` line within eight non-blank lines of its heading. Three-value CLOSED vocabulary: `Bug` (product defect / regression / user-visible broken behaviour), `Feature` (new capability not previously offered to end users), `Task` (internal workstream – refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The Issues_Summary file carries the Type column for every active item. All three Issues / Issues_Summary / Fixed file types kept in sync (Markdown + HTML + PDF). Pre-build gates `CM-ITEM-TYPE-TRACKING` + `CM-COVENANT-114-16-PROPAGATION` enforce the mandate.

§ . . – Universal-vs-project classification of new rules (User mandate, – –)

Before adding ANY new rule, mandatory constraint, covenant clause, gate, or “MUST”-bearing statement to a project’s Constitution / CLAUDE.md / AGENTS.md (or to a submodule’s equivalents), the author MUST classify it as **universal** (reusable across any project → goes into this constitution submodule) or **project-specific** (references particular hardware / vendor / package / region → stays in the project / submodule layer). The commit message MUST carry a `Classification:` line stating the choice + one-sentence rationale. Universal rules that leak project-specific

assumptions (hardware part numbers, vendor names, geographic regions, internal asset names) MUST be genericised first or downgraded to project-specific. When uncertain, default to project-specific (the narrower scope – lifting to universal later is cheap; the reverse is expensive). Pre-build gate `CM-UNIVERSAL-VS-PROJECT-CLASSIFICATION` audits new rule commits for the classification statement; paired mutation strips it and asserts gate FAILs.

§ . . – Subagent-driven-by-default mandate (User mandate, – –)

When the runtime supports subagent delegation (Claude Code Agent tool, Cursor task-runners, Aider sub-sessions, etc.), the primary agent MUST default to subagent delegation for any task that has multi-step scope ($\geq$ phases), parallelisable independent subtasks, long-running diagnostic loops, OR specialised domain workflows (code review, security audit, doc propagation). Foreground-only is reserved for single-file edits, mid-execution operator clarification, critical-state sequencing (commits / pushes / tags), or tasks so quick that subagent overhead exceeds the work. Sub-discipline: **tight scope** (– tasks, not “do everything”), **checkpoint commits** after each major task, **anti-stall protection** explicit in prompts, **anti-bluff verification** of subagent claims via repo state. Parallel subagents MUST partition non-overlapping files; `commit_all.sh --auto-cascade` bundles both via `git add -A`. Gate `CM-SUBAGENT-DELEGATION-AUDIT` (when implemented in consuming project) flags foreground multi-step work as a § . . violation.

§ . . – Script documentation mandate (User mandate, – –)

Every Bash / shell / POSIX-sh script anywhere in a project (`scripts/`, `bin/`, `tests/`, library directories, deployment hooks, CI helpers – depth-N recursive) MUST carry: () an

in-source documentation block (Purpose / Usage / Inputs / Outputs / Side-effects / Dependencies / Cross-references) at the top of the file; () an external user guide under docs/scripts/<script-name>.md covering Overview / Prerequisites / Usage examples / Edge cases / Internal behaviour / Related scripts / Last verified date. When a script is modified, BOTH the in-source block AND the external user guide MUST be updated in the SAME commit. **No documentation ever can be out of sync with its codebase.** Pre-build gate CM-SCRIPT-DOCS-SYNC walks every *.sh / *.bash under script directories, verifies a companion docs/scripts/<name>.md exists, AND verifies doc was modified in the same commit (or doc-mtime ≥ script-mtime as a softer floor). Paired mutation strips the doc-sync invariant and asserts gate FAILs.

\$. . – Fixed-document column-alignment mandate (User mandate, – –)

Every project that maintains an open-work tracker AND a closed-archive tracker MUST keep the two structurally aligned along the same lifecycle axes (Status + Type). For the Fixed archive that means: () every ### / #### heading in Fixed.md (or equivalent) carries a **Status:** line and a **Type:** line within non-blank lines of its heading; () a Fixed_Summary.md companion exists with the same column structure as Issues_Summary.md (# | Level | Status | Type | One-line description); () all three file formats (.md + .html + .pdf) for BOTH Fixed.md and Fixed_Summary.md stay in sync via the same single-shot wrapper that handles Issues + Issues_Summary; () closure migration is atomic – when an Issues entry resolves it moves to Fixed.md in the same commit, disappears from Issues_Summary (open-only), and appears in Fixed_Summary (closed-only). Status values for closed items are drawn from {Fixed (→ Fixed.md) | Fixed – pending device verification | Fixed – RECLASSIFIED} . Type values follow

\$. . : {Bug | Feature | Task} . Pre-build gate
 CM-FIXED-COLUMN-ALIGNMENT (+ invariants) – Fixed_Summary.md
 exists, table header carries Status+Type columns, mtime
 (Fixed_Summary ≥ Fixed), generator script present, sync
 wrapper invokes it, HTML+PDF exports for both Fixed and
 Fixed_Summary present. Paired mutation strips Status column
 from Fixed_Summary table header → gate FAILs. Classification:
 universal (per \$. .). No escape hatch.

\$. . – Operator-blocked status + self-resolution exhaustion (User mandate, – –)

Operator-blocked is the \$. . Status closed-set's
 th value:
 {Queued | In progress | Ready for testing | In testing | Reopened |
 Operator-blocked | Fixed (→ Fixed.md)} . It is a **last-resort
 classification**, earned only after the agent documents
 exhaustion of every applicable self-resolution path: (a)
 CLI / ADB / SSH / API access already available, (b) subagent
 delegation per \$. . , (c) existing repo tooling
 (scripts / helpers / libraries), (d) captured fallback
 (synthetic event, asset substitution, mock, \$. .
 topology SKIP), (e) external research per \$. . . Every
 Operator-blocked item MUST carry an ****Operator-Block-Details:****
 line within non-blank lines of its heading stating: **WHAT**
 (concrete action), **WHY** (each exhausted alternative
 enumerated), **UNBLOCK CONDITION** (observable signal), **WHO**
 (handle / contact / doc pointer). Issues_Summary.md lists
 Operator-blocked as a sortable Status value. Items MUST be re-
 evaluated every Nth tag cycle (project- defined, recommended
 ≥ rd cycle) – operator dependencies change. Fake Operator-
 blocked (no exhaustion audit) is a \$. . covenant
 violation at the planning layer, severity-equivalent to a
 PASS-bluff. Gates: CM-ITEM-OPERATOR-BLOCKED-DETAILS (every
 Operator-blocked heading has the details line), CM-OPERATOR-

BLOCKED-SELF-RESOLUTION- AUDIT (NEW Operator-blocked commits contain “Attempted: a – ...; b – ...; c – ...” trail).

Classification: universal (per § 1.1.4.2.2.2.6.0.5.1.4). No escape hatch.

§ 1.1.4.2.2.2.6.0.5.1.4 – Document-sync commit discipline (User mandate, – –)

Every project tracking work items through an Issues / Fixed lifecycle MUST provide a **lightweight commit path** distinct from the full-repo commit wrapper. The lightweight path stages, commits, and pushes ONLY the status-tracking doc set – Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + their HTML + PDF exports + any auto-generated audit artifact – so doc-status never drifts behind working-tree reality when the full-repo wrapper is unavailable (in-flight rebase, large submodule churn, partial network). The wrapper MUST: (a) auto-invoke the project’s export-regeneration pipeline first so Markdown + HTML + PDF stay in sync; (b) stage ONLY the explicit doc-set list – NEVER `git add -A`; (c) use a separate flock disjoint from the full-tree wrapper’s lock; (d) push to every parent-repo remote; (e) exit 3 on nothing-to-commit (informational, not error). The wrapper MUST be invocable standalone OR as a delegation flag on the full-tree wrapper (e.g. `commit_all.sh --docs-only`) so operators have a single mental model. Inherits the project’s § preflight discipline (refuses to run mid-meta-test). Gate CM-COMMIT-DOCS-EXISTS verifies wrapper + guide + flag + doc-set enumeration. Paired mutation strips the doc-set array → gate FAILs. Classification: universal (per § 1.1.4.2.2.2.6.0.5.1.4). Composes with § 1.1.4.2.2.2.6.0.5.1.4 (export-sync), § 1.1.4.2.2.2.6.0.5.1.4 (status tracking), § 1.1.4.2.2.2.6.0.5.1.4 (script documentation), § 1.1.4.2.2.2.6.0.5.1.4 (CONTINUATION maintenance). No escape hatch – doc-status drift is a § 1.1.4.2.2.2.6.0.5.1.4 PASS-bluff at the documentation layer.

§ . . – Build-resource stats tracking mandate (User mandate, – –)

Every project under this Constitution with a build exceeding minute wall-clock MUST run a host-side resource sampler for every build that captures memory used, CPU%, load average, disk read/write at a fixed interval (recommended s) and computes per-metric **min / max / mean / p** at stop. Per-build summaries appended to a TSV registry; the registry is the single source of truth – the human-readable Markdown report (and its HTML + PDF exports per § . .) is derived. Top of the report MUST surface **ever-values** (min / max / mean across all tracked builds). Per-build entries sorted most-recent-first; each row carries SUCCESS / FAIL / UNKNOWN + reason for FAIL. Sampler MUST itself stay under MB RSS and % CPU (Heisenberg-class observer constraint). Stop hook MUST be called from both success AND failure paths of the build wrapper. The Stats.{md,html,pdf} triple is committed via the project's § . . lightweight doc-sync wrapper. Gate `CM-BUILD-RESOURCE-STATS-TRACKER` + paired mutation hiding the monitor aside → gate FAILs. Classification: mixed (per § . .) – the discipline universal, the implementation paths project-specific. Composes with § . . / § . . / § . . / § . . (the host-safety forensic anchors are the empirical motivation for this telemetry). No escape hatch – build-resource debugging without time-series data is the bluff this anchor forbids.

2024 05 15

§ . . – Full-Automation-Coverage Mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product!”

For every consuming project, no feature / functionality / flow / use case / edge case / service / application on any supported platform may be considered **deliverable** until it is covered by automation tests proving six invariants: () anti-bluff posture (captured runtime evidence per § . + § .); () proof of working capability end-to-end on the target topology (per § . , not in a mock); () working implementation matching the documented promise; () no open issues / bugs surfaced by the suite (cross-checked against § . / § . trackers); () full documentation (user manual entry + § . for scripts) kept in sync per § . ; () four-layer test floor per § (pre-build + post-build + runtime + paired mutation). Consuming projects MUST publish a coverage ledger (feature × platform × invariant- .. × status), regenerated as part of release-gate sweeps, with gaps tracked per § . . Classification: universal .. (§ .). No escape hatch. A project that ships a feature without all six invariants is **not delivering a stable product** – severity-equivalent to a § . PASS-bluff at the release-gate layer. See Constitution § . for the full mandate (cross-cutting reach, composition, audit requirements).

§ . . – Constitution-Submodule Update Workflow Mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE committed and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!”

Before ANY modification to `constitution/Constitution.md`, `constitution/CLAUDE.md`, or `constitution/AGENTS.md`, the agent or operator MUST execute the following pipeline in order:

- . **Fetch + pull first** – inside the constitution submodule worktree run `git fetch` against every remote, then `git pull --ff-only` (or `--rebase` if non-FF-mergeable; never `--strategy=ours` / `--allow-unrelated-histories` without explicit authorization). The submodule MUST be at upstream tip BEFORE any local edit.
- . **Apply the change** – classify per § . . (only universal additions belong here; project-specific clauses stay in the consuming project’s governance). Cite the verbatim user mandate if one originated the change.
- . **Validate before commit** – run `meta_test_inheritance.sh` (or equivalent); verify no merge-conflict markers (`<<<<<<<`, `=====`, `>>>>>>>`); verify Constitution + CLAUDE + AGENTS cross-reference the new clause consistently.

- **Commit + push to ALL upstreams** – stage only the governance files (NEVER `git add -A` inside the submodule); commit message cites the user mandate + § 11.4.17 classification; push to every configured remote. A commit landing on one upstream but not others is a § 11.4.17 violation AND a § 11.4.17 violation.
- **Conflict resolution** – if `pull --ff-only` reports non-fast-forward, merge carefully (preserve union of governance content, no clause silently dropped, re-classify, re-validate). Force-push to “make conflicts go away” is FORBIDDEN (§ 11.4.17). Nothing about the constitution may be broken, made faulty, corrupted, or rendered unusable.
- **Post-merge validation + verification** – after the push lands, `git submodule update --remote --init` and re-run the consumer project’s cascade verifier (per CONST-11.4.17) to confirm the new clause reaches every owned submodule. Any cascade gap closed in the same change-window.
- **Bump consuming project pointer** – `.gitmodules`-tracked submodule pointer MUST be advanced to the new constitution HEAD in the SAME commit as any cascade work. Out-of-sync pointers are § 11.4.17 violations.

Classification: universal (§ 11.4.17). No escape hatch. A constitution-submodule change violating § 11.4.17 is a release blocker for every consuming project, severity-equivalent to a force-push without § 11.4.17 authorization. See Constitution § 11.4.17 for the full mandate (operational scope, cross-cutting reach).

§ 11.4.21 – Submodule-Dependency-Manifest Mandate (User mandate, 11.4.21)

Every owned-by-us submodule MUST ship a machine-readable dependency manifest at canonical path `helix-deps.yaml` (or `.json/.toml`) listing its own-org Git SSH dependencies. Schema (per Constitution § 11.4.21): `schema_version`,

```
deps: [{name, ssh_url, ref, why, layout: flat|grouped}] ,
transitive_handling.recursive: true ,
transitive_handling.conflict_resolution: operator-required ,
language_specific_subtree: bool .
```

Tooling: `incorporate-submodule <ssh-url>` adds the submodule at its declared canonical path (CONST- (C) flat/grouped), reads its `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/.helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per § 11.4.33. A manifest without this proof is a § 11.4.33 violation.

\$. . is the operational complement of

\$. . / CONST- (C): nested own-org submodule

chains are FORBIDDEN, manifests are the bridge that lets

consumers reconstruct the dependency graph at the parent

root.

Classification: universal (§ 1.1). Composes with § 1.1, § 1.2, § 1.3, § 1.4, § 1.5, § 1.6, § 1.7, § 1.8, § 1.9, § 1.10, § 1.11, § 1.12, § 1.13, § 1.14, § 1.15, § 1.16, § 1.17, § 1.18, § 1.19, § 1.20, § 1.21, § 1.22, § 1.23, § 1.24, § 1.25, § 1.26, § 1.27, § 1.28, § 1.29, § 1.30, § 1.31, § 1.32, § 1.33, § 1.34, § 1.35, § 1.36, § 1.37, § 1.38, § 1.39, § 1.40, § 1.41, § 1.42, § 1.43, § 1.44, § 1.45, § 1.46, § 1.47, § 1.48, § 1.49, § 1.50, § 1.51, § 1.52, § 1.53, § 1.54, § 1.55, § 1.56, § 1.57, § 1.58, § 1.59, § 1.60, § 1.61, § 1.62, § 1.63, § 1.64, § 1.65, § 1.66, § 1.67, § 1.68, § 1.69, § 1.70, § 1.71, § 1.72, § 1.73, § 1.74, § 1.75, § 1.76, § 1.77, § 1.78, § 1.79, § 1.80, § 1.81, § 1.82, § 1.83, § 1.84, § 1.85, § 1.86, § 1.87, § 1.88, § 1.89, § 1.90, § 1.91, § 1.92, § 1.93, § 1.94, § 1.95, § 1.96, § 1.97, § 1.98, § 1.99, § 2.00.

See Constitution § 1.1 for the full mandate.

§ . . – Post-Constitution-Pull
Validation Mandate (User mandate,
– –)

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run a full-project + recursive-submodule validation sweep BEFORE the new constitution HEAD is treated as canonical for any other work.

Sweep contract (canonical script: `scripts/verify-all-constitution-rules.sh`): re-runs the governance- cascade verifier; for every rule with a programmatic gate (CONST- `.gitignore` audit, CONST- `(C)` nested-own-org-chain audit, CONST- `case` audit, CONST- `(A)` mock-from-production audit, CONST- `anti-bluff` smoke), runs the gate against post-pull tree. Failures produce directed FAIL entries → tracker per `$ . .` with Status: `Reopened`, Type: `Bug`. Closure requires positive-evidence per `$ .` anti-bluff covenant.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook or commit wrapper); operator-explicit manual invocation also available.

Anti-bluff: sweep's own meta-test (paired mutation `$ .`) plants a known violation of each enforced gate and asserts sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a `$ . .` violation.

`$ . .` is the **enforcement engine** for every other `$ . .x` and CONST-NNN rule – without it, new rules cascade as anchors but never get enforced in the codebase.

Classification: universal (`$ . .`). Composes with every rule that has a programmatic gate. See Constitution `$ . .` for the full mandate.

`$ . .` – **.gitignore + No-Versioned-Build-Artifacts Mandate** (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“every project module, every Submodule, every service and application MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivative which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be committed every time we are preparing to execute commit! If any violation is detected it MUST be fixed before commit is executed!”

Every project module / owned-by-us submodule / service / application MUST ship a proper .gitignore covering the forbidden-from-version-control classes:

- **Build artefacts:** /bin/ , /build/ , /dist/ , /out/ , target/ , *.exe , *.dll , *.so , *.dylib , *.a , *.o , *.class , *.pyc , generator-produced files (when the generator is committed).
- **Cache files:** __pycache__/ , .pytest_cache/ , .mypy_cache/ , .ruff_cache/ , node_modules/ , .next/ , .nuxt/ , .cache/ , .gradle/ , .terraform/ , language-server caches.
- **Temp files:** *.tmp , *.swp , *~ , .DS_Store , Thumbs.db , *.orig , *.rej .
- **Sensitive-data files:** .env , .env.* (allow .env.example placeholder) , *.pem , *.key , *.crt , id_rsa* , id_ed25519* , .netrc , secrets/ , api_keys.sh .
- **Generated reports/logs:** *.log , coverage.out , htmlcov/ , runtime captures unless reference assets.
- **OS/IDE personal state:** .idea/ , .vscode/ (except shared settings) , .history/ .

Anti-bluff invariant: `.gitignore` line alone is not sufficient – no file matching the forbidden patterns may be currently tracked. A tracked `*.log` despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged` + `git status` BEFORE the commit. Forbidden-class hits abort the commit until fixed (un-stage, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation.

Secret-leak intersection: $\$ \dots$ composes tightly
with $\$ \dots + \$ \dots (\text{CONST-} \dots) - a \dots \text{.env}$
leak is BOTH a $\$ \dots$ and a $\$ \dots$
violation, requiring rotation + post-mortem.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it's a build derivative and MUST be ignored. Generators MUST be committed so consumers regenerate on demand.

Classification: universal (§ 1.1.1). No escape hatch beyond enumerated exceptions. Severity-equivalent to § 1.1.1. PASS-bluff at the repository-hygiene layer. Composes with § 1.1.1, § 1.1.2, § 1.1.3, § 1.1.4, § 1.1.5, § 1.1.6, § 1.1.7, § 1.1.8, § 1.1.9, § 1.1.10, § 1.1.11, § 1.1.12, § 1.1.13, § 1.1.14, § 1.1.15, § 1.1.16, § 1.1.17, § 1.1.18, § 1.1.19, § 1.1.20, § 1.1.21, § 1.1.22, § 1.1.23, § 1.1.24, § 1.1.25, § 1.1.26, § 1.1.27, § 1.1.28, § 1.1.29, § 1.1.30, § 1.1.31, § 1.1.32, § 1.1.33, § 1.1.34, § 1.1.35, § 1.1.36, § 1.1.37, § 1.1.38, § 1.1.39, § 1.1.40, § 1.1.41, § 1.1.42, § 1.1.43, § 1.1.44, § 1.1.45, § 1.1.46, § 1.1.47, § 1.1.48, § 1.1.49, § 1.1.50, § 1.1.51, § 1.1.52, § 1.1.53, § 1.1.54, § 1.1.55, § 1.1.56, § 1.1.57, § 1.1.58, § 1.1.59, § 1.1.60, § 1.1.61, § 1.1.62, § 1.1.63, § 1.1.64, § 1.1.65, § 1.1.66, § 1.1.67, § 1.1.68, § 1.1.69, § 1.1.70, § 1.1.71, § 1.1.72, § 1.1.73, § 1.1.74, § 1.1.75, § 1.1.76, § 1.1.77, § 1.1.78, § 1.1.79, § 1.1.80, § 1.1.81, § 1.1.82, § 1.1.83, § 1.1.84, § 1.1.85, § 1.1.86, § 1.1.87, § 1.1.88, § 1.1.89, § 1.1.90, § 1.1.91, § 1.1.92, § 1.1.93, § 1.1.94, § 1.1.95, § 1.1.96, § 1.1.97, § 1.1.98, § 1.1.99, § 1.1.100. See Constitution § 1.1.1 for the full mandate.

§ . . – Lowercase-Snake_Case-Naming
Mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (-):

“naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MUST HAVE lowercase names with space separator between the words of ‘_’ character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!”

Every directory, submodule, and file MUST use lowercase snake_case names (ASCII letters / digits / underscores, words separated by `_`). Existing non-compliant names MUST be renamed as part of the migration window opened by this clause. Every reference (configs, docs, links, source-code imports, governance files) MUST be updated atomically with the rename – reference drift after a rename is a § . . . violation of equal severity to the rename itself.

Exceptions (common-sense, must-not-break-technology).

Language- mandated case (Java/Kotlin package paths, Android resource directories, Apple framework dirs, C /Swift project layouts) is preserved inside the language-root. Submodule root directory follows our convention; language-specific subtree follows its own. Vendor/upstream third-party submodules keep their upstream names. Build artefacts (`node_modules/` , `__pycache__/` , `.git/` , `target/` , `build/` , `bin/`) keep tool-mandated names. “Does renaming break the technology?” trumps the rule.

Upstreams/ → upstreams/ transition. Constitution submodule’s `install_upstreams.sh` (exported via `.bashrc` / `.zshrc`) MUST support BOTH `Upstreams/` and `upstreams/` directory layouts during migration. Lowercase wins when both present. Uppercase fallback retires only by deliberate amendment.

“All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere, Bear-Suite, BoatOS, Helix-Flow, Helix-Track, Server-Factory – we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project’s codebase! We MUST work on that code as much as we do with main project’s codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE accessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project – directly from project’s root project_name/submodule_name or some more proper structure project_name/submodules/ submodule_name!”

Three cooperating invariants:

(A) Equal-codebase. Every owned-by-us submodule (orgs: vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory – dynamically discoverable via gh / glab CLIs) is an **equal part** of the consuming project’s codebase. Same engineering attention as main: analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, SQL, websites, all materials). A round that improves main while leaving an owned-submodule deficiency unaddressed is a § . . violation, severity-

equivalent to a \$. PASS-bluff at the project-scope layer. Coverage ledgers (\$. .) list every owned submodule as in-scope.

(B) Decoupling / reusability. Owned submodules MUST stay fully decoupled, project-not-aware, reusable, modular, completely testable. NEVER inject project-specific context (hardcoded paths, hostnames, asset names) INTO a submodule. When a submodule needs parent-project info, use configuration injection (env var, config file, constructor parameter) – never a hardcoded reach.

(C) Dependency-layout. Every dependency consumed by an owned submodule MUST be accessible from the parent project's root at:

```
<project_root>/<submodule_name>/  
<project_root>/submodules/<submodule_name>/
```

Nested own-org submodule chains are FORBIDDEN. A submodule MUST NOT have its own .gitmodules entries pulling in further owned- by-us repos. Add the dependency at the parent's root path; the submodule reaches it via documented import / SDK / runtime resolver. Third-party submodules exempt.

Gates: CM-OWNED-SUBMODULE-EQUAL-ENGINEERING (release-gate sweep audits parity), CM-OWNED-SUBMODULE-DECOUPLING (pre-commit greps for parent-project context inside the submodule diff), CM-OWNED-SUBMODULE-LAYOUT (pre-merge verifies canonical location + no nested own-org submodules + dependency lookup from root). Paired mutations (\$.) for all three. Classification: universal (\$. .). No escape hatch. Composes with \$, \$, \$. . , \$. . , \$. . , \$. . , \$. . , CONST- . See Constitution \$. . for the full mandate.

§ . . – No-Fakes-Beyond-Unit-Tests +
%-Test-Type-Coverage Mandate (User
mandate, - -)

Forensic anchor – verbatim user mandate (- -):

“Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interact with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE % covered with every supported test type: unit tests, integration tests, e e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).”

Two cooperating invariants:

(A) **No-fakes-beyond-unit-tests.** Mocks, stubs, fakes, placeholders, TODO , FIXME , “for now”, “in production this would”, or empty-implementation patterns are PERMITTED only in unit-test sources. Every other test type – integration, E E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA – MUST exercise the real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths. Gate CM-NO-FAKES-BEYOND-UNIT-TESTS + paired mutation.

(B) % test-type coverage. Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/ Challenges submodule fully incorporated), HelixQA (HelixDevelopment/ HelixQA submodule fully incorporated). HelixQA autonomous sessions drive end-to-end execution of every registered test bank with captured wire evidence per check.

Required dependency submodules (recursive per CONST-):
- Challenges - git@github.com:vasic-digital/Challenges.git - HelixQA - git@github.com:HelixDevelopment/HelixQA.git - Any other functionality submodules under vasic-digital/* / HelixDevelopment/* orgs the project depends on.

Pointers bumped to upstream HEAD in same commit as cascade work (\$. . step); pointer drift = \$. . violation.

Classification: universal (\$. .). Severity-equivalent to a \$ PASS-bluff at the release-gate layer. No escape hatch. Composes with \$, \$. , \$. . -\$. . (esp. \$. . - this is its strict expansion into per-type-of-test territory). See Constitution \$. . for full mandate.

\$. . - Type-aware closure-status vocabulary (User mandate, - -)

Every project that tracks work items by Type per \$. . MUST close them with the Type-appropriate closure-status word, drawn from this -element closed map:

Item	**Type:**	Closure	**Status:**	value
Bug		Fixed	(→ Fixed.md)	
Feature		Implemented	(→ Fixed.md)	
Task		Completed	(→ Fixed.md)	

The (→ Fixed.md) suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per \$. .) keeps working without per-Type branching. Generators (generate_issues_summary.sh , generate_fixed_summary.sh , status-counter helpers, the \$. . colorizer) MUST treat the three terminal values as semantically equivalent (all map to “closed, positive evidence captured”) while preserving the literal in the emitted document.

Closing a Feature with Fixed (→ Fixed.md) or a Task with Implemented (→ Fixed.md) is a \$ violation. Pre-build gate (recommended) CM-CLOSURE-VOCAB-TYPE-AWARE walks every Fixed.md heading + every Issues.md heading whose **Status:** is one of the three terminal values and asserts the Status-Type match. Composes with \$. . (status tracking), \$. . (type tracking), \$. . (Fixed-document column alignment), \$. . (colourisation). Classification: universal (per \$. .). No escape hatch.

\$. . – Reopened-source attribution mandate (User mandate, – –)

Every Issues.md (or equivalent project tracker) heading whose `**Status:**` is `Reopened` MUST carry, within non-blank lines of the heading, a `**Reopened-Details:**` line capturing four sub-facts:

- **By:** `AI` or `User` (source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). `User` covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).
- **Reason:** one-line cause classification – chosen from the closed vocabulary { `test-failed` | `manual-testing-detected` | `captured-evidence-contradicts` | `end-user-report` | `cycle-re-discovered` | `design-reconsidered` }. Other values are permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen – log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are \$. . / \$. . violations: the reopen IS a demotion-from-Fixed classification change, and demotion requires positive evidence captured under the conditions that re-exposed the defect.

The Issues_Summary.md (or equivalent) Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for “reopens by AI in the last days” is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)` .

A `Reopened` entry without `**Reopened-Details:**` is a \$. . violation. Pre-build gate (recommended) `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (\$. . walk pattern). Composes with \$. .

(no-guessing – Reason from closed vocabulary), § . . .
 (demotion-evidence – reopen IS a demotion from Fixed),
 § . . . (item-status tracking), § . . .
 (Operator-blocked discipline – same audit-line pattern).
 Classification: universal (per § . . .). No escape
 hatch.

§ . . . – Canonical-root inheritance clarity (User mandate, – –)

11 4 17

The constitution submodule's three files (`constitution/`
`Constitution.md` , `constitution/CLAUDE.md` , `constitution/AGENTS.md`)
 ARE the canonical root – also called the parent files. They
 contain only universal rules per §

The consuming project's repository-root files (`<project-root>/`
`CLAUDE.md` , `<project-root>/AGENTS.md` , optionally `<project-root>/`
`Constitution.md` or equivalent) are consumer extensions. They
 open with the inheritance pointer (either the Claude-Code
 native `@constitution/CLAUDE.md` import or the portable `##`
`INHERITED FROM constitution/CLAUDE.md` heading defined in this
 file's "How inheritance works" section). They contain only
 project- specific rules per § . . . – rules that
 reference particular hardware, vendor names, regulatory
 regions, internal asset names, or project-private
 conventions.

11 4 17

When in doubt about which file to edit: universal rule → edit
 constitution submodule's file; project-specific rule → edit
 consumer's file. Default consumer-side when uncertain (per
 § . . . , narrower scope is cheap to widen).

Terminology: when prose references "the parent CLAUDE.md" or
 "the root Constitution," the referent is the constitution-
 submodule file at `constitution/<filename>` , never the consumer's
 file. When it references "the project CLAUDE.md" or "this

project's AGENTS.md," the referent is the consumer-side file at `<project-root>/<filename>`. AI agents resolve ambiguous references via this rule.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit – `git mv` of a section if it's a clean clone, or an explicit "Lifted from to constitution per § . . ." / "Demoted from constitution to per § . . ." line in the commit message.

Pre-build gate (recommended) `CM-CANONICAL-ROOT-CLARITY` verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer (either `@import` or `## INHERITED FROM constitution/CLAUDE.md` heading), (b) the constitution submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with § . . . (universal-vs-project classification – § . . . defines the file-layer split that § . . . classifies INTO). Reading order: this anchor first, then § Classification: universal (per § . . .). No escape hatch.

§ . . . – Mandatory install_upstreams on clone/add Mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zshrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository’s upstreams!”

Every clone / add of a Git repository under any consuming project MUST be followed by `install_upstreams` invocation from that repository’s root IF its tree contains an `upstreams/` directory (or legacy `Upstreams/` per `$ . .` transition) populated with `*.sh` recipe files declaring upstream Git SSH URLs.

`install_upstreams` is a host-system utility on operator’s `PATH` (exported via `.bashrc / .zshrc`), implemented in this constitution submodule (`install_upstreams.sh`). The utility reads recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs across all declared upstreams.

Skipping the invocation when `upstreams/` IS present silently breaks `$ . .` (Multi-upstream push is the norm) – the next push lands on only one upstream. Gate `CM-INSTALL-UPSTREAMS-ON-CLONE` + paired mutation (`$ . .`).

Automation: `incorporate-submodule` (`$ . .`) and `scripts/init-submodules.sh` patterns auto-invoke `install_upstreams` when applicable. Operator-explicit manual invocation remains supported.

Pre-commit attention: before the first commit in the newly-cloned working tree, verify `git remote -v | grep -c push` reports the expected upstream count.

Classification: universal (§ 1.1.1). Composes with § 1.1.2, § 1.1.3, § 1.1.4, § 1.1.5, § 1.1.6, § 1.1.7, § 1.1.8, § 1.1.9, § 1.1.10, § 1.1.11, § 1.1.12, § 1.1.13, § 1.1.14, § 1.1.15, § 1.1.16, § 1.1.17, § 1.1.18, § 1.1.19, § 1.1.20, § 1.1.21, § 1.1.22, § 1.1.23, § 1.1.24, § 1.1.25, § 1.1.26, § 1.1.27, § 1.1.28, § 1.1.29, § 1.1.30, § 1.1.31, § 1.1.32, § 1.1.33, § 1.1.34, § 1.1.35, § 1.1.36, § 1.1.37, § 1.1.38, § 1.1.39, § 1.1.40, § 1.1.41, § 1.1.42, § 1.1.43, § 1.1.44, § 1.1.45, § 1.1.46, § 1.1.47, § 1.1.48, § 1.1.49, § 1.1.50, § 1.1.51, § 1.1.52, § 1.1.53, § 1.1.54, § 1.1.55, § 1.1.56, § 1.1.57, § 1.1.58, § 1.1.59, § 1.1.60, § 1.1.61, § 1.1.62, § 1.1.63, § 1.1.64, § 1.1.65, § 1.1.66, § 1.1.67, § 1.1.68, § 1.1.69, § 1.1.70, § 1.1.71, § 1.1.72, § 1.1.73, § 1.1.74, § 1.1.75, § 1.1.76, § 1.1.77, § 1.1.78, § 1.1.79, § 1.1.80, § 1.1.81, § 1.1.82, § 1.1.83, § 1.1.84, § 1.1.85, § 1.1.86, § 1.1.87, § 1.1.88, § 1.1.89, § 1.1.90, § 1.1.91, § 1.1.92, § 1.1.93, § 1.1.94, § 1.1.95, § 1.1.96, § 1.1.97, § 1.1.98, § 1.1.99, § 1.1.100. See Constitution § 1.1.101 for the full mandate.

§ 1.1.101 – Fetch-before-edit mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“Make sure that `feedback_fetch_before_edit` memory rule is part of our constitution Submodule – the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them!”

The FIRST git-touching action of any session, on any consuming project, MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u} --no-walk --no-pager # parent
git submodule foreach --recursive 'git fetch --all --prune --quiet'
```

If `HEAD..@{u}` is non-empty, integrate (ff-merge / rebase / surface to operator per § 1.1.102) BEFORE any local edit, scanner run, or test cycle. Multi-agent / multi-upstream codebases (Claude Code + Cursor + Aider + operator sessions in parallel) routinely lap each other; a –second fetch prevents the agent from redoing work a parallel session already finished, from filing a false-confidence “completion” of already-done work, and from doubling the multi-upstream conflict surface (§ 1.1.103) with sibling commits of the same change.

The check is non-negotiable even when the operator says “do X immediately” – skipping it on the basis of “nothing could have changed in the last N minutes” is a § 11.4.26 (no-guessing) violation: remote state is not knowable without a fetch. The fetch+log output (even if empty) is the captured evidence.

11 4 27

Scope: consuming project root + every owned submodule recursively (§ 11.4.27) + the constitution submodule itself (§ 11.4.27 step 1 made this explicit for constitution-side edits; § 11.4.27 generalises to ALL edits) + any dependency cloned via `incorporate-submodule` (§ 11.4.27) or `git submodule add` (§ 11.4.27).

1 1

Pre-build gate `CM-FETCH-BEFORE-EDIT-AUDIT` (when implemented in the consuming project) audits the most-recent commit range against the upstream HEAD at the commit’s parent – non-aligned parent = FAIL. Paired mutation (§ 11.4.17): synthetic commit whose parent was N commits behind the then-current upstream HEAD → gate FAILs.

11 4 27

Classification: universal (§ 11.4.27). No escape hatch. Composes with § 11.4.27, § 11.4.27, § 11.4.27, § 11.4.27, § 11.4.27, § 11.4.27. See Constitution § 11.4.27 for the full mandate.

3.2.2.6 0.5 1.7

§ 11.4.27 – Installable-Asset Evidence Mandate (User mandate, – –)

For any user-distributable build artifact (package, bundle, installer, or container image produced by the build pipeline and distributed to end users), tests and challenges MUST open the artifact and verify each user-visible asset is **present** and **non-degenerate**.

A PASS without opening the artifact and verifying the asset chain end-to-end is a § 11.4.27 PASS-bluff. The specific failure mode: source file exists → build packages it → post-

build checks pass at the source layer → artifact produced with the asset stripped or misconfigured, and no gate ever opens the artifact to verify.

Each consuming project ships one challenge script per artifact type that opens the produced artifact and verifies every declared user-visible asset. The challenge MUST run as part of the project's standard QA gate.

Classification: universal (\$. .). No escape hatch. See Constitution \$. . for the full mandate.

\$. . – Full-suite retest before release tag mandate (User mandate, – –)

A release tag MUST NOT be created until a COMPLETE retest with ALL existing tests has been executed on a clean baseline AFTER every workable item in the batch is done, fixed, polished, and individually verified. Spot-check retests that run only the tests touched by the batch are FORBIDDEN – they miss interaction defects between batch fixes and previously-stable code.

The complete retest comprises: () pre-build full sweep, () post-build full sweep, () on-device -phase cycle on EVERY owned device, () meta-test full mutation sweep, () Challenge bank full sweep, () Issues.md/Fixed.md state audit, () CONTINUATION.md sync check.

Time is essential – typically – h elapsed effort. NOT optional, NOT abbreviated. Skipping is the exact “tests pass but feature broken” failure mode \$. . prohibits.

Composes with \$. . (per-fix retest still required at fix granularity) + \$. . (full-suite retest is authoritative baseline for closures) + \$. . (per-feature on-device validation runs as step of the full-suite retest).

Classification: universal (§ 2.0.2.6). No escape hatch.
See Constitution § 2.0.2.6 for the full mandate.

§ 2.0.2.6 – Pre-Force-Push Merge-First Mandate (User mandate, 2.0.2.6)

Forensic anchor – verbatim user mandate (2.0.2.6):

“make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides!”

Any force-push (`--force` , `--force-with-lease` , `+<ref>` , equivalent history-rewrite) authorised under CONST-MUST be preceded by a -step merge-first pipeline:

- **Fetch every remote** – `git fetch --all --prune --tags` against origin + every upstream; capture output.
- **Integrate every divergent commit locally** – rebase / merge / operator-confirmed cherry-pick per the appropriate strategy for every non-empty `HEAD..<remote>/<branch>` range.
- **Audit the integrated tree** – no conflict markers anywhere (`grep -rn '^<<<<<< \|^===== $\|^>>>>>>'` returns empty in governance + source + test files); no file silently dropped; previously-passing tests still pass; captured-evidence artefacts still validate.
- **Force-push** – only after steps - produce clean integration evidence: `git push --force-with-lease` (NEVER `--force` alone unless authorised per § 2.0.2.6 sub-clause 2.0.2.6).

Two-gate composition with CONST- . § . . does NOT relax CONST- 's operator-approval requirement – it adds a SECOND mechanical gate. Both required: CONST- alone authorises a push that loses remote work; § . . alone risks pushing without operator awareness.

Three failure modes prevented: (a) remote-side content loss when parallel sessions land work between fetches; (b) stale-state acts when --force-with-lease reads stale local refs; (c) conflict-driven corruption when markers get committed verbatim (observed - - in helix_qa + containers governance files).

Verification artefact – docs/changelogs/<tag>.md “Force-push merge-first audit” section captures fetch output, per-remote divergence log, integration strategy, conflict- marker scan, test delta, push output with lease SHA, CONST- authorisation quote. Gate CM-FORCE-PUSH-MERGE-FIRST + paired mutation.

Classification: universal (§ . .). No escape hatch. Composes with § . , § . . , § . . , § . . , § . . , § . . , CONST- , CONST- . See Constitution § . . for the full mandate. § . . – Iteration-discipline mandate (User mandate, - -)

Project work proceeds in priority-ordered iteration cycles. Each cycle has five mandatory steps: () select TOP + MIDDLE critical items only (defer LOW until critical batch closed), () batch implementation with § . . four-layer coverage + § . . batch-source-fixes-before-rebuild, () smoke gate (< min – batch- touched tests + critical-path regression probe + anti-bluff baseline), () ONLY if smoke GREEN and no new operator/user report arrived, run § . . full retest (- h), () release-ready OR loop back to step with new evidence added to queue.

Composes with § 11.4.40 (per-fix retest inside step),
 § 11.4.42 (closures require same-conditions evidence; step
 is authoritative baseline), § 11.4.43 (source-side
 batching inside step), § 11.4.44 (Reopened items
 attribute source), § 11.4.45 (the multi-hour retest IS
 step 11.4.5 – § 11.4.46 is the meta-loop conductor).

Anti-bluff coupling: every smoke and full-system PASS MUST
 carry positive captured evidence per § 11.4.42 +
 § 11.4.43. Tests AND HelixQA Challenges bound equally.

No escape hatch – no --skip-priority-batch , no --skip-smoke ,
 no --full-suite-only . Subagents default to the § 11.4.43
 path.

Classification: universal (§ 11.4.43). See Constitution
 § 2.0.2.6 for the full mandate.

§ 11.4.47 – TDD-Fix-Discipline mandate (User mandate, – –)

Every fix MUST follow the –step TDD-fix workflow: **RED**
 (failing test FIRST, real product defect per § 11.4.47 ,
 captured evidence per § 11.4.48) → **LIVE-ADB-PROBE** (try fix
 on running device via adb shell for mutable surfaces –
 setprop persist.* , settings put , pm clear , am ... , boot-
 script push; INFEASIBLE for kernel / framework / HAL /
 vendor / init.rc / sepolicy / ro. / Android.bp – must
 rebuild; cite LIVE_PROBE_INFEASIBLE: <reason> in commit message)
 → **GREEN** (source patch achieves same effect, batched per
 § 11.4.49 , four-layer coverage per § 11.4.50) → **VERIFY**
 (re-run RED test, must PASS under SAME conditions per
 § 11.4.51 , captured positive evidence per § 11.4.52 ,
 –iteration reliability per § 11.4.53) → **DOCUMENT**
 (Issues.md → Fixed.md with type-aware closure vocabulary per
 § 11.4.54 , CLAUDE.md Applied Fixes row, changelog,
 guides, HelixQA bank, CONTINUATION.md per § 11.4.55 – all
 in the SAME commit).

No escape hatch – no `--skip-red-test`, `--no-live-probe`, `--skip-verify` flag. “Test added after the fix” is a `$` `PASS-bluff`: it demonstrates the test agrees with the fix, not that the test catches the bug.

Classification: universal (`$` `.` `.` `.`). Composes with `$` `.` `.` `.` / `$` `.` `.` `.` / `$` `.` `.` `.` / `$` `.` `.` `.` / `$` `.` `.` `.` / `$` `.` `.` `.` / `$` `.` `.` `.` . See Constitution `$` `.` `.` `.` for the full mandate.

`$` `.` `.` `.` – Document revision header mandate (User mandate, `-` `-` `-`)

Every tracked document in scope (Issues.md, Issues_Summary.md, Fixed.md, Fixed_Summary.md, CONTINUATION.md, docs/guides/, **docs/research/**, docs/scripts/, **docs/changelogs/**, docs/superpowers/plans/, **docs/hardware/**, all other docs/ tracked Markdown) MUST...carry a header block directly below the H title containing two MANDATORY fields: ****Revision:**** N (monotonic positive integer, never reset, never skipped) and

****Last modified:**** YYYY-MM-DDTHH:MM:SSZ (ISO UTC). Optional fields (Description, Authority, Maintainer, Scope) encouraged but not gated. CLAUDE.md / AGENTS.md / README / LICENSE / VERSION / rendered HTML / rendered PDF artifacts are explicitly OUT of scope (revision tracked via VERSION file or auto-derived from source Markdown).

Auto-bump: `scripts/doc_revision_bump.sh` <file> (idempotent), pre-commit hook for automatic staged-doc bumps, `scripts/testing/sync_issues_docs.sh` auto-bumps Issues_Summary / Fixed_Summary after regeneration, `scripts/commit_docs.sh` calls the bump before stage. CONTINUATION.md’s existing `Last updated:` line per `$` `.` `.` `.` IS the `$` `.` `.` `.` Last

modified: line – composed, not duplicated. HTML/PDF exports inherit revision from source Markdown via pandoc pipeline. No escape hatch – no `--skip-revision-bump` flag exists anywhere.

Pre-build gates `CM-DOC-REVISION-HEADER-PRESENT` + `CM-COVENANT-114-44-PROPAGATION` ..+ paired mutations per `$ . .`. Composes with `$ . .` (CONTINUATION.md header reuse), `$ . .` (Issues_Summary regen), `$ . .` (commit_docs.sh hook entry), `$ . .` (HTML colorizer preserves revision), `$ . .` (companion doc for doc_revision_bump.sh).

Classification: universal (`$ . .`). See Constitution `$ . .` for the full mandate.

`$ . .` – Integration-status-doc maintenance mandate (User mandate, – –)

Every non-trivial domain integration MUST have a `docs/<domain>/<integration>/Status.md` document that () exists when more than one `fix/test/gate` has landed for the integration, () carries the `$ . .` revision header, () is auto-synced (HTML + PDF) on every related test cycle and every fix touching the integration, () is auto-colored per `$ . . . .`, () has a sync wrapper invocable as `bash scripts/testing/sync_integration_status.sh` or a per-integration thin shell, () lives under `docs/<domain>/<integration>/`, () includes a captured-evidence- driven status table per `$ . .` (every claim cites the test log / recording / sink-probe report that backs it), (.) uses the closed status vocabulary `PASS / FAIL / SKIP / PENDING-FORENSICS / OPERATOR-BLOCKED`, () lists operator-blocked items at the top so operators find action items in `O( )`, () is referenced from `docs/CONTINUATION.md` `$` (Active work) when any item is non- terminal.

\$. . is the generic form of \$.
 (CONTINUATION.md) applied to every integration domain.
 Without this generalisation each new integration re-invents
 the same sync wrapper, revision-header discipline, captured-
 evidence requirement, and operator-blocked surface.

Pre-build gates CM-COVENANT-114-45-PROPAGATION + CM-AF-
 INTEGRATION-STATUS-DOCS + paired mutations (propagation strip /
 Revision-line delete / ..sync-staleness / vocabulary
 violation). Composes with \$. . (captured evidence),
 \$. . (sync wrapper pattern reused), \$. .
 (sink-side evidence is a specific instance), \$. .
 (status vocabulary), \$. . (commit_docs.sh wrapper),
 \$. . (colorizer), \$. . (revision
 header), \$. (CONTINUATION.md references Status.md
 paths).

No escape hatch – no --skip-status-sync , no --no-revision-bump-
 on-status , no --allow-stale-html flag.

Classification: universal (\$. .). See Constitution
 \$. . for the full mandate.

\$. . – Validate-recent-work-before- post-flash-tests mandate (User mandate, – –)

After every device flash, the orchestrator MUST first run a
 recent- work validation pass (targeted on-device tests for
 items currently in Issues.md In progress / Ready for testing /
 Reopened , Fixed.md items closed within last days,
 CONTINUATION.md \$ active work). Only if that pass is
 % green does the orchestrator proceed to the full post-
 flash suite.

Helper: `scripts/testing/recent_work_validate.sh --device <serial>`
writes `/data/local/tmp/.recent_work_validated` IFF green; the
full suite (`test_all_fixes.sh`) refuses to start without it.
Marker is invalidated on reboot (stores device boot epoch).

Composes with `$ (STOP-on-discovery) + $ (no-guessing) + $ (demotion-evidence) + $ (full-suite gate) + $ + $ + $`. Each recent-item fix MUST have a paired `$ RED-then-GREEN` – a GREEN with no prior RED is a bluff.

Pre-build gates `CM-COVENANT-114-46-PROPAGATION` + `CM-AF-RECENT-WORK-VALIDATION-GATE` + `CM-AF-VALIDATION-ARTIFACT-FILE` + paired mutations.

Classification: universal (`$`). No escape hatch – no `--skip-validation`, `--full-suite-always`, `--ignore-recent-work` flag. See Constitution `$` for the full mandate.

`$` – Firebase data review mandate (User mandate, – –)

Before every “bigger working round” (pre-build, pre-flash, pre-tag, daily, post-deployment burn-in) the operator/loop MUST execute `scripts/firebase/review_round.sh`. The pass queries Crashlytics (fatals + non-fatals + ANRs) + Analytics + Performance, classifies findings by severity, dedup-maps to existing Issues.md entries via a three-tier algorithm (exact Firebase Issue-ID match → stacktrace- similarity cluster hash → operator merge review), and drafts new Issue entries for unrecognised findings with full Firebase Console URL refs + `$` (a) systematic-debugging output. Skipping the pass is a `$ PASS-bluff` – Firebase IS the captured evidence from real end-user devices.

Five mandatory elements: () –trigger cadence (pre-build / pre- flash / pre-tag blocking, daily / burn-in non-blocking), () –source query (all three sources), ()

- § 11.4.52. – Live-ADB-First Maximization Mandate (User mandate, 2026-01-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs: § 11.4.52. § 11.4.51. § 11.4.51.
- § 11.4.51. – Autonomous-Validation Mandate (User mandate, 2026-01-19) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs: § 11.4.51. > “Make sure we have full automation tests which will do all this > work in full automation! IMPORTANT: Make sure that all existing > tests and Challenges do work in anti-bluff manner – they MUST > confirm that all tested codebase really works as expected! We had > been in position that all tests do execute with success and all > Challenges as well, but in reality the most of the features does > not work and can’t be used! This MUST NOT be the case and execution > of tests and Challenges MUST guarantee the quality, the completion > and full usability by end users of the product!”

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via `adb shell` + scripted automation, captured runtime evidence per § 11.4.51, PASS/FAIL verdict WITHOUT a human present to drive UI, observe screen, or make decisions. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY validation path is operator-attended is a § 11.4.51 violation – the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact “tests pass but feature doesn’t work for users” failure mode § 11.4.51 forbids.

Acceptable autonomous paths: programmatic instrumentation APK (SDK-API exercises like `MediaCodec.createDecoderByName` + JSON result file), headless intent dispatch + state poll (`am start --es` + `dumpsys / /proc/<pid>/maps / media.metrics polling`), ADB-driven uiautomator (ONLY if `uiautomator dump | grep -c clickable=true` ≥ 1 – near-empty hierarchy proves UI-driven

INFEASIBLE and demands fallback to APK/intent), network-side sink probe (Arvus dashboard, Sonos REST, etc. per § 11.4.23), HelixQA autonomous QA session (§ 11.4.27).

Per-feature coverage ledger (§ 11.4.25) MUST classify each row as AUTONOMOUS_VERIFIED / AUTONOMOUS_DESIGNED / OPERATOR_ATTENDED_ONLY / NOT_APPLICABLE. OPERATOR_ATTENDED_ONLY is a release blocker until promoted, citing a tracked migration work item per § 11.4.26 + § 11.4.28. Autonomous paths themselves MUST be anti-bluff: positive captured evidence per § 11.4.32, paired meta-test mutation per § 11.4.29.

Composes with § 11.4.30 (full-automation-coverage – § 11.4.31 strictens invariants +), § 11.4.33 (no-fakes-beyond-unit + § 11.4.34 % type coverage – operational layer closing the gap), § 11.4.35 (per-feature on-device end-user validation – refined to mandate autonomous drivability), § 11.4.36 (TDD-fix – autonomous path RED before fix, GREEN after), § 11.4.37 (UI-driven – fallback to APK/intent when uiautomator hierarchy empty), § 11.4.38 (dual-approach – Intent variant often IS the autonomous path), § 11.4.39 (deterministic consistency – autonomous paths scale to N iterations), § 11.4.40 (live-ADB-first – instrumentation APK is LIVE_ADB_TESTABLE).

Pre-build gates CM-COVENANT-114-52-PROPAGATION (anchor literal across canonical files) + CM-AF-AUTONOMOUS-PATH-PER-FEATURE (coverage-ledger classification column non-empty + valid value). Paired mutations strip the anchor literal AND inject an OPERATOR_ATTENDED_ONLY row without a tracked migration item. No escape hatch – no --allow-operator-attended-only, --skip-autonomous-path, --manual-validation-suffices flag.

Canonical authority: constitution submodule Constitution.md
§ 11.4.41.

Non-compliance is a release blocker regardless of context.

- \$. . - Fixed_Summary parity mandate (User mandate, - -) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs: \$. . > “Note: Just like for Issues we have Issues_Summary, for Fixed we > MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and > up to date and ALWAYS exported into the PDF and HTML! Add this > mandatory rule / constraint into the root (constitution Submodule) > Constitution, AGENTS.MD and CLAUDE.MD.”

docs/Fixed_Summary.md is the symmetric short-form summary of docs/Fixed.md . MUST be regenerated whenever Fixed.md changes. HTML + PDF exports MUST travel with the markdown (identical mtimes within sync_issues_docs.sh granularity). Stale exports are \$. . violations regardless of whether the underlying .md is correct – an operator (or future agent) reading the HTML or PDF gets a divergent view of which items are closed, and the \$. CONTINUATION resumption guarantee silently breaks. Same discipline as \$. . Issues_Summary applied to Fixed.md.

Generator: scripts/testing/generate_fixed_summary.sh (canonical, MUST be executable, MUST emit a markdown table whose header columns include Status and Type per \$. . . column-alignment). Auto- sync wrapper: scripts/testing/sync_issues_docs.sh regenerates BOTH Issues_Summary AND Fixed_Summary in one shot (stages . . a + b), then exports HTML + PDF (stage), then colorizes per \$. . (stage), then re-renders the PDFs from the colorized HTML (stage b). MUST be invoked after any edit to Fixed.md . No --issues-only flag exists, and \$. . prohibits adding one.

Sort order: closure date DESC (most-recent-Fixed first), \$-letter / Fix- secondary. Documented at the top of the generated file.

Composes with § 23 . . . (Issues_Summary parity is the sibling rule; § 11 . . . + § 11 + 23 . . . are the canonical pair), § 11 . . . (atomic Issues→Fixed migration triggers Fixed_Summary regen – column-aligned structure), § 11 . . . (visual-cue + grouping colorizer post-processes both summaries), § 11 . . . (type-aware closure vocabulary – Fixed_Summary respects Fixed (→ Fixed.md) / Implemented (→ Fixed.md) / Completed (→ Fixed.md) terminal values literally), § 11 . . . (revision header applies to Fixed_Summary.md), § 11 . . . (CONTINUATION.md resumption guarantee depends on divergent-summary problem NOT existing).

Pre-build gates CM-FIXED-SUMMARY-SYNC (invariants: Fixed_Summary exists; HTML + PDF mtime ≥ md mtime; Fixed_Summary mtime ≥ Fixed mtime; generator script exists + executable; sync wrapper invokes generator) + CM-COVENANT-114-53-PROPAGATION (anchor literal across canonical files + per-consumer propagation). Paired mutations strip the anchor literal AND move the generator aside AND backdate Fixed_Summary mtime. No escape hatch – no --skip-fixed-summary-sync , --issues-only , --summary-not-applicable flag.

Canonical authority: constitution submodule Constitution.md
 § 11 . . . 2026 05 13

Non-compliance is a release blocker regardless of context.

- § 11 . . . – ATM-NNN ticket identifier mandate (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs:
 § 11 . . . > “Every workable item in Issues.md / Issues_Summary.md / Fixed.md / > Fixed_Summary.md MUST carry a stable, unique, auto-incremental > ATM-NNN ticket identifier. ATM- prefix, monotonic, never > renumbered, append-only.”

Every workable item in `docs/Issues.md` AND `docs/Fixed.md` MUST carry a `[ATM-NNN]` ticket identifier in its heading (form `## $X.Y. [ATM-NNN] <title>`, zero-padded $\geq$ digits). Identifiers are allocated by `scripts/testing/assign_atm_ticket_ids.sh` and persisted to an append-only state file `scripts/testing/.atm_ticket_state.json` (jsonl: `atm_id`, `heading_hash`, `type`, `current_location`, `current_status`, `reopens_count`, `created_at`, `last_modified`). Once assigned, an ATM-NNN MUST NEVER be renumbered, reused, decremented, or skipped. `heading_hash` is the binding key – wording reflows preserve the binding via the state-file lookup.

`Issues_Summary.md` and `Fixed_Summary.md` MUST carry an `ATM ID` column as the leftmost data column. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`) emit the column so operators / agents can sort + filter on it.

Composes with `$ . . . (Status)`, `$ . . . (Type)`, `$ . . . (column-alignment)`, `$ . . . (closure vocabulary)`, `$ . . . + $ . . . (Issues_Summary + Fixed_Summary regen – helper invoked from sync_issues_docs.sh)`, `$ . . . (per-item Reopens.md path key)`, `$ . . . (README.md doc-link cross-reference key)`.

Pre-build gates `CM-ATM-TICKET-IDS-COMPLETE` (every heading carries `[ATM-NNN]`) + `CM-ATM-TICKET-IDS-UNIQUE` (no duplicates) + `CM-ATM-TICKET-IDS-MONOTONIC` (no gaps) + `CM-COVENANT-114-54-PROPAGATION` (anchor literal across canonical files). Paired mutations strip an `[ATM-NNN]` heading token, dup an ID in the state file, gap the sequence at `NNN=`, strip the anchor literal. No escape hatch – no `--skip-atm-assignment`, `--renumber`, `--no-atm-id-required` flag.

Canonical authority: constitution submodule `Constitution.md`
`$ . . .`

Non-compliance is a release blocker regardless of context.

11 4 55

- § . . – Reopens-history tracking + per-item Reopens.md doc (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs:
 § . . > “Add a Reopens-count column. For any item whose reopens-count > , > create docs/issues/ATM-NNN/ Reopens.md (+ HTML + PDF) with > comprehensive reopen history + Fixed cycles. Each reopen MUST > include By (AI / User), On, Reason, Evidence, and each > Fixed-marking with reasoning chain.”

1 11 4 44

2

Every workable item with `reopens_count > 0` MUST have a companion document at `docs/issues/<ATM-NNN>/Reopens.md` (+ HTML + PDF). The document MUST contain: () § . . revision header, () item identification (ATM ID, Title, Type, Current Status, Current Location, link back to live heading), () cycle counters (Total reopens, Total fixed cycles), () chronological timeline – one entry per state-change event, each with By (AI/User per § . .), On (ISO date), Event (Opened/Reopened/Fixed/Implemented/ Completed), Reason (closed-vocabulary value from § . . for reopens; captured-evidence summary for closures), Evidence (path or short description), Outcome, () reasoning chain for each closure (root-cause analysis, captured-evidence under same conditions per § . . , gate / mutation pair), () most-recent state-change pointer.

11 4 23

2

Issues_Summary.md and Fixed_Summary.md MUST carry a `Reopens` column; cells with count > hyperlink to the per-item Reopens.md. The § . . colorizer MAY apply a visual cue when reopens > .

11 4 55

11 4 34

11 4 44

11 4 45

Composes with § . . (per-event capture in current heading – § . . is the per-item history aggregation), § . . (ATM-NNN provides the stable path), § . . (revision header), § . .

(Status.md per-integration analogue), § . .
(Fixed_Summary parity – Reopens column symmetric on both summaries).

Pre-build gates CM-REOPENS-DOC-EXISTS-WHEN-COUNT-GT-ZERO + CM-REOPENS-DOC-REVISION-HEADER + CM-REOPENS-COL-IN-SUMMARIES + CM-COVENANT-114-55-PROPAGATION . Paired mutations delete a Reopens.md for a reopens_count= item, strip the revision header, remove the column from Issues_Summary, strip the anchor literal. No escape hatch – no --skip-reopens-doc , --collapse-history , --reopens-not-applicable flag.

Canonical authority: constitution submodule Constitution.md
§ . . .

Non-compliance is a release blocker regardless of context.

- § . . – Status_Summary parity + two-audience format (User mandate, –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs:
§ . . > “Every Status.md doc gets a Status_Summary parity companion > ALWAYS in sync, exported to HTML + PDF. Two-page format: page = > non-developer audience (team-specific), page = software > engineer summary. Auto-generated after every main Status update.”

For every docs/<domain>/<integration>/Status.md (§ . .) a companion Status_Summary.md MUST exist with: ()
§ . . revision header, () **Page – For the – audience- specific heading** (audio team for docs/dolby/* , video team for docs/video/* , etc.) – plain-language summary, What works (– bullets), What’s broken or pending (– bullets), Operator / team actions if any. NO code references, NO §-letter jargon, NO captured-evidence file paths, NO gate / mutation names. () **Page – For software engineers** – §-letter references, gate names, commit hashes, captured-evidence paths, ATM-NNN cross-references. HTML + PDF exports travel with the markdown.

Generator: `scripts/testing/generate_status_summary.sh <Status.md path>` produces both pages. Invoked from `scripts/testing/sync_integration_status.sh` (`$ . . sync wrapper`) on every `Status.md` update.

Composes with `$. . (Status.md per integration – Status_Summary.md COMPLEMENTS, never replaces), $. . + $. . (parity discipline), $. . (revision header), $. . (colorizer for tracked-item references on page), $. . (CONTINUATION.md – non-developer stakeholders read Status_Summary; engineers read Status + CONTINUATION + Issues).`

Pre-build gates `CM-STATUS-SUMMARY-EXISTS-FOR-EVERY-STATUS + CM-STATUS-SUMMARY-TWO-AUDIENCE + CM-STATUS-SUMMARY-REVISION-HEADER + CM-COVENANT-114-56-PROPAGATION`. Paired mutations delete a `Status_Summary.md`, remove the `Page` heading, strip the anchor literal. No escape hatch – no `--skip-status-summary`, `--engineer-only`, `--no-audience-split` flag.

Canonical authority: constitution submodule `Constitution.md`
`$ . . .`

Non-compliance is a release blocker regardless of context.

- `$ . . – README.md doc-link section + revision metadata (User mandate, – – ) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:`
`$ . . > “Add a doc-link section to README.md – links to Issues + > Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + ALL > Status docs + their exports. Each link shows revision + > last-modified.”`

Every project’s top-level `README.md` MUST contain a section titled `Tracked-Items + Status Documents` (heading MUST contain literal `Tracked-Items`). The section is a markdown table with columns: `Document`, `Last modified` (ISO UTC from `$ . . header`), `Revision` (integer from same header), `Markdown` link, `HTML` link, `PDF` link. The section MUST link

```

11  4  19      11  4  59
12  18
to: Issues.md + Issues_Summary.md ($ . . ,
$ . . , $ . . ), Fixed.md + Fixed_Summary.md
($ . . , $ . . ), CONTINUATION.md
($ . . ), every docs/**/Status.md + its Status_Summary.md
pair ($ . . , $ . . ). Status docs are auto-
discovered by the generator via find docs -name 'Status.md' .

```

Generator: scripts/testing/update_readme_doc_links.sh extracts each doc's Revision + Last modified, renders the markdown table, replaces the section between markers (<!-- doc-link-section:begin --> / <!-- doc-link-section:end -->) in README.md. Invoked from sync_issues_docs.sh (Issues / Fixed edits) AND sync_integration_status.sh (Status edits).

```

11  4  53      11  4  44      11  4  43      11  4  58
Composes with $ . . + $ . . +
$ . . (Issues / Fixed / summaries – README links all
four), $ . . (revision header data source),
$ . . + $ . . (Status pairs enumeration),
$ . . (CONTINUATION.md explicit link).

```

Pre-build gates CM-README-DOC-LINK-SECTION-PRESENT (literal Tracked-Items + section markers) + CM-README-DOC-LINK-ROWS-COMplete (every canonical doc appears as a row) + CM-README-DOC-LINK-FRESHNESS (Last modified matches source within sync-wrapper granularity) + CM-COVENANT-114-57-PROPAGATION. Paired mutations strip the markers, remove the CONTINUATION row, backdate a Last modified cell, strip the anchor literal. No escape hatch – no --skip-readme-doc-links, --collapse-status-rows, --no-freshness-check flag.

Canonical authority: constitution submodule Constitution.md

```

11  4  57      2026  01  13
$ . .

```

Non-compliance is a release blocker regardless of context.

- \$. . – Parallel-development methodology (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs: \$. . > “We MUST DO one comprehensive research

and planning in the background in > parallel with current mainstream work: our current methodology of the > development is very slow. ... We MUST CREATE adjusted improved version of > working methodology where multiple workable items could be done in > parallel, all come into the central (main) branch as they are done, then > we at particular moment rebuild and reflash the System and in background > full testing with validation and verification is done. Each parallel work > on one workable (or more) item(s) must use parallel agents as much as > possible! ... Writing in depth tests (all supported types of the tests) > with Challenges and full HelixQA use is MANDATORY! Every test we execute > besides executed with success MUST RESULT in proof that actual > functionality being tested REALLY DOES WORK with NO BLUFF of any kind! > Heavt enforcement of no-bluff / anti-bluff policy IS MANDATORY!"

Project work proceeds through the **Parallel Work Unit (PWU) pipeline** rather than sequential phase-chain. Each PWU is a self-contained workable item with mandatory components: ATM-NNN identifier per \$. . . , Issues.md entry per \$. . . +\$. . . , file-scope manifest, \$. . . RED test, source patch, pre-build gate per \$. . . (b) layer , post-flash test per \$. . . (b) layer , paired \$. . . meta-test mutation, \$. . . (b) layer HelixQA Challenge bank entry, captured-evidence directory per \$. . . +\$

-stage pipeline: (Stage DEVELOP – parallel PWU agents in isolated worktrees) → (Stage MERGE – serial conductor via `commit_all.sh flock + $ . . . -step merge-first`) → (Stage REBUILD+FLASH – parallel where hardware allows) → (Stage VALIDATE – parallel on D +D +meta-test+ coverage) → (Stage SWEEP – parallel HelixQA Challenges + Fixed.md migration + README refresh). Stage of round N+ overlaps with Stages – of round N – the throughput multiplier.

Synchronization mechanism: -layer lock hierarchy (L parent flock / L per-submodule git ops / L contention-path advisory locks for the forbidden cross-PWU paths / L per-PWU git worktree). Disjoint-scope PWUs run fully parallel. Conflict detection at Stage 1: `git fetch --all --prune --tags` + scope-overlap check → overlap detected rejects PWU back to Stage 1.

```

Anti-bluff enforcement at merge time (all four required): C
$ . . . RED-test captured-evidence (proves test
catches regression), C $ . . . paired meta-test mutation
(proves gate is not bluff), C $ . . .
deterministic-consistency ( . . . or . . . iterations
identical), C $ . . . captured-evidence (audio WAV /
video screen recording / UI uiautomator dump / HelixQA
result.json ). Metadata-only / configuration-only / absence-
of-error / grep-without-runtime PASS all REJECTED.

```

HelixQA mandatory. Every user-visible PWU MUST add a Challenge entry in `tools/helixqa/banks/atmosphere.yaml` referencing the PWU's ATM-NNN + dispatching to its on-device test + scoring PASS only on positive captured evidence per § . . .

Pre-build gates CM-PWU-LOCK-HIERARCHY + CM-PWU-ANTI-BLUFF-COVERAGE + CM-PWU-MERGE-QUEUE-DISCIPLINE + CM-PWU-PARALLEL-AGENT-LIMIT + CM-COVENANT-114-58-PROPAGATION (anchor literal across consumer files). Paired mutations strip the lock helpers / forge a READY marker without evidence / bypass the merge queue / spawn a 11 4 4 th concurrent agent / strip the anchor literal – every mutation FAILs its gate.

No escape hatch — no `--skip-merge-queue`, `--allow-bypass`, `--no-anti-bluff-check`, `--unlimited-agents`, `--sequential-phase-chain-mode` flag. Composes with \$. , \$. ,
\$. , \$. , \$. , \$. ,
\$. , \$. , \$. , \$. ,

§ . . , § . . , § . . , § . . ,
 § . . , § . . , § . . , § . ,
 § . , § . , § . , § . .

Canonical authority: constitution submodule `Constitution.md`
 § . . . Project-specific implementation reference in
 consumer-side `docs/guides/PARALLEL_DEVELOPMENT_METHODODOLOGY.md` .

Non-compliance is a release blocker regardless of context.

- § . . . – README always-sync mandate (User mandate,
 –) [full → `CLAUDE_ANCHORS_FULL.md` +
`Constitution.md`] • refs: § . . § . .
 § . . § . . § . .
 § . . § . . § . .
- § . . . – Documentation always-sync composite
 covenant (User mandate, – –) [full →
`CLAUDE_ANCHORS_FULL.md` + `Constitution.md`] • refs:
 § . . § . . § . .
 § . . § . . § . .
 § . . § . . § . .
 § . . § . . § . .
 § . . § . . § . .
- § . . . – Workable-items procedure docs as single
 source of truth (User mandate, – –) [full
 → `CLAUDE_ANCHORS_FULL.md` + `Constitution.md`] • refs:
 § . . § . . MANDATORY HOST-
 SESSION SAFETY (Constitution §)

Every script, test, helper, and AI agent MUST respect host-session safety. Non-compliance is a release blocker.

Forbidden – directly OR indirectly

- Suspending the host (`systemctl suspend` , etc.).
- Hibernating / hybrid-sleeping the host.
- Logging out the user (`loginctl terminate-session` , `pkill -u <user>` , anything targeting `user@<uid>.service`).

- . Unbounded-memory operations inside `user@<uid>.service` cgroup – any command expected to exceed ~ GiB RSS MUST be wrapped in a bounded execution scope.
- . Programmatic `rkill` toggles, lid-switch handlers, power-button handlers – these cascade into idle-actions.
- . Disabling session managers “to make things faster”.

Required safeguards for heavy scripts

- . Source the project’s host-safety helper library.
- . Call its pre-flight check and abort if it fails.
- . Wrap any subprocess expected to exceed ~ GiB RSS in a bounded execution scope.
- . Cap parallelism to fit available RAM.

\$. Memory-Budget Ceiling – % MAXIMUM

Forensic anchor – verbatim user mandate:

“First make sure that whatever we do through our procedures related to this project MUST NOT use more than % of total system memory! All processes MUST be able to function normally!”

Project procedures MUST NOT use more than % of total system RAM. The remaining % is reserved for the operator’s other workloads. No escape hatch – bypassing this is the bluff \$. forbids.

§ . Continuation document maintenance

`docs/CONTINUATION.md` MUST exist at the project root and reflect the live state of the work. Every non-trivial state change updates it in the same commit. Any agent must be able to resume work exactly where the previous session left off by reading this single file.

MANDATORY ABSOLUTE DATA SAFETY – ZERO RISK (Constitution §)

Every destructive repository operation (history rewrite, force-push, branch deletion, bulk file removal, submodule de-init, object pruning) MUST follow the full § safety protocol WITHOUT EXCEPTION:

- . **Backup first, always** – hardlinked mirror of `.git` to a sibling backup directory
(`cp -al .git <backup>/repo.git.mirror`) is near-instant and uses zero additional disk.
- . **Record metadata** – refs, tags, submodule pointers, HEAD commit, HEAD tree hash, tree content sha .
- . **Define expected post-op state.**
- . **Run the destructive operation** – never with `--no-verify` , never with `--force` that bypasses hooks, never auto-force on failure.
- . **Post-op gate** – HEAD tree identical, all tags preserved, all submodule pointers intact, per-entry archive integrity %, pre-build gates green. If any check fails → restore immediately.
- . **Force-push authorization** – force-push is NEVER automatic. Each force-push event requires explicit human authorization AND requires the post-op gate to have passed.
- . **Audit trail** – every history rewrite gets a `docs/changelogs/` “Force-push audit” section.

Hardlinked backup is so cheap (zero disk, < 1s) that there is NEVER an excuse to skip it.

MANDATORY COMMIT & PUSH CONSTRAINTS

- . Use the project's official commit wrapper for the main repo (e.g. `scripts/commit_all.sh`).
- . NEVER use `git add`, `git commit`, or `git push` directly on the main repo unless the project Constitution explicitly carves out a use case.
- . Multi-upstream push is the norm – every commit pushed to ALL configured upstream remotes. The constitution submodule ships an `install_upstreams.sh` (and an `Upstreams/` directory) that configures all remotes locally.
- . NEVER skip hooks (`--no-verify`, `--no-gpg-sign`) unless the user explicitly authorizes it for the session.

MANDATORY TESTING CONSTRAINTS

Tests MUST be run at every stage WITHOUT EXCEPTION:

- . Pre-build / pre-merge verification – BEFORE every build.
- . Post-build / packaging verification – AFTER every build.
- . Post-deploy verification – AFTER every deploy / flash.

NEVER skip tests. NEVER mark a test as “broken – disable for now” without fixing the underlying issue. NEVER ship a release with unresolved WARNs.

Code conventions

Universal conventions applicable to most projects:

- Use the project's preferred language for new code. Don't mix languages in one module without explicit Constitution permission.
- Static analyzers / linters / type-checkers MUST run clean (zero warnings) before commit.
- Style is set by the project's formatter; do not hand-edit style in PRs.
- Public APIs are documented at the source.

When in doubt

- Read `constitution/Constitution.md` for the canonical text.
- Cross-reference the project's `CLAUDE.md` / `AGENTS.md` for project-specific extensions.
- If still unclear, ask the operator. Do NOT guess. Do NOT bluff.

2026 03 19

11 4 12 11 4 18 11 4 23

- `$ 11 4 44 • 11 4 • 11 4 52` – Universal Markdown export mandate (User mandate, `10 11 4 63` – –) [full → `CLAUDE_ANCHORS_FULL.md + Constitution.md`] • refs:

`$ 11 4 66 . . $ . . $ . .`
`$ . . $ . . $ 2024 15 19`
`$ . . $ . . $ . .`
`$ 11 4 6 11 4 7 11 4 48 11 4 41 . .`
`$ . . $ . .`

- `$ 11 4 42 • 11 4 • 11 4 66` – Blocker-resolution interactive-clarification mandate (User mandate, – –) [full → `CLAUDE_ANCHORS_FULL.md + Constitution.md`] • refs:

`$ . . $ . . $ . . $ . .`
`$ . . $ . . $ . .`

- § 11 4 1 11 4 4 11 4 4 11 4 10 – Shell-script target-shell-parseability mandate (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:


```

$ . . $ . . $ . . $ . .
$ . . $ . .

```
- § 11 4 40 11 4 10 11 4 50 – Positive sink-side / downstream evidence mandate (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:


```

$ . . $ . . $ . . $ . .
$ . . $ . . $ . .
$ . . $ . .

```
- § . . – Universal sink-side positive-evidence taxonomy + mechanical enforcement (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . > “THIS MUST HAPPEN NEVER AGAIN!!! We MUST HAVE this all working! > Not just for audio but for every single piece of the System!!! > Proper full automation when executed with success MUST MEAN that > manual testing will be as much positive at least regarding the > success results! ... Solution MUST BE universal, generic that > solves working flows for all System components and for all > future and all existing projects! ... Everything we do MUST BE > validated and verified with rock-solid proofs and anti-bluff > policy enforcement and fulfillment!”

Universal generalisation of § . . (audio-specific) across every user-visible feature class. Closes the PASS-bluff pattern where tests reported green while end users hit broken features (– – → D audio “ / PASS” + empty Arvus Codec-In-Use).

The mandate. Every user-visible feature MUST map to one entry in the closed-set § . . sink-side evidence taxonomy (audio_output, audio_input, video_display, network_throughput, network_connectivity, bluetooth_a dp, bluetooth_pair, touch_input, sensor, gpu_render,

storage_read, storage_write, mediacodec_decode, mediacodec_encode, miracast, cast, boot_service, package_install, permission_grant, wifi_link, wifi_throughput, ethernet_link, display_topology, drm_playback, subtitle_render – open to additions). Every PASS for a feature in the taxonomy MUST cite a captured-evidence artefact path matching the required evidence shape.

Helper contracts (additive during grace; mandatory after - -):

- `ab_pass_with_evidence <description> <evidence_path>` – the new canonical PASS helper. Verifies path exists AND non-empty; emits `PASS: <description> [evidence: <path>]` .
- `ab_skip_with_reason <description> <closed-set-reason>` – reasons: `geo_restricted` , `operator_attended` , `hardware_not_present` , `topology_unsupported` , `network_unreachable_external` , `feature_disabled_by_config` . Forbids `network_unreachable_external` for any taxonomy feature with a sink-side probe.
- Bare `ab_pass` deprecated – WARN pre-grace, FAIL post-grace (- -).

Mechanical enforcement. Three pre-build gates + three paired § . meta-test mutations:

- `CM-SINK-EVIDENCE-PER-FEATURE` – walks tests for # §11.4.69 `FEATURE: <class>` annotation + verifies taxonomy probe + `ab_pass_with_evidence` use.
- `CM-NO-FAIL-OPEN-SKIP` – audits sink-side probe helpers; FAILs if any code path converts empty/unreachable response to PASS-counting SKIP for a feature class with a sink-side probe.
- `CM-AB-PASS-WITH-EVIDENCE-EVERYWHERE` – pre-grace WARN, post-grace FAIL on bare `ab_pass` calls.

Composes with § (FAIL-bluffs forbidden),
§ (recorded-evidence), § (audio +
video -layer quality), § (no-guessing),
§ (sink-side captured-evidence), §
(no-fakes-beyond-unit), § (deterministic
consistency), § (autonomous-validation),
§ (audio-specific sink-side – § is
the universal generalisation).

No escape hatch – no `--skip-evidence`, `--config-only-pass`, `--allow-fail-open-skip`, `--legacy-ab-pass-permitted` flag. The discipline exists because the – forensic incident demonstrated the failure: tests reported audio-routing PASS while the user heard nothing and the Arvus Codec-In-Use field was empty.

Propagation gate `CM-COVENANT-114-69-PROPAGATION` enforces this anchor literal across the ~ -file consumer fleet. Paired mutation strips the literal → gate FAILs.

Canonical authority: constitution submodule `Constitution.md`
§

Non-compliance is a release blocker regardless of context.

- § – Subagent-driven execution is the default (User mandate, –) [full → `CLAUDE_ANCHORS_FULL.md` + `Constitution.md`] • refs:
§ > “Always do if possible Subagent-driven! Add this into our root > (constitution Submodule) `Constitution.md`, `CLAUDE.md` and `AGENTS.md`. > This should be the default choice ALWAYS!”

When executing implementation plans authored via `superpowers:writing-plans` (or any equivalent task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development`. Inline execution via `superpowers:executing-plans` is permitted ONLY

when (a) the task is trivial AND fits in a single sub-
line edit, OR (b) the operator explicitly requests inline
execution at brainstorm-handoff time.

Subagents bring an isolated context window per task (no
conductor context bloat), a structurally separated review
seam (conductor reviews subagent output, eliminating self-
review blind spots), parallel-PWU compatibility
(`$ . . .` – subagents ARE the parallel work units), and
resumability across operator absence (subagents resume from
on-disk plan + spec inputs).

Composes with `$ . . .` (four-layer coverage), `$ . . .`
(no-guessing – subagent’s captured output IS the evidence),
`$ . . .` (iteration discipline), `$ . . .` (TDD-
fix), `$ . . .` (deterministic consistency),
`$ . . .` (LIVE_ADB_FIRST), `$ . . .` (parallel-
development PWU).

No escape hatch – `--inline-execution-required`, `--no-subagents`,
`--monolithic-execution` are NOT permitted flags. Skipping
subagent-driven for non-trivial work without recorded
operator authorisation is itself a `$ . . .` PASS-bluff. Pre-
build gate `CM-COVENANT-114-70-SUBAGENT-DEFAULT-PROPAGATION`
enforces this anchor literal across the `~`-file consumer
fleet. Paired meta-test mutation strips the literal → gate
FAILs.

Canonical authority: constitution submodule `Constitution.md`

`$ . . .`

Non-compliance is a release blocker regardless of context.

- `$ . . .` – Pre-Push Fetch + Investigate + Integrate
Mandate (User mandate, – –) [full →
`CLAUDE_ANCHORS_FULL.md` + `Constitution.md`] • refs:
`$ . . .` `$ . . .` `$ . . .` `$ . . .`
`$ . . .` `$ . . .` `$ . . .`
`$ . . .` `$ . . .` `$ . . .`
`$ . . .`

- § 11 4 73 . . – Audio Top-Priority Mandate (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .
§ 11 4 44 . 11 4 59 11 4 61
- § 11 4 65 . 11 4 60 – Main-specification document versioning + revision discipline (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:
§ 2026 05 20 . . § . . § . .
§ . . § . . 11 4 10 11 4 61
- § 11 4 65 . 11 4 60 11 4 74 – Submodule-catalogue-first discovery + extend-don't-reimplement (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .
§ 11 4 40 . 11 4 41 11 4 65 § . . § . .
- § 11 4 66 . 11 4 60 11 4 71 – Mechanical Enforcement Without Exception (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:
§ 11 4 76 . . § . . § . .
§ . . 2026 01 21 § . . § . .
§ . . § . . § . .
§ 11 4 28 . 11 4 31 11 4 36
- § 11 4 71 . 11 4 60 11 4 75 – Containers-submodule mandate (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:
§ . . § 2026 05 20 . . § . .
§ . . § . . § . .
§ 11 4 6 11 4 11 11 4 65 11 4 66
- § 11 4 71 . 11 4 60 11 4 75 – Regeneration-mechanism-required mandate (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:
§ . . § . . § . . § . .
§ . . § . . § . .
§ . . § . .

- § 11 4 30 11 4 10 11 4 65 – CodeGraph code-intelligence mandate (User mandate, 72 11 4 78 –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:
 § 11 4 79 . . § . . § . . § . .
 § . . § . . § 2026 05 21 .
 § . . § . . § . .
 § 11 4 10 11 4 74 11 4 78
- § 11 4 79 . . – Own-org submodules MUST be included in the CodeGraph index (User mandate, 11 4 80 –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:
 § . . § . . § . .
 § 11 4 10 11 4 45 11 4 53
- § 11 4 65 11 4 10 11 4 79 – CodeGraph regular-update + sync automation mandate (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:
 § . . 2026 05 21 § . . § . .
 § . . § . . § . .
 § 11 4 1 11 4 2 11 4 3 11 4 4
- § 11 4 5 11 4 11 4 23 11 4 – Cross-platform-parity mandate (User mandate, 70 11 4 81 –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:
 § . . § 2026 05 21 . . § . . § . .
 § . . § . . § . . § . .
 § . . § . . § . .
- § 11 4 24 11 4 12 11 4 43 – Iteration-speedup discipline mandate (User mandate, – –) [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:
 § 11 4 83 . . § . . § . . § . .
 § . . 2026 05 22 § . . § . .
 § . . § . . § . .
 § . . § . . § 2026 05 22 . .
 § . . – docs/qa/ end-user evidence mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“every feature that ships MUST carry a recorded end-to-end communication transcript + any attached materials under docs/qa/<run-id>/ (per-feature subdirectories). A feature with no QA transcript is itself a § PASS-bluff – it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof.”

Every feature that ships MUST carry a recorded end-to-end communication transcript plus any attached materials (screenshots, request/response payloads, audio, file uploads) committed under docs/qa/<run-id>/ – one directory per feature run. A feature with no QA transcript is itself a § PASS-bluff: it claims to work but has no auditable runtime evidence that an end user actually exercised the feature through the same interface they will use in production.

Operative rule. () Every consuming project MUST maintain a docs/qa/ tree. Each new feature lands under docs/qa/<run-id>/ where <run-id> is monotonic + greppable (timestamp, HRD-NNN, ATM-NNN, other workable-item ID per §). () Transcripts MUST be full bidirectional – every prompt/command sent + every response received. One-sided is not a transcript. () Attached materials MUST be committed in-repo (no external-only links – that is § sink-side violation). () Bot-driven / agent-driven QA automation MUST preserve the full conversation thread as the proof artefact. () CI / release gates MUST refuse to tag a version that has any feature-shipping commit without its matching docs/qa/<run-id>/ directory.

Composes with § , § , § , § , § , § .

Canonical authority: constitution submodule Constitution.md

\$. . .

Non-compliance is a release blocker. No --qa-evidence-optional escape hatch.

\$. . . – Working-tree quiescence rule
for subagent commits (User mandate,
– –)

Short tag: working-tree quiescence .

Forensic anchor – verbatim user mandate (– –):

“no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before git add , the committing agent MUST grep its own working tree for mutation markers (MUTATED for paired , // always pass , return json.Marshal shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT.”

No subagent (or main-thread) commit may proceed while any concurrent mutation gate, paired-mutation experiment, or other in-flight mutation is live in the same checkout. Before git add , the committing agent MUST grep its own working tree for mutation markers (MUTATED for paired , // always pass , return json.Marshal shortcut paths, // MUTATION / # MUTATION annotations, _mutated_* filename suffixes, etc.) and explicitly account for every modified file in the staging area. Any unexplained file → ABORT.

Lesson (forensic case study). A consuming project’s logo-fix subagent (Herald commit 72e81ab , – –) ran in a checkout where a paired \$. mutation gate had temporarily introduced an // always pass shortcut into a JWT verify path. The subagent’s git add swept the mutation residue into the same commit as the unrelated logo fix, and

the resulting commit was pushed to all four mirrors before any other agent caught it. The fix (Herald `d5bd360`, “SECURITY FIX: restore commons_auth/middleware.go JWT verify”) landed within the hour, but the window during which production-equivalent binaries shipped with a bypassed JWT verify is a real security-defect window. The lesson is now constitutional.

Operative rule. () Pre-`git add` MUST `grep` for mutation markers + cross-check `git status --porcelain` against the subagent’s declared scope; unaccounted entries → ABORT. () Any active mutation gate MUST be serialised – mutate → assert FAIL → restore → assert PASS – and the working tree MUST be verifiably clean BEFORE any unrelated commit. () Concurrent subagents in the SAME checkout MUST coordinate through a lockfile (`.git/MUTATION_IN_PROGRESS`); the cleaner solution is `git worktree add` per subagent (composes with `$ . . / $ . .`). () Post-commit `mutation-residue-scanner` MUST run before push; any commit containing a mutation marker → push BLOCKED.

Composes with `$ .` , `$ . .` , `$ . .` , `$ . .` , `$ . .` , `$ . .` , `$ . .` , `$ . .` .

Canonical authority: constitution submodule `Constitution.md`
`$ . .` .

Non-compliance is a release blocker. A mutation marker that lands in a tagged commit is a critical defect regardless of how briefly it persisted.

`$ . .` – **Stress + Chaos Test Mandate**
 (User mandate, – –)

Short tag: `stress-chaos-mandate` .

Forensic anchor – verbatim user mandate (– –):

“Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!”

Every fix or improvement landed in a consuming project MUST ship with full-automation **stress** AND **chaos** test suites that exercise edge cases, sustained load, concurrent contention, and failure-injection. Happy-path coverage alone is a `$ . / $` PASS-bluff at the resilience layer.

Stress (closed-set, mechanically auditable): sustained load ($N \geq$ iterations OR $\geq$ s wall-clock, per-iteration latency `p / p / p` recorded) + concurrent contention ($N \geq$ parallel invocations, no deadlock, no resource leak) + boundary conditions (empty / max / off-by-one input – every boundary produces a categorised result).

Chaos (closed-set, mechanically auditable, applied per fix-class appropriateness): process-death injection (kill primary or upstream mid-call, categorised recovery) + network-fault injection (drop/delay/reorder, `category=network|upstream` per `$ . .`) + input-corruption injection (corrupt `.env` / `config` / input file mid-test, detected + reported) + resource-exhaustion injection (disk full, OOM, FD exhaustion – refuse cleanly OR degrade gracefully, NEVER crash) + state-corruption injection (mid-flight lock loss, partial-write fault – recovery restores consistent state).

Anti-bluff (mandatory). Every stress + chaos test PASS MUST cite a captured-evidence artefact path per `$ . .` + `$ . .` (per-iteration `latency.json`, `categorised_errors.txt`, `state_delta_snapshot.json`, `recovery_trace.log`). Helper library `stress_chaos.sh` provides `ab_stress_run`, `ab_stress_concurrent`, `ab_chaos_kill_pid_during`,

ab_chaos_drop_network_during , ab_chaos_corrupt_file_during ,
 ab_chaos_oom_pressure_during , ab_chaos_disk_full_during , each
 composing with ab_pass_with_evidence / ab_skip_with_reason per
 § . . . Chaos-injection cleanup is non-negotiable –
 corrupt-restore, disk-fill-cleanup, process-restart MUST run
 in trap '...' EXIT ; cleanup failure = § . .
 violation.

-layer coverage per § . . (b): pre-build gate (test
 files exist + executable + sh -n + bash -n parseable; library
 exists; fix's pre-build gate cites the stress+chaos test
 path) + paired meta-test mutation per § . (strip chaos-
 injection or evidence-capture → gate FAILs) + on-device test
 (if LIVE_ADB_TESTABLE per § . . , dispatched against a
 real device with captured evidence under qa-results/<run-id>/
 stress_chaos/) + HelixQA Challenge entry (if user-visible
 feature per § . . (b) layer).

Composes with § . . / § (resilience IS end-user
 quality), § . . (FAIL-bluffs forbidden – set-u crashes
 from chaos step are bluffs), § . . (captured-evidence
 content quality applies to latency distribution + error
 categories), § . . . (no guessing – categorised errors
 only), § . . (TDD RED-first under load/chaos),
 § . . (N iterations produce identical exit +
 identical evidence-hashes), § . . (autonomous
 validation), § . . (universal sink-side positive-
 evidence taxonomy), § . . (recovery transcripts ARE
 end-user-channel proofs).

Canonical authority: constitution submodule Constitution.md
 § . . .

Non-compliance is a release blocker regardless of context. No
 escape hatch – no --skip-stress , --no-chaos , --happy-path-
 suffices , --stress-test-later flag exists.

\$. . . – Roster/corpus-backed Status-doc auto-sync mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs – any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!”

Some Status docs (\$. . .) are backed by a **tracked roster** (installed apps/components) or a **tracked asset corpus** (test/media asset directory) rather than narrative alone. Their freshness MUST NOT depend on operator vigilance – the moment a roster/corpus member changes (player app added/removed/renamed; asset added/modified/removed) the Status doc + Status_Summary + HTML + PDF MUST resync **out of the box**, mechanically.

Mechanism (all must hold): () **drift-proof fingerprint** – sha of the sorted member list (NOT mtime), persisted in a sidecar beside the Status doc; () **sync helper** that regenerates the fingerprint + re-exports HTML+PDF (via the \$. . . exporter), wired so sync is automatic; () **pre-build gate** that FAILS when the live fingerprint differs from the persisted one (mirrors \$. . . CM-ISSUES-SUMMARY-SYNC + \$. . . sync_integration_status); () **paired \$. . . mutation** that corrupts the fingerprint and asserts the gate FAILS.

Composes with \$. . . / \$. . . (roster/corpus specialisation) / \$. . . / \$. . . / \$. . . / \$. . . / \$. . . / \$ Classification: universal

(§ . .) – the consuming project supplies the specific docs, roster/corpus sources, helper, and gate name per § . . .

Canonical authority: constitution submodule Constitution.md
§ . . .

Non-compliance is a release blocker regardless of context. No escape hatch – no --skip-roster-sync , --allow-status-drift , --roster-sync-not-applicable flag exists.

§ . . – Endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing mandate (User mandate, – –)

When the operator instructs an AI agent to “continue in endless loop fully autonomously” (or any semantically-equivalent phrasing), the agent MUST treat this as a HARD-CONTRACT covenant: (A) continue working until docs/Issues.md Status-column has zero non-terminal entries AND docs/CONTINUATION.md § Active work is empty AND no background subagent is mid-execution AND no external dependency is in-flight; (B) dispatch background subagents for parallelisable work – main + every subagent operate concurrently, “waiting for results” is the ONLY acceptable idle reason; (C) every closure lands four-layer test coverage per § . . (b) with captured-evidence (audio/video/network/UI/sysfs – “physical proofs”); (D) the § . anti-bluff covenant family (§ . . /§ . . /§ . . /§ . . / § . . /§ . . /§ . . /§ . . / § . . /§ . .) is the operative truth-discipline – tests AND HelixQA Challenges bound equally per the historical forensic anchor “we had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and

can't be used"; (E) loop terminates ONLY on all-conditions-
met, explicit operator STOP, host-session-safety demand, or
scheduled wake on a known-future-actionable signal.

[illegible]

CM-COVENANT-114-87-PROPAGATION + paired § . meta-test
mutation.

Canonical authority: constitution submodule Constitution.md

Non-compliance is a release blocker regardless of context. No escape hatch — no `--idle-OK` , `--skip-endless-loop` , `--bluff-permitted-for-this-task` , `--metadata-only-test-suffices` , `--no-physical-proof-required` flag exists.

```
$ . . - Background-push mandate:
commit-lock release immediately after commit,
push runs detached (User mandate,
- - )
```

Forensic anchor: - - a single commit_all.sh held its flock ~ hours because do_push ran synchronously after the commit landed – every subsequent commit was blocked on a slow mirror push that had nothing to do with the local commit’s durability. Implementation seam for § . . (B) zero-idle.

The mandate: (A) `.git/.commit_all.lock` MUST be released IMMEDIATELY after `git commit` returns – the commit is durable on local disk regardless of remote push outcome; (B) push runs detached via `nohup ./push_all.sh ... > <log> 2>&1 & + disown` – orchestrator's exit code reports COMMIT success, NOT

push success; (C) `push_all.sh` acquires per-remote flock `.git/.push.<remote>.lock` so concurrent invocations targeting same remote serialize but different-remote invocations run in parallel; (D) backgrounded push failures land in `..qa-results/push_failures/<ts>_<remote>.log` – next autonomous-loop tick checks per `$ . .` (A) “no external dependency in-flight” gate; (E) synchronous-push escape: explicit `--sync-push` CLI flag preserves legacy behaviour for `$ . .` force-push merge-first audit paths.

Composes with `$ . / $ . / $ . . / $ . . / $ . .`. Pre-build gates `CM-COVENANT-114-88-PROPAGATION` + `CM-BACKGROUND-PUSH-WIRED` + paired `$ .` meta-test mutations.

Canonical authority: constitution submodule `Constitution.md`
`$ . . .`

Non-compliance is a release blocker. Synchronous push (without `--sync-push`) = `$ .` PASS-bluff at execution layer. No escape hatch beyond `--sync-push` for force-push events.

`$ . .` – Background test execution mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):
 “Any tests we are executing, especially long test cycles, MUST BE performed in background in parallel with main work stream! This MUST NOT block our capabilities to work on queued workable items. Main work stream can be blocked or sit iddle only if absolutely needed and if it depends hard on results of some background execution.”

Forensic incident: - - conductor invoked `pre_build_verification.sh` synchronously with foreground `timeout 360`, blocking main stream for - minutes on \$JV/\$JW/\$JX/\$JY scaffolding work. Symmetric anchor to \$. . (background push) at the test-execution layer.

Mandate: (A) Long-running tests (> s expected: `pre_build`, `meta_test`, `test_all_fixes`, `recent_work_validate`, `HelixQA` banks, -phase cycles, full-suite retests, `audio_loop_supervisor`, `dual_display_record`) MUST run via `nohup ... > <log> 2>&1 & + disown` with log placed under known dir (`qa-results/<test_id>_<ts>.log`); (B) Main stream proceeds to \$. . priority queue immediately; (C) Hard-dependency gating: poll exit-status file or `pgrep -af <test>` before steps that need exit code – surface as \$. . interactive options if test still running; (D) Failures land in `<log>` files, next loop tick checks; (E) Foreground execution permitted ONLY for < s tests OR explicit operator authorisation; (F) Per-script flock serialises same-script invocations, different-script invocations parallel.

Composes with \$. . / \$. . /
 \$. . / \$. . / \$. . /
 \$. . / \$. . . Pre-build gates CM-
 COVENANT-114-89-PROPAGATION + CM-BACKGROUND-TEST-EXECUTION-WIRED +
 paired \$. meta-test mutations.

Canonical authority: constitution submodule `Constitution.md`
 \$. . .

Non-compliance is a release blocker. No escape hatch beyond explicit per-invocation operator authorisation.

**\$. . . – Obsolete status + per-item
obsolescence audit (User mandate,
– –)**

Forensic anchor – verbatim user mandate (– –):
 “Bug No – Albums cover etc. seems obsolete after latest
 request for new behavior when audio and video content is
 being played ... mark obsolete tickets with some light gray
 background ... text – the description to be strikethrough
 styled ... review all existing open or resolved workable items
 if they are obsolete – not valid any more ... There MUST NOT be
 any mistake! No bluff is allowed of any kind!”

\$. . . Status closed-set extended with terminal
 Obsolete (→ Fixed.md) value (orthogonal to Type per
 \$. . .). Obsolescence reasons (closed vocabulary):
 superseded-by-design-change | superseded-by-later-mandate | feature-
 removed | duplicate-of | unsupported-topology | not-reproducible
 (not-reproducible = a reported defect that does NOT reproduce
 on the canonical tree/baseline – an environment/isolated-
 worktree artifact, not a real product defect; triple-check
 evidence captures the canonical-tree non-reproduction). Every
 Obsolete heading MUST carry ****Obsolete-Details:**** line (Since
 + Reason + Superseding-item + Triple-check evidence) within
 non-blank lines. \$. . . colorizer adds cell-
 status-obsolete class – light-gray #E0E0E0 background +
 strikethrough description. Audit cadence: every release-gate
 sweep per \$. . . + \$. . . Triple-check non-
 negotiable per operator mandate. Composes \$. . . /
 \$. . . / \$. . . / \$. . . /
 \$. . . / \$. . . / \$. . . /
 \$. . . / \$. . . / \$. . . /
 \$. . . Pre-build gates CM-COVENANT-114-90-PROPAGATION +
 CM-ITEM-OBSOLETE-DETAILS + CM-OBSOLETE-COLORIZER-WIRED + paired
 \$. . . mutations.

Canonical authority: constitution submodule Constitution.md
 \$. . . Non-compliance is a release blocker.

§ . . - Summary-doc clarity mandate
(User mandate, - -)

Forensic anchor – verbatim user mandate (–):

“Summary docs – Issues_Summary some not clear one line descriptions – like ‘Composes with’ ... For each workable item we MUST HAVE clearly understandable meaning ... every team member can clearly understand what that particular workable item is exactly about! There cannot be misunderstanding or unclarity of any kind and no bluff allowed!”

Every summary entry (Issues_Summary, Fixed_Summary, README doc-link, Status_Summary page + , all one-liners) MUST contain self-contained meaningful description ≥ words OR ≥ chars naming SUBJECT + PROBLEM/GOAL. Forbidden one-liner anti-patterns: section labels (Composes with , Closure criteria , Fix direction , etc.); bare metadata fragments (Critical , Bug , In progress , etc.); section-marker echoes; \$-letter alone. Generators (generate_issues_summary.sh / generate_fixed_summary.sh / update_readme_doc_links.sh / generate_status_summary.sh) MUST extract from H /H heading line per \$. . ATM-NNN convention, NEVER from arbitrary downstream text. Generators refuse anti-pattern rows – emit (MISSING_DESCRIPTION – fix source heading) placeholder with visual highlight. Pre-build gate CM-SUMMARY-CLARITY-DESCRIPTIONS.. scans every summary; anti-pattern match = FAIL. Audit cadence: every \$. . + \$. . sweep. Composes \$. . / \$. . / \$. . / \$. . / \$. . / \$. . / \$. . /

```
Canonical authority: constitution submodule Constitution.md
$ . . . Non-compliance is a release blocker.
```

Every non-trivial change MUST pass -pass evaluation BEFORE commit-ready: (**Pass**) Main-task verification – change achieves stated goal, captured-evidence per \$. . / \$. . ; (**Pass**) Regression-blast-radius analysis – enumerate every direct dependency, demonstrate no contract break; (**Pass**) Cross-feature interaction analysis – parallel features sharing state/timing/hardware/shell environment audited; (**Pass**) Deep-research validation per \$_{11 A 08 . 11 A 02 11 A 06} – external precedent OR “NO external solution found – original work” + CodeGraph queries per \$. . / \$. . ; (**Pass**) Anti-bluff confirmation per \$. /\$. . /\$. . /\$. . /\$. . /\$. . /\$. . /\$. . – no new bluff surface introduced. Documentation per pass (commit footers OR docs/ entries OR qa-results/ evidence). Only AFTER all passes complete may commit/push/test/release proceed.. Trivial exemption: typo / revision-bump / MD-export-regen IF zero source touched AND commit message cites exemption explicitly. Composes \$. . / \$_{11 A 02 . 11 A 03} / \$_{11 A 05} . . / \$. . / \$_{11 A 07 . 11 A 09} / \$. . / \$. . / \$. . / \$. . / \$. . / \$. . / \$. . Pre-build gates CM-COVENANT-114-92-PROPAGATION + CM-MULTI-PASS-EVALUATION-EVIDENCE + paired \$ mutations.

11 4 93
Canonical authority: constitution submodule Constitution.md

§ . . . Non-compliance is a release blocker.

2024 05 27
§ . . . – SQLite-backed single-source-of-truth for workable items (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):
“There MUST be single source of truth for all of our workable items – SQLite database ... proper scripts (we recommend Go programs) ... reduce a chance for sync to be broken ... generate always all docs from DB or to re-generate Db from all docs we have in opposite direction”.

11 4 77
Text-based Issues/Fixed/Summary/CONTINUATION constellation converted to SQLite-DB-backed single source of truth. DB at docs/.workable_items.db (gitignored per § . . . + § . . . regen mechanism). Schema mandatory tables: items (atm_id PK + Type + Status incl. Obsolete + Severity + title + description ≥ chars. + created/modified + composes_with JSON + current_location); item_history (append-only audit per § . . . By/Reason/Evidence); obsolete_details (§ . . .); operator_block_details (§ . . .); firebase_metadata (§ . . .); meta (schema version + last sync + integrity hash). Go binary at cmd/workable-items/ : sync md-to-db / db-to-md / diff / validate / add / close. Bidirectional regen byte-identical round-trip (closed-set tolerance for whitespace/section-order). commit_all.sh refuses on diff non-empty. sync_issues_docs.sh invokes Go binary. pre_build runs workable-items validate . Anti-bluff: unit + integration + stress (–row insert + concurrent writers) + chaos (mid-write SIGKILL + corrupt-DB recovery + disk-full) + paired § . . . mutation + HelixQA Challenge CME-WORKABLE-ITEMS- . . . -phase migration: \$LA Issues entry → Go binary scaffold + DDL → md-to-db → db-to-md round-trip CI → generator shims → text-direct edits prohibited. Cross-

queued item genuinely blocked on external dep (hardware / network upstream / build/test completion conductor cannot accelerate) OR operator STOP OR § host-safety; “don’t see what to do” is NEVER valid; (B) before ANY wake/sleep MUST survey parallel-work feasibility per § . . . + § . . . + § . . . , identify non-contending items, dispatch in parallel per § . . . / § . . . (subagent) + § . . . (PWU disjoint scope) + § . . . (background long tests); (C) priority order MANDATORY – pick highest-severity+§ . . . audio-first the conductor can autonomously progress; (D) subagent-driven default for non-trivial; (E) background default for > s wall-clock work via nohup+disown; (F) stability-preserving – composes with § . . . multi-pass + § . . . quiescence + § . . . -§ . . . host safety; (G) progress updates surfaced when operator asks OR at milestone boundaries. Anti-pattern forbidden: scheduling wake without queue survey; “nothing to do – sleeping” with non-trivial items progressable; serialising parallelisable work; picking lower priority over higher. Pre-build gates CM-COVENANT-114-94-PROPAGATION + CM-PARALLEL-WORK-AUDIT + paired § . . . mutations.

Canonical authority: constitution submodule Constitution.md § Non-compliance is a release blocker.

§ . . . – Workable-items SQLite DB
TRACKED in git, NEVER gitignored (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):
 “We shall not Git ignore our workable items SQLite DB since it is our single source of truth ... workable items SQLite DB regularly committed and pushed to all upstreams!”

§ . . . ’s earlier “gitignored per § . . . ”
 clause is AMENDED – the DB at docs/workable_items.db is TRACKED in git, NEVER gitignored. It IS authoritative source

data, NOT a build artefact. Every `workable-items sync md-to-db` that mutates state MUST stage+commit+push the DB alongside the MD regen per `$ . . atomic-move + $ . multi-upstream push`. WAL-checkpoint (`PRAGMA wal_checkpoint(TRUNCATE)`) required before commit-stage so transient `.db-wal` + `.db-shm` sidecars (gitignored per `$ . .`) are safely discardable. `$ . .` regeneration mechanism does NOT apply – DB IS the source. Destructive DB ops require `$ .` hardlinked-backup + operator authorization; `$ . .` force-push merge-first applies if DB history ever needs `11 4 15` rewrite. Pre-build gates `CM-COVENANT-114-95-PROPAGATION` + `CM-WORKABLE-ITEMS-DB-TRACKED` + paired `$ .` mutation.

`11 4 15`
Canonical authority: constitution submodule `Constitution.md`
`$ . .` . Non-compliance is a release blocker.
`2024 05 27`

`$ . .` – Safe-parallel-work-with-long-build catalogue + mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):
 “Are there except AOSP build process any other active jobs being done at the moment? Can we work on something in parallel while build is in progress so we slowly cleanup our slate? ... If yes, and that was not already clear by our root Constitution, we should add all mandatory details into it ... do as much as possible work in background in parallel with main work stream and oreferably using subagents-driven approach!”

Operational catalogue for the canonical long-running workload (– h AOSP containerised build per `$ .`):

SAFE during build: (A) MD/docs work under docs/+README+CLAUDE/AGENTS/QWEN+Status+Issues/Fixed+changelogs+research notes; (B) generator/helper script work under scripts/ scripts/testing/ scripts/llm/ scripts/firebase/; (C) pre-build + meta-test gate authoring + paired

§ . mutations; (D) on-device test scripts under tests/
 test_...sh; (E) constitution submodule edits + push to
 remotes; (F) any submodule commit + push per § . . . ;
 (G) D /D read-only live-ADB probes (dumpsys/getprop/cat /
 proc/asound/screencap/logcat); (H) subagent dispatch per
 § . . . /\$. . . + \$. . . quiescence;
 (I) web research + external API queries with § . .
 credentials; (J) workable-items DB ops per
 § . . . + \$. . . ; (K) pre-build + meta-test
 execution backgrounded per §

UNSAFE during build: (α) git checkout/reset -hard/clean -df
 on source tree (use git worktree instead); (β) mass file
 deletes/renames under device/+frameworks/+hardware/+vendor/
 +kernel- . . . /+external/+packages/+bionic/+bootable/+build/
 +system/+cts/+tools/+prebuilts/; (γ) submodule pointer
 updates affecting built APKs (defer to between-build
 windows); (δ) out/ directory mutations; (ε) make clean/m
 clobber/rm -rf out/; (ζ) container destruction; (η) disk-
 filling operations breaching § . . free-space minimum;
 (θ) § . . . host-session-safety breaches.

Conductor responsibility: before EVERY pause point during
 long build, consult catalogue, identify (A)-(K) queue items
 per § . . . + \$. . . , dispatch ≥ per
 § . . . /\$. . . subagent default +
 § . . . background. “Build running, nothing else to
 do” NEVER true per § . . . + \$ Pre-build
 gates CM-COVENANT-114-96-PROPAGATION + CM-PARALLEL-WORK-DURING-
 BUILD-AUDIT + paired § . . . mutations.

Canonical authority: constitution submodule Constitution.md
 § Non-compliance is a release blocker.

§ . . – Full-Automation Anti-Bluff Mandate – Live tests MUST be re-runnable end-to-end without manual intervention (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):
 “Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! ... Make sure there is no false positives in testing! Every test and its results MUST obtain real proofs of everything working! No bluff is allowed!”

Composes with § . + § . . + § . . +
 § . . + § . . + § . . +
 § . . + § . . – closes the **manual-intervention gap** they did not explicitly forbid. A live/integration/e e/Challenge test requiring a human action during execution (typing a chat message, clicking a UI, hand-triggering a webhook, anything beyond test start + PASS/FAIL report) is **by definition a § . PASS-bluff at the automation layer**, regardless of how thorough the manual run is – cannot run continuously in CI, cannot validate regressions between manual runs, human dependency masks drift.

- . Binding rule: every test this Constitution governs – unit/integration/e e/Challenge/stress/chaos/live – MUST be fully self-driving end-to-end; reports PASS/FAIL/SKIP-with-reason without any further human action after startup. (B) Single permissible exception: one-time credential bootstrap OUTSIDE test execution (`.env` from vault, shell exports in `~/.bashrc` , OAuth approval at first install, MTProto session activation at first run) – configuration, not test driving. (C) Concrete requirements for live messenger/channel/agent tests: () no “operator MUST type a message”

prompts – drive programmatically. (MTPROTO for Telegram, real-user-token API for Slack, IMAP-test-account for email, webhook fixture, in-process loopback – never human keystrokes); () no hard-coded session UUIDs that collide with active dev session (Herald – – lesson: `claude --resume <UUID>` on same UUID as dev session returns silent exit –); () no s human-response windows (\$. . determinism violation); () re-runability proof – PASS at `-count=3` consecutive automated invocations with self-cleaning state; () \$. . obsolescence audit – every existing test classified COMPLIANT vs NON-COMPLIANT; () no false-positive PASS – silent-skip-reported-as-PASS forbidden, stale-evidence forbidden, SKIP-with-reason per \$. . is correct. (D) Composes with \$. . + \$. . + \$. . + \$. . – together = continuously-validated, fully-automated, non-flake, anti-bluff regime. (E) Inheritance per \$. . – every consuming repo’s CLAUDE.md/AGENTS.md/QWEN.md restates citing literal anchor 11.4.98 ; pre-build gate `CM-COVENANT-114-98-PROPAGATION` enforces literal presence; paired \$. mutations strip → gates FAIL. (F) Enforcement: commit adding manual-action test BLOCKED at release-gate; manual-dependency test not rewritten within “ ” days graduates to \$. . Obsolete citing \$. . as obsolescence reason.

Canonical authority: constitution submodule `Constitution.md`
 \$. . . Non-compliance is a release blocker.

§ 4.05.2. – Latest-Source Documentation Cross-Reference Mandate – instructions/guides/manuals MUST be verified against latest official online sources BEFORE publication (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):
“Make sure we ALWAYS check against latest versions of services we use web / online docs before creating instructions! This situation is illustration of how we can misguide ourselves or get banned! ... These are mandatory rules / constraints and the result is consistency and safety of created instructions, guides and manuals!”

Case study (Herald – –): first-draft Herald MTProto setup guide recommended VoIP/Google-Voice/Twilio fallback AND omitted the `recover@telegram.org` pre-login email – both directly contradicted Telegram’s official docs + gotd/td maintainer guidance, and following the guide could have caused a permanent Telegram account ban. Misguidance-by-stale-docs is the same severity class as a § PASS-bluff at the documentation layer.

Composes with § + § + § + § Pass + § + § . Closes the gap § Pass alludes to but does not explicitly mandate. (A) Pre-commit cross-reference: every operator-facing instruction document MUST () fetch the LATEST official online docs of the service/library via WebFetch/MCP/direct browsing (NEVER training data, memory, or prior committed docs); () cross-reference each instruction against the source; () seek secondary authoritative sources when the official docs are sparse/silent (library maintainer SUPPORT.md, official changelogs, vetted community FAQs); () cite source URL + date in a “ Sources verified” footer on the doc; () cite the cross-reference in the commit-message footer (Sources verified <date>: <urls>). (B) Negative findings: gaps/silences/contradictions MUST be documented

explicitly so the next reader doesn't assume absence-of-contradiction is authoritative agreement. (C) Re-verification cadence: documents older than months are STALE – re-verify before citing as operator authority, at every vN. . release boundary, on service breaking-change announcements, or when an operator reports an error following the guide. (D) Risk-classified services (Telegram/WhatsApp messengers, cloud APIs, payment systems, AI/LLM providers, code-hosting services, package managers) – -day max staleness; explicit safety warnings cited against latest policies. (E) Composes with § . . Pass . . but is INDEPENDENT – cannot substitute. (F) Inheritance per § . . – restate literal anchor 11.4.99 in every consumer's CLAUDE/AGENTS/QWEN; pre-build gate CM-COVENANT-114-99-PROPAGATION (when implemented) enforces; paired § . mutations strip → gates FAIL. (G) Enforcement: commit without “Sources verified” doc-footer AND commit-message-footer BLOCKED at release-gate; stale-beyond-grace docs graduate to § . . Obsolete with Reason=stale-documentation .

Canonical authority: constitution submodule Constitution.md
§ . . . Non-compliance is a release blocker.

- § . . . – RETIRED. Demoted to consumer project (a consuming project's video-color/visual-quality fidelity... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] . refs: § . . § . . § . . § . . § . . – Autonomous-decision-over-blocking mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven’t answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!”

When operating in an autonomous / endless-loop mode (per § 1.1.1), the agent MUST minimize operator-blocking and instead make the safe, reliable, reversible decision itself – so work is NOT stalled (e.g. overnight) waiting for input. § 1.1.2 says keep working; § 1.1.3 says HOW to clear the decision points that would otherwise force a stop-and-wait.

Decision rule (closed-set – proceed autonomously when ALL hold): (a) the action is reversible OR has a captured pre-op backup per § 1.1.1; (b) the agent can determine the safe choice from captured evidence per § 1.1.2 (no guessing – **LIKELY** is not a determination); (c) a wrong choice’s blast radius is bounded AND recoverable; (d) it composes with anti-bluff § 1.1.4, host-safety § 1.1.5, data-safety § 1.1.6.

Block-only-when rule (BLOCK via § 1.1.7 ONLY when ALL hold): the action is irreversible AND high-blast-radius AND the safe choice cannot be determined from evidence – e.g. external-account state the agent cannot inspect, hardware it cannot access, destructive ops without backup, force-push (also § 1.1.8 + § 1.1.9), spending / sending to third parties. **Operator-blocked** per § 1.1.10 is reached only after this rule fires AND the self-resolution-exhaustion audit completes.

“Make sure that we ALWAYS trigger / start the”/ superpowers:systematic-debugging” skills when any issues happen! If this is possible to activate and use in this situations out of the box when we spot problems / issues / bugs / misalignments / inconsistencies we MUST activate the skill(s) and make strongest efforts in full in depth analysis / debugging and determine root causes of all problem or obtain relevant data and information we need! ... we MUST make sure that “/using-superpowers” skill is ALWAYS loaded, applied and used! All dependencies (plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!”

Three cooperating invariants – the difference between guess-and-retry and investigate-to-root-cause-first.

(A) Mandatory systematic-debugging activation. On ANY spotted issue / bug / test failure / gate failure / regression / misalignment / inconsistency / unexpected behaviour, the agent MUST activate `superpowers:systematic-debugging` (or the platform-equivalent structured-debugging discipline) **BEFORE proposing, writing, or applying any fix** – the **Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST**. Full four-phase arc: root-cause (read the real failure, reproduce, gather facts) → pattern (classify the defect, search for the same pattern elsewhere) → hypothesis (form a falsifiable cause, prove/disprove with captured evidence) → implementation (design the fix only against the proven root cause). Guess-and-retry, symptom-patching, and re-running a failed test hoping it passes (“probably transient / flaky”) **WITHOUT** a completed investigation are § . . . violations. Real-world signal: calling a failure `transient` / `flaky` / `intermittent` / `probably-timing` without captured forensic evidence is simultaneously a § . . . (no-guessing) and

§ . . (demotion-evidence) violation – § . . makes the corrective response mechanical (the classification is forbidden until the systematic-debugging arc proves it).

(B) Mandatory always-loaded using-superpowers .

superpowers:using-superpowers (or the platform-equivalent skill-discovery / capability-index discipline) MUST be loaded and applied at session start and consulted before any task. Operative rule: before acting on ANY request, survey available skills; if ANY skill could apply – even at % relevance – it MUST be invoked rather than improvised from memory. Skipping skill-discovery at session start, or improvising a task a loaded skill exists for, is a § . . violation.

(C) Mandatory plugin / dependency availability. Every skill plugin / marketplace package / capability dependency the project relies on (Claude Code, its skill marketplaces, or any other runtime's plugin ecosystem) MUST be installed + loadable BEFORE the dependent work proceeds. A missing plugin that blocks a mandated skill (e.g. superpowers absent so systematic-debugging cannot launch) is a **release-blocker** until installed + confirmed loadable. Install mechanism: the runtime's own plugin/marketplace install path – for Claude Code the in-session /plugin marketplace flow (add marketplace → install plugin → confirm the skill appears in the available-skills list); for other runtimes the documented package/extension installer. Anti-bluff: confirm by observing the skill in the ^{11 4 6}live capability list, never assume install succeeded (install exit ^{11 4 6} ≠ skill loadable, per the § . . lesson).

Composes with § . . (test-interrupt – first action after STOP is launch systematic-debugging), § . . (no-guessing – (A) is the procedural enforcement producing the facts § . . demands), § . . (demotion-evidence – the arc captures same-conditions evidence), § . . (deep-web-research – inside the hypothesis phase),

§ . . (TDD-fix – RED test written against proven root cause), § . . (subagent-driven – the investigation is a natural subagent dispatch), § . . (A) (generalises Phase- -forensic-before-speculative-patch to ANY fix + adds skill-discovery + plugin-availability), § . . (feeds Pass + Pass).
 Classification: universal (§ . .). Propagation gate CM-COVENANT-114-102-PROPAGATION (literal 11.4.102 across consumer fleet) + paired § . meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch – no --skip-systematic-debugging , --guess-and-retry-OK , --symptom-patch-permitted , --skip-skill-discovery , --plugin-optional , --missing-plugin-is-warning flag.

Canonical authority: constitution submodule Constitution.md

§

Non-compliance is a release blocker regardless of context.

§ . . – Continuous parallel-stream working routine (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“Do this working approach continuously and make it part of regular working routine and add it to the Constitution Submodule documented fully.”

Promotes the proven multi-stream operating pattern into the project’s **standing default working routine** (not a per-request opt-in). Binds § . . /§ . . /§ . . / § . . /§ . . and adds the load-bearing invariant: **the main work stream MUST always stay FREE.**

(A) **Main stream stays FREE.** ALL commit AND push operations run detached (`nohup ... & + disown` , per \$. . . commit-lock-release-immediately + detached-push) – the main stream returns to the priority queue the moment the local commit is durable, never blocking on a push or a slow mirror. (B) **≥ parallel background streams at all times + auto-backfill** (User mandate – – ; raised from ≥) – run at **least three** subagent-driven background streams (per \$. . . /\$. . . , isolated per \$. . . PWU worktree + \$. . . quiescence) alongside the main stream whenever three-plus non-contending actionable items exist; **the moment any one stream is FULLY done, a new stream MUST immediately start and take its place** (claim next-highest-priority non-contending item), so the active-stream count NEVER drops below while actionable items remain. Idle below is permitted ONLY when no remaining non-contending actionable items OR all remaining are externally blocked (\$. . . /\$. . . /\$. . .). Standing band – , bounded above by \$. . . % memory + \$. . . -agent cap. (C) **Most-critical + most-visible first; audio always top** per \$. . . + \$. . . priority order. (D) **Safe-during-build scope only** – while the GB containerised AOSP rebuild (\$. . .) or any heavy `gradle / m -j` build runs, streams restrict to the \$. . . SAFE catalogue (investigation/forensic/docs/test-authoring/gate-authoring/read-only probes/submodule edits/research/DB ops/backgrounded pre-build+meta-test); NEVER a second concurrent heavy build (\$. . .). (E) **Heavy anti-bluff on every closure** – root cause proven or UNCONFIRMED: / UNKNOWN: / PENDING_FORENSICS: per \$. . . , captured evidence per \$. . . +\$. . . , deterministic consistency per \$. . . , paired \$. . . mutations, \$. . . systematic-debugging on any spotted problem. (F) **Idle ONLY when genuinely externally blocked** (hardware/network upstream/in-flight build-test-push completion) OR operator STOP OR \$. . . host-safety, per

§ . . (A)+§ . . ; “nothing visible to do” with
 progressable items is NEVER valid; a block parks one work
 unit, not the loop (§ . .).

Composes with § . . / § . . /
 § . . / § . . / § . . /
 § . . / § . . / § . . /
 § . . / § . . / § . . /
 § . . / § . . / § . . / § . . /
 § . . / § . . / § . . Classification: universal
 (§ . .). Propagation gate CM-COVENANT-114-103-
 PROPAGATION (literal 11.4.103 across consumer fleet) + paired
 § . . meta-test mutation (strip literal → gate FAILs; gate-
 code = separate work item). No escape hatch – no --block-main-
 stream , --synchronous-commit , --synchronous-push , --single-
 stream-only , --skip-parallel-streams , --serialise-actionable-
 work , --idle-without-queue-survey flag.

Canonical authority: constitution submodule Constitution.md

§ 11 4 104 . .

Non-compliance is a release blocker regardless of context.

§ . . – Participant identity,
 attribution & notification-tagging (User
 mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“Every supported messenger must relate messages to
 participants (Subscribers/Users); the same logical person
 may have a different username on every messenger.
 Workable items must carry who created them and who they
 are assigned to. Notifications must @-tag the right
 participant – but never the operator (who drives the
 system) and never the system agent.”

MANDATORY for every consumer that ships a messenger/
notification surface (Herald + flavor binaries are the
reference impl; others inherit per \$. .). Detailed
spec: Herald docs/design/PARTICIPANT_ATTRIBUTION.md – restated,
not redefined. (A) **Participant identity**. Every messenger MUST
relate messages to a **Participant** (logical Subscriber/User);
the SAME person MAY have a DIFFERENT username per messenger –
modelled as a logical subscriber (canonical messenger-neutral
handle , kind ∈ {human,agent,service}) + per-channel aliases
(channel , channel_user_id , the @username used for tagging).
Canonical handle closed set: Claude (reserved system-agent
sentinel; never tagged) OR a subscriber @username . (B)

Operator = env var, not a DB flag – the one human who drives
via the agent CLI, designated by

HERALD_<CHANNEL>_OPERATOR_USERNAME

(e.g. HERALD_TGRAM_OPERATOR_USERNAME); a normal Participant
whose handle equals that value. (C) **Workable items MUST carry**
created_by + **assigned_to** (canonical handles): CLI-
prompt→Operator; system/agent-detected→ Claude ; received-
message→sender's resolved @username . assigned_to defaults to
Operator, overridable. Legacy items carry "" and MUST still
parse + validate. (D) **Tagging matrix**: tag assigned_to if it
is a human ≠ Operator; tag created_by if it is a human ≠
Operator ≠ Claude ; NEVER tag Claude (system) or the Operator
(no self-ping); de-dup; resolve each handle to the channel
@username , skip if not on that channel. (E) **Anti-bluff**
(composes \$. .): real SQLite round-trip with the new
columns byte-identical (incl. legacy fixtures WITHOUT the
fields); tagging matrix proven by a truth-table + a cell-flip
mutation forcing FAIL; E E real-event → real-message
asserting the exact @username s + a NEGATIVE case proving the
Operator is NOT tagged; evidence under docs/qa/<run-id>/ .

Composes with \$. . + \$\$. . (anti-
bluff covenant) / \$. . / \$. . /
\$. . / \$. . / \$. . /
\$. . / \$. . Classification: universal

§ 11.4.104 – projects with no messenger surface inherit it latently (binds the moment they ship one, per the § 11.4.104 pattern). Propagation gate CM-COVENANT-114-104-PROPAGATION (literal 11.4.104 across consumer fleet) + paired § 11.4.104 meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch – no --skip-attribution, --no-participant-tagging, --tag-operator-anyway, --attribution-later flag.

Canonical authority: constitution submodule Constitution.md § 11.4.104; detailed spec Herald docs/design/PARTICIPANT_ATTRIBUTION.md.

Non-compliance is a release blocker.

§ 11.4.105 – Natural-language intent recognition & clarification (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“Users must NOT need to know command syntax (no COMMAND: ... prefix). They send a clear natural-language message; the System determines the intent. The System recognizes the commands it has; if none matches it infers the exact intent; if it is totally unable it replies, tags the user (@user ...), and asks to clarify precisely. We MUST always do our best to determine exact intent so we never annoy end users. This is a CORE part of the System.”

MANDATORY for every consumer that ships a messenger/command surface (Herald + flavor binaries are the reference impl; others inherit per § 11.4.105). Detailed spec: Herald docs/design/INTENT_RECOGNITION.md – restated, not redefined. **(A) No required command syntax.** Users MUST NOT be required to know any command syntax (no COMMAND: prefix); they send plain natural language and the System determines the intent. **(B)**

Three-tier resolution (first that succeeds wins): TIER₁ – recognize the System’s existing command set from natural language → action (confident deterministic match); TIER₂ – when no command matches, infer the exact intent (LLM dispatch maps language → action), NEVER guessing; TIER₃ – when neither a command nor a confident intent can be determined, REPLY to the message, TAG the sender (@username , resolved via the \$. . . IdentityResolver) and ask a PRECISE clarifying question NAMING the candidate intents – no guessing, no silent drop. (C) **Never guess, never drop:** a wrong action is worse than a clarifying question (composes \$. . . no-guessing); a message is NEVER silently dropped; only genuine ambiguity reaches Tier₃ , which always replies-tags-and-asks. (D) **Anti-bluff (composes \$. . .):** every tier ships unit + integration + E E + full-automation tests with real captured evidence – Tier₁ truth-table (natural-language → action+fields, plus conservative negatives that MUST fall through to “no match”); Tier₂ E E whose recorded reply body is EXACTLY @<sender> <specific question> + a NEGATIVE proving a clear command does NOT trigger clarify; a paired \$. . . mutation breaking the confidence guard (false-match) OR dropping the clarify tag MUST FAIL a test; evidence under docs/qa/<run-id>/ .

Composes with \$. . . + \$\$. . . (anti-bluff covenant) / \$. . . (no-guessing) /

\$. . . (clarify reply tags the sender) /

\$. . . / \$. . . / \$. . . / \$. . .

Classification: universal (\$. . .) – projects with no messenger surface inherit it latently (binds the moment they ship one, per the \$. . . pattern). Propagation gate

CM-COVENANT-114-105-PROPAGATION (literal 11.4.105 across consumer fleet) + paired \$. . . meta-test mutation (strip literal → gate FAILs; gate-code = separate work item). No escape hatch – no --require-command-syntax , --guess-intent-ok , --skip-clarify , --drop-on-ambiguous flag.

Canonical authority: constitution submodule Constitution.md
\$. . ; detailed spec Herald docs/design/
INTENT-RECOGNITION.md .

Non-compliance is a release blocker.

\$. . – Docs Chain – mechanical
documentation/DB sync engine (Operator mandate,
– –)

Forensic anchor – operator mandate (– –): Docs Chain is the canonical mechanical enforcer of the documentation-sync mandates; consumers MUST use the engine instead of ad-hoc per-project doc-sync scripts, register their chains via per-context YAML, and never accept a faked transform.

MANDATORY for every consumer. Docs Chain (the vasic-digital/docs_chain engine – a universal Go bidirectional document-and-database dependency-propagation engine) is the canonical mechanical enforcer of the documentation-sync mandates.

Detailed spec: the engine docs ~/Projects/docs_chain → docs/CONSTITUTION_INTEGRATION.md (distribution + inheritance + anchor mapping table) + docs/USE_CASE_CATALOGUE.md (chain recipes) – restated, not redefined. (A) Use the engine, never ad-hoc scripts – consumed by reference (the flat-layout sibling ~/Projects/docs_chain / the constitution-exposed path), inherited like \$. . 's codegraph_* scripts: referenced, NEVER copied; ad-hoc sync_* / generate_*_summary /

update_readme_doc_links scripts are superseded and retired per registered context. (B) Consumer-owned contexts – the engine is project-agnostic; the consumer registers its chains as data via ..docs_chain/contexts/*.yaml (\$. .

decoupling); state.json + *.docs_chain.tmp are gitignored.

(C) Anchors it mechanizes (per the CONSTITUTION_INTEGRATION mapping table): \$. . / \$. . /

\$. . / \$. . / \$. . /

\$. . / \$. . / \$. . /

\$. . / \$ / \$. . /
\$. / \$. . . - with content-hash change
detection (NOT mtime, \$. .), atomic-rename + SQLite-
txn commit + rollback (\$.), both-dirty sync → conflict-
not-silent-merge (\$. .), verify as the deterministic
CI/pre-build gate (\$. .), per-run captured evidence
to qa-results/docs_chain/<run-id>/ (\$. .). (D) NOT a
replacement for authoring discipline – the source author
still writes the \$. . revision header; the engine
only keeps exports in sync. (E) **Anti-bluff (composes**
\$.): the engine NEVER fakes a transform – a missing
pandoc/weasyprint surfaces a typed ToolAbsentError + honest
\$. SKIP-with-reason, never a fake PASS / partial
write; every sync / verify carries real captured evidence.
Composes with \$. . + \$\$. . (anti-
bluff covenant) / \$. . (no-guessing – conflict not
silent merge). / \$. . (engine/context decoupling) /
\$. . (inherited-by-reference, never copied) /
\$. / \$. . / \$. . / \$. . /
\$. , plus the sync anchors it mechanizes.
(\$. . /. /. /. /. /. /. /. /. /.
\$.). Classification: universal (\$. .) –
projects with no derived-export/DB-sync surface inherit it
latently (binds the moment they ship one, \$. .
pattern). Status (\$. .): engine Phases – AND the
CLI/YAML loader (Phase) IMPLEMENTED+tested – cmd/
docs_chain/main.go registers sync / verify / diff (alias)/
doctor., go test -count=1 ./... GREEN / packages; this
CLI/loader is in the working tree but UNTRACKED and the
engine is NOT yet a registered git submodule (in-tree at ./
docs_chain, HEAD = parent HEAD); only submodule distribution
(Phase) remains PLANNED + OPERATOR-GATED – wire as phases
land, claim no unshipped behaviour. Propagation gate CM-
COVENANT-114-106-PROPAGATION (literal 11.4.106 across consumer
fleet) + paired \$. meta-test mutation (strip literal →

gate FAILs; gate-code = separate work item). No escape hatch
– no `--ad-hoc-sync-ok` , `--skip-docs-chain` , `--fake-transform` , `--sync-evidence-optional` flag.

Canonical authority: constitution submodule `Constitution.md`
§ . . ; detailed spec the Docs Chain engine docs
`docs/CONSTITUTION_INTEGRATION.md` + `docs/USE_CASE_CATALOGUE.md`
(`vasic-digital/docs_chain`).

Non-compliance is a release blocker.

§ . . – **Anti-bluff AV/test-validation techniques mandate** (User-driven research, – –)

Forensic anchor (genericised, – –): a test PASSED on a SINGLE captured frame showing “a picture” on the target output – but the picture was a FROZEN / STALE frame from the previously-played content (stale-producer / stuck-decoder), so the feature was broken for the user while the test was green; a sibling incident FLASHED media on the WRONG output for ~ s before routing, with no test sampling that window; a third class shipped a comparator that PASSED its own deliberately-degraded fixture (the analyzer, not the feature, was the bluff). § . . mandates captured evidence + a presence pass; § . . raises the bar to **liveness + correct-routing + self-validated-analyzer**.

Every test asserting audio/video output is genuinely playing/advancing for the end user MUST satisfy ALL of: () a **single captured frame is NOT proof** – prove LIVE, ADVANCING frames over a steady-state window via a **freeze-detection oracle** (near-duplicate / `freezedetect` -class filter OR perceptual-hash adjacent-frame distance, **NOT byte-identical compare** – byte-identity is only a zero-cost pre-filter); () an **independent frame-advance counter from the platform’s compositor/decoder telemetry** must increase across the window (a different-domain oracle – flat counter ⇒ stuck decoder ⇒

FAIL even if pixels appear to move); () **loading/buffering is a distinct state** – wait for genuine playback-start before judging liveness, never false-FAIL a still-loading stream nor false-PASS a spinner; timeout+unreachable $\Rightarrow$ SKIP-with-reason per § 11.4.6, timeout+reachable $\Rightarrow$ FAIL; () **not-stale-from-previous cross-check** – new content’s first frame $\neq$ previous content’s last frame; () **measured FPS / no-lost-frames within tolerance**; () **no-flash-on-the-wrong-output** – sample the non-target output at high frequency during a routing transition, any content frame there $\Rightarrow$ FAIL (content-protection regime classified explicitly, never guessed); () **drive through the realistic feed/UI path, not deep-link shortcuts** (shortcuts bypass the transition paths where bugs live; UI-not-introspectable $\Rightarrow$ operator-attended fallback per § 11.4.6, never fake-PASS); () **metamorphic relations solve the oracle problem when there is no golden source** (same content on output-A vs output-B must match; paused $\Rightarrow$ counter stops; $\times$ speed $\Rightarrow \sim \times$ advance rate); () **full-reference quality metrics (SSIM/VMAF/ ΔE) vs a golden source for owned content**; () **mutation-test every analyzer with a golden-good + golden-bad fixture pair** so the analyzer itself provably cannot bluff (an analyzer that PASSes its golden-bad fixture is a bluff gate – § 11.4.6 applied to the analyzers); () **per-channel audio RMS/loudness (EBU R128) + XRUN/underrun census** – a single aggregate RMS misses a dead channel; () **OCR overlay/subtitle detection needs a per-word confidence floor + ROI** to avoid BOTH false-positive (false-FAIL) and false-negative (false-PASS); () **thresholds calibrated on the project’s own fixtures, not hardcoded from literature** (§ 11.4.6 no-guessing).

Honest gap (§ 11.4.6): true photon FPS at the sink has no clean software oracle – compositor/decoder counters measure the presentation pipeline; flag the gap, never claim it.

-layer coverage per § 11.4.107 (b): pre-build gate (a CM-AV-LIVENESS-NO-FROZEN-FRAME -class gate asserting every output-is-playing test references the liveness battery not a single frame + an analyzer-self-validation gate wiring the golden-good/golden-bad fixtures into meta-test) + on-device/runtime test + paired § 11.4.107 meta-test mutation (single-frame-only assertion → gate FAILs; analyzer that PASSes its golden-bad fixture → self-validation FAILs) + HelixQA Challenge. Every PASS via ab_pass_with_evidence citing the motion / not-stale / frame-advance / fps / per-channel-loudness / metamorphic / self-validation artefacts (video_display / audio_output / subtitle_render per § 11.4.107) – never a single screenshot.

Classification: universal (§ 11.4.107) – platform-neutral AV/test-validation techniques reusable by ANY project validating media playback or any pixel/audio output; the project supplies its concrete capture mechanism + calibrated thresholds per § 11.4.107. Composes with § 11.4.107 (its strict expansion), § 11.4.107, § 11.4.107, § 11.4.107, § 11.4.107, § 11.4.107, § 11.4.107, plus § 11.4.107 / § 11.4.107 / § 11.4.107 / § 11.4.107 / § 11.4.107 / § 11.4.107. Propagation gate CM-COVENANT-114-107-PROPAGATION enforces the literal anchor 11.4.107 across the consumer fleet; paired § 11.4.107 meta-test mutation strips the literal → gate FAILs (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md § 11.4.107.

Non-compliance is a release blocker regardless of context. No escape hatch – no --single-frame-proves-playback, --skip-liveness, --byte-identical-freeze-OK, --no-frame-advance-counter, --skip-not-stale-check, --allow-wrong-output-flash, --deep-link-shortcut-OK, --unvalidated-analyzer-OK, --aggregate-rms-suffices, --hardcoded-thresholds-OK flag exists.

§ . . – Four-layer fix-verification + runtime-signature-as-definition-of-done mandate (systematic-debugging Phase . , – –)

Forensic anchor (genericised, – –): across one batch, multiple fixes were “green” at every gate yet NOT working for the end user – a change present in source and passed by both the source pre-build gate AND the post-build gate never reached the boot-time command-line embedded in the deployed boot artifact (output feature dead despite all-green); sibling fixes correctly built into the system image were masked by stale per-user overlay copies from a previous deployment (the running code was the stale shadow, not the fresh build); deployment did not wipe the mutable overlay so the shadow survived; validation ran against whatever was running – the stale shadow – and reported green on code that was never exercised. Per `superpowers:systematic-debugging` Phase . : each fix revealing a fresh “fixed-but-not-working” in a DIFFERENT place is NOT independent bugs – it is ONE architectural VERIFICATION flaw; patching each symptom is thrashing.

A fix crosses FOUR distinct layers, and “fixed” at one does NOT imply fixed at the next: () **SOURCE** (committed in the source file – what a `grep-the-source` pre-build gate checks), () **ARTIFACT** (the change’s BYTES actually landed in the produced build artifact – image / bundle / installer / embedded command-line), () **RUNTIME-ON-CLEAN-TARGET** (active on a CLEAN/fresh deployment – the layer the end user experiences – with no stale overlay shadowing the deployed code), () **USER-VISIBLE** (the feature works for the end user – the § . . /§ . . captured-evidence layer). Green at layer is the cheapest, least conclusive signal and says nothing about layers – . A gate verifying ONLY the source layer is itself a § . bluff surface.

The mandate (ALL must hold): () **Runtime-signature-as-definition-of-done** – a fix is DONE only when its declared runtime signature verifies on a CLEAN/fresh deployment; source-committed $\neq$ artifact-contains-it $\neq$ active-on-clean-target $\neq$ user-visible-working. () **Every fix declares ONE machine-checkable runtime signature** – a single observable on a clean target proving the fix is BOTH active AND working (a running-system property, a downstream/sink-side report per § . . . , a § . . . /§ . . . captured-evidence assertion, or a counter/state delta – NEVER a regrep of the source); this **registry of per-fix runtime signatures is the SINGLE SOURCE OF TRUTH** for “fixed” – it REPLACES “a gate greps the source” as the definition of done. () **Gates span all four layers** – source (pre-build), artifact (post-build – assert the change’s BYTES landed in the artifact, not merely that the source still contains the change), runtime-on-clean-target (post-deploy – assert the runtime signature on a freshly-deployed clean target), user-visible (§ . . . /§ . . .). () **Eliminate the stale-deployment / shadow layer by construction** – deployment MUST yield a CLEAN state (wipe the mutable overlay) OR a pre-validation assertion MUST prove `running-artifact == built-artifact` BEFORE any validation runs; validation against possibly-stale deployed state is INVALID and any PASS it produces is a § . . . PASS-bluff (the test exercised code that was never deployed). () **Meta-rule** – $\geq$ “fixed-but-not-working” discoveries in one cycle signal an architectural VERIFICATION flaw, NOT three independent bugs; on the rd, STOP patching symptoms (§ . . .), fix the VERIFICATION pipeline, and re-certify EVERY item through it on a clean target. () **A batch is “validated” only after COMPREHENSIVE per-item runtime-signature verification on a clean baseline** – NOT after the touched items’ own tests pass.

Classification: universal (§ . . .) – platform-neutral verification-pipeline disciplines reusable by ANY project that builds an artifact, deploys it, and validates the

deployed result; the project supplies its concrete artifact format, clean-deployment / equal-artifact mechanism, and per-fix runtime-signature observables per § Composes with § . . . / § . . . / § . . . (clause . . 's STOP-and-fix-pipeline is its § . . . specialisation; “clean baseline” = clause . . 's clean deployment) / § . . . / § . . . (every layer asserted by evidence, never assumed to propagate) / § . . . / § . . . (§ . . . 11 4 102 adds the cross-layer per-item runtime-signature dimension) / § . . . (clean-baseline / equal-artifact pre-flight) / § . . . / § . . . / § . . . (a runtime signature is a taxonomy-class observable) / § . . . (Phase . . . architectural-flaw recognition IS clause . . 's trigger). Propagation gate CM-COVENANT-114-108-PROPAGATION (literal 11.4.108 across the consumer fleet) + recommended per-fix gate CM-RUNTIME-SIGNATURE-REGISTRY + paired § . . . meta-test mutations (strip the literal → propagation gate 11 4 108 FAILs; downgrade a fix to source-only verification → registry gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md

§

Non-compliance is a release blocker regardless of context. No escape hatch – no --source-green-is-done , --skip-artifact-byte-check , --validate-against-running-state , --no-clean-deployment , 11 4 109 --skip-runtime-signature , --spot-validate-touched-only flag exists.

§ . . . – **Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist (Operator mandate)**

Short tag: anti-forgetting-enforcement . UNCONFIRMED forensic anchor (pending operator's verbatim mandate quote).

Background: emulator subagents ran raw host-direct emulator /

adb because the orchestrator forgot to inject the Containers-submodule rule. A rule forgotten at dispatch is not enforcement. Fix: (A) a PreToolUse guard hook (constitution/scripts/hooks/guard-forbidden-commands.sh) that blocks host-direct emulator, force-push/bypass, sudo, and host-power commands at the tool-call boundary regardless of agent memory; (B) a canonical docs/AGENT_GUARDRAILS.md preamble the orchestrator pastes verbatim into every subagent dispatch; (C) an **ORCHESTRATOR PRE-ACTION CHECKLIST** in the same document. Hook = the floor; preamble = the ceiling.

Consuming projects MUST: () wire constitution/scripts/hooks/guard-forbidden-commands.sh as a PreToolUse hook in .claude/settings.json (or equivalent runtime settings); () maintain docs/AGENT_GUARDRAILS.md containing the SUBAGENT CONSTITUTIONAL PREAMBLE and ORCHESTRATOR PRE-ACTION CHECKLIST headings, with the anchor literal 11.4.109 ; () provide a hermetic hook test suite (≥ cases: every blocked class exits , every allowed command exits , escape hatch fires for non-power classes, host-power rejects even with escape marker). The hook is inherited by reference – NEVER copied locally (a copy diverges silently).

Gates: CM-ANTI-FORGETTING-ENFORCEMENT (hook present + wired + guardrails doc present + test present) + CM-COVENANT-114-109-PROPAGATION (anchor literal 11.4.109 across every consumer CLAUDE.md / AGENTS.md / QWEN.md). Paired \$. mutations: remove hook entry → gate () FAILs; delete guardrails doc → gate () FAILs; strip hook from constitution → gate () FAILs; strip 11.4.109 → propagation gate FAILs.

Classification:.. universal (\$. .). Composes with
 \$. . / \$. . / \$. . /
 \$. . / \$. . / \$. . /
 \$. . / \$. . / \$. . /
 \$. . / \$. . / \$.

Canonical authority: constitution submodule Constitution.md
\$. . ; reference implementation constitution/scripts/
hooks/guard-forbidden-commands.sh + constitution/docs/
AGENT_GUARDRAILS.md .

Non-compliance is a release blocker. No escape hatch – no --
skip-pretooluse-hook , --no-guardrails-doc , --anti-forgetting-
optional , --single-layer-sufficient flag exists.

\$. . – Pre-build build-readiness
verdict + change-impact clash detection mandate
(operator mandate, – –)

Forensic anchor (genericised, – –): a fix
shipped a new system-property *read* but introduced no matching
security-policy grant + no property-context type entry – the
read was silently denied at runtime + the feature was dead,
while every pre-build gate stayed green because the gate only
grepped the source file. The defect class generalises: a
change introduces a new dependency on a *second* artifact
(security policy, context file, service registry, interface
freeze-snapshot, symbol table, build-graph node) that the
pre-build never cross-checks. This is \$. . 's
SOURCE→ARTIFACT gap shifted LEFT to pre-build time – most
such clashes are statically catchable from the change diff
itself, before any build. The mandate (ALL hold): () a
single deterministic **READY-FOR-BUILD verdict** gates the
rebuild (orchestrator refuses to start the build unless
READY); () a **diff-driven change-impact + clash detector**
cross-checks every newly-introduced second-artifact
dependency (new property read ⇔ property-context type + read-
grant; new service ⇔ service-context entry; new init service
⇔ security label; new/changed stable interface ⇔ freeze-
snapshot updated; new native-lib dep ⇔ module/prebuilt
resolves; new policy rule ⇔ every type/attribute defined; two
batch changes on the same seam ⇔ collision acknowledged);
() **coverage-completeness is a gate** – every changed file

maps to $\geq$ gate + $\geq$ deployed-target test + $\geq$ paired
 \$. mutation, baseline ratchets upward per \$. . ;
 () **two-speed honesty** – grep-speed always-on gates vs
 REQUIRES_BUILD heavy gates (build-graph parse-only dry-run,
 full neverallow compilation, ABI diff) as diff-gated opt-in
 stages, bounded per \$. /\$. ; () every gate +
 wired analyzer is **anti-bluff by paired \$. mutation**; ()
honest boundary – a READY verdict proves static internal-
 consistency + ready-to-build, NOT that the feature works on
 the deployed target; the regime empties the *preventable*
 defect class, not the *all-defects* class (runtime/USER-VISIBLE
 remains \$. . 's job).

Classification: universal (\$. .) – platform-neutral
 pre-build-rigor disciplines reusable by ANY project that
 builds an artifact from source; the consuming project
 supplies its concrete property/service/policy registries,
 build-graph parse-only command, interface-freeze mechanism,
 and changed-file→gate mapping per \$. . . Composes
 with \$. . / \$. . / \$. . /
 \$. . / \$. . / \$. . /
 \$. . / \$. . / \$. . /
 \$. . (\$. . is the SOURCE→ARTIFACT
 half shifted left to pre-build; \$. . owns
 RUNTIME-ON-CLEAN-TARGET→USER-VISIBLE – together they span all
 four layers). Propagation gate CM-COVENANT-114-110-PROPAGATION
 (literal 11.4.110) + recommended per-family gates CM-READY-
 FOR-BUILD-VERDICT / CM-CHANGE-IMPACT-CLASH-DETECTOR / CM-COVERAGE-
 COMPLETENESS-GATE / CM-BUILDGRAPH-DRYRUN-WIRED / CM-SEPOLICY-
 NEVERALLOW-WIRED + paired \$. mutations (gate-code =
 separate work item).

Canonical authority: constitution submodule Constitution.md
 \$. . . Non-compliance is a release blocker. No
 escape hatch – no --source-green-is-ready , --skip-clash-

```
detector , --skip-coverage-gate , --no-ready-verdict , --grep-  
proves-neverallow , --skip-buildgraph-dryrun ,  
--build-without-ready-verdict flag.
```

\$. . – Resolve-by-stable-name-not-by-enumeration-index mandate (research-derived, – –)

Forensic anchor (genericised, – –): a platform bound an audio output to a kernel-enumerated device index (card=0); a second device of a different class enumerated FIRST at boot and took slot , shifting the intended device to slot – the static index binding now pointed at the wrong device (policy mis-attached the sink, mis-assigned its TYPE, the output switcher labelled the AV receiver “Wired Headphone” + routing collapsed to stereo). The lower layer of the SAME stack already resolved the device correctly **by name** (scanning the controller-name registry) – proving the brittleness was the *index* binding, not the resolution capability. Generalises to every enumerated resource whose ordinal is assigned at discovery/boot/hotplug time (non-deterministic across reboots, device additions, topology changes). The mandate: any binding to a hardware device / resource handle / enumerated entity (audio cards, display connectors, network interfaces, storage devices, GPU render nodes, input/camera devices, container/process slots) **MUST** resolve by a **stable identifier** (name / UUID / serial / label / controller-name / content-hash / sink-reported identity) and **MUST NOT** bind by enumeration index / ordinal / slot, **UNLESS** the platform documents that ordinal as deterministically pinned **AND** the pin is itself captured + asserted as part of the binding. Where a stable identifier exists at one layer, every other layer binding the same resource **MUST** use the same identifier (mixed by-name-here / by-index-there is the structural weak link forbidden here). Honest boundary (\$. .): when only an ordinal exists, pin it deterministically via the platform’s own mechanism,

capture the pin in the binding's § 11.4.17 runtime signature, and document the residual fragility as UNCONFIRMED: -class risk – never silently trust an unpinned ordinal.

Classification: universal (§ 11.4.17) – platform-neutral binding-robustness discipline reusable by ANY project binding to enumerated hardware/resources/handles; the consuming project supplies its stable-identifier mechanism (ALSA card name, DRM connector name, NIC predictable name, block-device UUID, etc.) + the layers that must agree per § 11.4.17. Composes with § 11.4.17 (no-guessing – “the index is *usually* stable” is the exact guess forbidden) / § 11.4.17 (mature stacks resolve by name/UUID – reproducing a known-brittle index binding when the by-name path is documented is a § 11.4.17 omission) / § 11.4.17 (sink-side evidence verifies the by-name binding's correctness) / § 11.4.17 (the by-name binding's runtime signature asserted on a clean target across the topology that broke the index) / § 11.4.17 (a new ordinal binding in a diff is a statically-catchable clash class). Propagation gate CM-COVENANT-114-111-PROPAGATION (literal 11.4.111) + recommended gate CM-RESOLVE-BY-NAME-NOT-INDEX + paired § 11.4.17 mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md § 11.4.17. Non-compliance is a release blocker. No escape hatch – no --allow-index-binding, --ordinal-is-stable-enough, --skip-name-resolution, --trust-unpinned-index flag.

§ 11.4.17 – Structural-impossibility won't-fix classification mandate (research-derived, – –)

Forensic anchor (genericised, – –): deep research (§ 11.4.17) into relocating protected (content-protection / secure-surface) video to a secondary display PROVED – from authoritative platform/HDCP docs + reproducible

captured behaviour (a secure surface is blanked on any output lacking the secure flag; mirror/screenshot of a secure layer returns black) – that the goal is **structurally impossible by platform design**, not a missing feature or unsolved engineering problem. Without a durable classification, such a goal is re-investigated every cycle (re-read the same sources, re-run the same probes, re-derive the same impossibility – compounding wasted effort). The mandate: when deep research per § 4.1.1 PROVES (cited authoritative sources AND, where applicable, reproducible captured evidence) a goal is structurally impossible on the target platform (forbidden by platform design / hardware-protocol constraint / documented kernel-or-API limitation – NOT merely unimplemented or hard), the goal MUST be: () classified **Won't-fix** + closed per § 4.1.2 with closure reason **structurally-impossible**; () documented with the impossibility evidence – cited authoritative source URLs (per § 4.1.3 latest-source verification) + the reproducible probe/captured evidence – in the tracker entry + relevant docs/ guide; () NOT re-attempted in future cycles – a reopen MUST cite NEW evidence the platform constraint changed (per § 4.1.4 + § 4.1.5), never merely re-derive the same impossibility; () paired with the correct posture – the entry states what the project DOES instead, so “impossible” is never confused with “broken/unhandled”. Honest boundary (§ 4.1.6): **structurally-impossible** is reserved for *proven* platform/hardware/protocol impossibility – “could not find a way” / “very hard” / “no time” are **Operator-blocked** (§ 4.1.7) or open work, NOT won't-fix; mislabelling them to avoid the work is a § 4.1.8 planning-layer bluff. A future platform change can make the impossible possible; the classification is durable but not eternal.

Classification: universal (§ 4.1.9) – platform-neutral effort-conservation + honesty discipline reusable by ANY project; the consuming project supplies the specific impossible goal, its platform constraint, and the cited

evidence per § 11.4.112. Composes with § 11.4.112 (FACT with cited evidence, never “probably can’t”) / § 11.4.112 (reopening requires NEW positive evidence) / § 11.4.112 (“NO external solution found – structurally impossible” is the citation) / § 11.4.112 (reopen attribution) / § 11.4.112 (structurally-impossible is a closure reason in the closed vocabulary) / § 11.4.112 (latest-source so the verdict is not stale). Propagation gate CM-COVENANT-114-112-PROPAGATION (literal 11.4.112) + recommended gate CM-WONT-FIX-STRUCTURAL-IMPOSSIBILITY + paired § 11.4.112 mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md
 § 11.4.112. Non-compliance is a release blocker. No escape hatch – no --wont-fix-without-proof, --reattempt-closed-impossible, --skip-impossibility-evidence, --impossible-equals-broken flag.

§ 11.4.112 – Absolute no-force-push + merge-onto-latest-main mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):
 “Any force-push is strictly forbidden! We must for every Submodule take as a base latest commit on Submodule’s main (or master) branch, then on top of it carefully to merge all changes that have to be pushed! Once all merging is carefully done we perform commit and push to all Submodule’s upstreams!”

Force-push is STRICTLY FORBIDDEN with NO exception – git push --force, --force-with-lease, +<ref>, or any history-rewriting overwrite of a remote ref, against EVERY repository this Constitution governs (main repo, this constitution submodule, every owned + nested submodule, every upstream). No operator-approval path, no “after a merge-first audit” path. The mandated -step integration procedure for any repo/submodule whose local has commits to publish OR whose mirrors

diverged: () `git fetch --all --prune --tags` all remotes; ()
 set the base to the LATEST commit on the canonical `main /`
`master` branch (the most-advanced mirror tip); () carefully
 MERGE every change to be published on top of that base –
 union, preserve BOTH sides, NEVER `-s ours` / `rebase` / `reset`
 that drops commits (per \$ `no-commit-loss`); () resolve
 every conflict carefully – no conflict markers, no file
 dropped, gates/tests still pass; () commit the merge (stage
 only intended files, NEVER `git add -A` in a submodule per
 \$ `. . .`); () push to ALL upstreams – each push is a
 fast-forward because the merge commit descends from every
 mirror tip, so NO force is ever needed (if an upstream still
 rejects, return to step `for it, merge its new tip, re-`
`validate, re-push`). **TIGHTENS** \$ `. . . /`
 \$ `. . . / $ . . . / CONST-` – **the force-push**
escape hatch is REMOVED: even WITH operator approval, even
 after a clean merge-first audit, force-push is forbidden,
 because the merge-onto-latest-main path is always available
 so force is never necessary. Those clauses' merge-first/
 fetch-first machinery stays in force as the integration
 discipline; only their terminal “...then force-push” step is
 struck.

Classification: universal (\$ `. . .`) – a platform-
 neutral integration discipline reusable across every
 repository. Composes with \$ `. . .` (multi-upstream push – step
`fans out`) / \$ `. . . / $ . . .` (absolute data safety – this
 is the no-loss push discipline that makes force
 unnecessary) / \$ `. . . / $ . . .` (remote state
 unknowable without the `step- fetch`) / \$ `. . .`
 (constitution-update conflict resolution IS this procedure) /
 \$ `. . .` (fetch-before-edit → fetch-before-push) /
 \$ `. . . / $ . . .` (merge-first stays, force-
 push step removed) / \$ `. . .` (same tightening) /
 \$ `. . .` (background-push still fast-forward-only) /
 CONST- (no force-push authorisable). Propagation gate
 CM-COVENANT-114-113-PROPAGATION (literal `11.4.113`) +

recommended gate `CM-NO-FORCE-PUSH-ABSOLUTE` (scan tracked scripts/hooks for `push --force` / `--force-with-lease` / `push +<ref>` + reject; a `$ . . -class PreToolUse` guard blocks the class at the tool-call boundary) + paired `$ . mutation` (inject a `git push --force` into a tracked script → gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md`
`$ . . . .` Non-compliance is a release blocker. No escape hatch – no `--force` , `--force-with-lease` , `--allow-force-push` , `--force-push-authorised` , `--skip-merge-onto-main` flag.

`$ . . . .` – **Last-known-good-tag**
regression isolation mandate (`. . -dev`
remediation, `- -`)

When a previously-working feature/behaviour is observed broken, the FIRST diagnostic action MUST be to identify the last release tag (or commit/build/deploy) at which it was KNOWN-GOOD and diff/bisect the broken state against it – BEFORE any open-ended root-cause hunt or speculative fix. The known-good revision is the regression oracle: it bounds the search to the commits between good and now (`git diff <good-tag>..HEAD --stat` of the feature’s files = captured evidence), tells you it is a regression REPAIR not a from-scratch design problem, and gives each suspect file a behavioural oracle. When the operator volunteers a known-good tag, that lead is load-bearing and MUST be acted on first. Default to a SURGICAL forward-fix (keep the post-good-tag features, revert ONLY the broken sub-part) over a wholesale revert that loses the batch’s other working features – unless the operator prefers wholesale revert. Honest boundary (`$ . . . .`): “it worked before” is a HYPOTHESIS until the known-good tag is identified AND the feature is confirmed working there; “probably regressed in the last batch” without the diff is a guess; a feature that NEVER worked is not a regression. Composes `$ . . . . / $ . . . . / $ . . . . /`

\$. . / \$. . / \$. . /
 \$. . . Propagation gate CM-COVENANT-114-114-
 PROPAGATION (literal 11.4.114) + recommended gate CM-
 REGRESSION-ISOLATED-AGAINST-KNOWN-GOOD + paired \$. mutation
 (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md
 \$. . . Non-compliance is a release blocker. No
 escape hatch – no --skip-known-good-diff, --root-cause-from-
 scratch, --assume-regression-source, --wholesale-revert-without-
 isolation flag.

\$. . – RED-baseline-on-the-broken-artifact + polarity-switch mandate (. . -dev remediation, – –)

Strict refinement of \$. . 's RED step. Every RED test MUST be authored to REPRODUCE the defect on the CURRENT, pre-fix artifact (the actual broken build/deployment), capturing positive evidence per \$. . / \$. . /
 \$. . that the defect is genuinely present – never a synthetic failure the fix is then written to agree with. The SAME test source MUST carry a single polarity switch (env flag / parameter, canonical RED_MODE, default 1 = reproduce-and-assert-defect-present) that flipped to 0 post-fix converts the test into the GREEN regression-guard asserting the defect is ABSENT. One source, two roles: the bug-catcher IS the regression-guard; no separate happy-path test is authored as the primary guard (that demonstrates only that the test agrees with the fix – the \$. . PASS-bluff). RED-on-broken-artifact then GREEN-on-fixed-artifact (on a clean target per \$. .) MUST both be captured. Honest boundary (\$. .): if the RED run does NOT fail on the broken artifact, that is a finding (close per \$. . with negative evidence, or fix the test) – a RED test that passes on the known-broken artifact is a blind test. Composes \$. . / \$. . / \$. . /

\$. . / \$. . / \$. . /
 \$. . / \$. . / \$. . /
 \$. . / \$. . . Propagation gate CM-
 COVENANT-114-115-PROPAGATION (literal 11.4.115) + recommended
 gate CM-RED-POLARITY-SWITCH-PRESENT + paired \$. mutation
 (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md

\$. . . Non-compliance is a release blocker. No
 escape hatch – no --green-only-test , --skip-red-reproduction ,
 --separate-happy-path-suffices , --synthetic-red-OK flag.

\$. . – Real-time
 conductor autonomous-test-framework sync
 channel mandate (. . -dev remediation,
 – –)

Any autonomous, long-running test/QA/validation framework an
 external orchestrator (conductor agent / operator) depends on
 for real-time decisions MUST expose a real-time sync channel:
 () a structured append-only event stream (JSONL or
 equivalent, one event per line, never rewritten) emitting at
 minimum session-start / phase-transition / per-test-or-
 challenge-start / captured-evidence-path / external-call
 (LLM / vision / sink-probe) / error / per-item-verdict
 events; () an atomically-rewritten status snapshot (single
 small file written write-temp-then-rename so a reader never
 sees a torn write) carrying current session/phase/item +
 counters + last verdict. Verdicts use the closed vocabulary
 PASS / FAIL / SKIP / OPERATOR-BLOCKED (\$. .). The
 conductor tails it live so it stays in real-time sync, can
 \$. . -interrupt on a fresh defect, and never idles
 blindly (\$. . / \$. .). Anti-bluff: a
 verdict event MUST carry the evidence path that backs it
 (\$. .) – a PASS event with no evidence path is a
 channel-layer PASS-bluff; a snapshot reporting PASS while the
 stream shows no evidence event for that item is a

contradiction → treat as FAIL; an item with no start-event cannot have a verdict-event. When the framework is an owned project-agnostic submodule (\$. .), the channel stays project-neutral — the consumer registers its data (endpoints, package ids, sink hosts) at runtime via the public API, never hardcoded. Composes \$. . / \$. . / \$. . / \$. . / \$. . / \$. . / \$. . / \$. . . Propagation gate CM-COVENANT-114-116-PROPAGATION (literal 11.4.116) + recommended gate CM-AUTONOMOUS-FRAMEWORK-SYNC-CHANNEL + paired \$. mutation (gate-code = separate work item).

Canonical authority: constitution submodule `Constitution.md`

\$. . . Non-compliance is a release blocker. No escape hatch — no `--no-sync-channel`, `--end-of-session-report-suffices`, `--verdict-without-evidence-path`, `--torn-status-write-OK` flag.

```
$ . . - Computer-vision / OCR pixel-
oracle fallback for non-introspectable UIs
mandate ( . . -dev remediation,
          - - )
```

Any test that needs to drive a UI control OR assert on-screen content MUST NOT assume the accessibility/semantic/DOM hierarchy is the source of truth for what the user sees. When the hierarchy is blank/partial/known-unreliable for the app under test (TV-Compose, leanback, canvas/ `SurfaceView` /GL, games, custom-rendered UIs), the test MUST fall back to a PIXEL ORACLE: () DRIVE input by computer-vision template-match (locate a control by its rendered appearance → tap its screen coordinates), not by a hierarchy node id; () ASSERT content by ROI OCR (read the rendered text the user reads – caption strip, title, error overlay) with a per-word confidence floor + region-of-interest per § . . (), not a hierarchy text attribute that

may be absent/stale. The tool MUST both drive input AND read pixels – a hierarchy-only tool is NOT a content oracle. This makes § 11.4.52’s “near-empty hierarchy → INFEASIBLE” constructive: pixel-drive, not only SKIP. Anti-bluff (§ 11.4.52 (1)): the CV/OCR analyzer is self-validated – golden-good fixture PASSES, golden-bad fixture FAILs, wired into meta-test; thresholds calibrated on the project’s own frames, not hardcoded (§ 11.4.52). Honest boundary: when BOTH hierarchy is blank AND pixel oracle is infeasible (secure surface black-captures per § 11.4.52, geo-unreachable), SKIP-with-reason per § 11.4.117 + tracked operator-attended migration item per § 11.4.117 – never fake PASS. Composes § 11.4.117 / § 11.4.117 / § 11.4.117 / § 11.4.117 / § 11.4.117 / § 11.4.117. Propagation gate CM-COVENANT-114-117-PROPAGATION (literal 11.4.117) + recommended gate CM-CV-OCR-PIXEL-ORACLE-FALLBACK + paired § 11.4.117 mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md § 11.4.117. Non-compliance is a release blocker. No escape hatch – no --hierarchy-is-content-oracle, --skip-pixel-fallback, --unvalidated-ocr-OK, --hardcoded-ocr-threshold-OK flag.

§ 11.4.117 – **Discovery-pressure to confirm known-issue-set completeness mandate** (11.4.117 -dev remediation, 11.4.117 -dev remediation)

A remediation/release cycle MUST NOT treat “every reported defect is fixed” as “the build is good” – the reported set is biased by what the operator happened to test (“we only see what we test”). After/alongside fixing the reported set, the cycle MUST run a discovery + stress pass across ALL target devices/environments that deliberately exercises subsystems, journeys, and edge cases BEYOND the reported defects – to CONFIRM the reported set is the COMPLETE critical set and

surface unreported defects before the end user does. The pass MUST produce PROVABLE coverage: an enumerated list of the subsystems/user-journeys/stress scenarios actually exercised, each with its outcome (no-new-issue, or a new tracker entry per § 11.4.118 / § 11.4.119). “We found no other issues” is a § 11.4.118 bluff unless accompanied by “here is the enumerated set we exercised” – absence of evidence of looking is not evidence of absence. New findings trigger § 11.4.118 interrupt + the § 11.4.118 / § 11.4.119 isolation→RED→fix loop. Honest boundary (§ 11.4.118): discovery reduces the unknown-unknown surface but does not prove zero remaining defects – the earned claim is “reported set + enumerated discovery set addressed,” with un-exercised subsystems stated as honest coverage gaps (§ 11.4.118), never silently implied clean. Composes § 11.4.118 / § 11.4.118 / § 11.4.118 / § 11.4.118 / § 11.4.118 / § 11.4.118 / § 11.4.118 / § 11.4.118 / § 11.4.118 . Propagation gate CM-COVENANT-114-118-PROPAGATION (literal 11.4.118) + recommended gate CM-Discovery-Coverage-Enumerated + paired § 11.4.118 mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md § 11.4.118 . Non-compliance is a release blocker. No escape hatch – no --reported-set-suffices , --skip-discovery-pass , --no-issues-without-coverage-list , --assume-complete flag.

2026 06 03

§ 11.4.118 – Single-resource-owner partitioning for parallel hardware testing mandate (11.4.118 -dev remediation, - -)

Strict refinement of § 11.4.118 / § 11.4.118 for hardware contention. When multiple parallel work/test/discovery streams exercise SHARED hardware or any exclusive-access resource (a media-playback path, a single HDMI/audio

§ . . – Fix-breaks-its-own-gate reconciliation mandate (. . -dev remediation, – –)

When a correct fix causes a pre-existing gate/test to FAIL because that gate asserted the OLD (now-removed-or-changed) behaviour, the gate FAIL is the CORRECT signal the fix landed – it MUST NOT be suppressed by either forbidden response:

() FAKE-PASSING the gate (editing it to `always pass`, weakening its assertion to a tautology, or deleting it – a § . gate-layer bluff + a § . . mutation-residue risk); () REVERTING the correct fix to satisfy the stale gate (re-introducing the defect). The required response is RECONCILIATION: rewrite the gate to assert the NEW mechanism the fix introduced, backed by captured evidence of the new correct behaviour, AND update its paired § . mutation so the mutation breaks the NEW invariant.

Discriminator vs bluffing: after reconciliation the gate + mutation still form a valid § . pair (mutate the new invariant → gate FAILs); a bluffed gate's mutation no longer makes it FAIL (the assertion became a tautology). The reconciliation MUST be a visible, evidence-cited change, never a silent assertion-weakening. Honest boundary (§ . .): a post-fix gate FAIL is NOT automatically “stale gate, reconcile” – investigate per § . . first; the FAIL may be the gate correctly catching a REGRESSION the fix introduced, in which case the FIX is wrong, not the gate. Reconcile ONLY when investigation PROVES the gate asserted old-correct-now-removed behaviour AND the new behaviour is the intended, evidence-confirmed mechanism.

Composes § . . / § . . / § . . /
§ . . / § . . / § . . /

§ . . Propagation gate `CM-COVENANT-114-120-PROPAGATION`
(literal `11.4.120`) + recommended gate `CM-GATE-RECONCILED-NOT-FAKE-PASSED` + paired § . mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md

\$. . . Non-compliance is a release blocker. No escape hatch – no --fake-pass-stale-gate , --revert-fix-for-gate , --weaken-assertion-to-pass , --delete-failing-gate flag.

\$. . . – No-commit-while-build-writes-tracked-artifacts mandate (. . -dev remediation, – –)

A commit (especially git add -A / any broad stage) MUST NOT run while a build/packaging/generation step is actively writing artifacts INTO tracked (version-controlled) directories – doing so races the writer and stages a PARTIAL or stale artifact (a \$. . . SOURCE→ARTIFACT integrity failure landed in version control). The commit MUST be deferred until the build step that writes tracked artifacts has COMPLETED, so the tree is quiescent at the artifact layer AND the committed artifacts are the FRESH, whole outputs (no stale pre-rebuild artifact, no half-written file). Before committing tracked build outputs, verify the writing step finished (process exit / completion marker / per-artifact mtime ≥ build-start) – a build still in flight writing tracked dirs is a HOLD on the commit, not a race to win. Build-output analogue of \$. . . (no commit while a mutation gate is in flight): both close the “commit captures transient non-final tree state” class at two write-sources. Where build outputs land OUTSIDE version control (gitignored out/ / dist/), the race does not apply. Honest boundary (\$. . .): “the build probably finished” is not “the build finished” – verify with a completion signal; committing source changes that DON’T touch the build’s tracked-artifact directories is fine mid-build. The PROJECT-SPECIFIC instance (which tracked directory + which build steps write it) is recorded in the consuming project’s own governance per \$ Composes \$. . . / \$. . . / \$. . . / \$. . . / \$. . . /

\$. . . Propagation gate CM-COVENANT-114-121-PROPAGATION (literal 11.4.121) + recommended gate CM-NO-COMMIT-DURING-ARTIFACT-WRITE + paired \$. mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md

\$. . . Non-compliance is a release blocker. No escape hatch – no --commit-during-build, --stage-partial-artifact-OK, --assume-build-finished, --skip-build-completion-check flag.

2024 06 03

\$. . . – No-silent-removal-of-existing-components-without-operator-confirmation mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

“Never ever remove any application, system component or service from already existing codebase / System without interactively asked question to us! THIS IS MANDATORY RULE / CONSTRAINT!”

Forensic case study (FACT). During the . . . -dev burn-down, two shipped capabilities – F (an Apple-TV-class application) and F (a Huawei HMS / Mobile-Services component) – were removed from the existing System WITHOUT first asking the operator; the operator reversed both. A removal the operator has to discover and reverse after the fact is a defect of the same severity class as a \$. PASS-bluff: the System silently lost a user-facing capability the operator never agreed to drop.

No application, system component, service, package, feature, driver, module, library, prebuilt asset – any already-existing end-user capability of the existing codebase / shipped System – may be removed (deleted, dropped from the

package set, disabled-into-non-shipping, un-bundled, de-listed, or otherwise made unavailable to the end user) WITHOUT FIRST interactively asking the operator and receiving an EXPLICIT keep-or-remove decision. The question MUST be posed through the platform's interactive clarification mechanism per § . . . (AskUserQuestion on Claude Code) – NEVER a free-text “should I remove X?” buried in narrative, NEVER a silent removal justified post-hoc, NEVER an autonomous removal decision. A silent removal is a **release blocker** regardless of how well-intentioned the rationale (deduplication, “it was broken anyway”, geo-restricted, incompatible, superseded) – the operator decides, the agent asks.

What counts as a removal (non-exhaustive): deleting an app/ APK/binary from the build's package set (PRODUCT_PACKAGES / device.mk / equivalent), removing a service from the init/ boot/service-registry set, dropping a kernel module / driver / config from the shipping configuration, un-bundling a prebuilt asset, deleting a submodule or its shipped output, removing a feature flag that gated a live capability, or any edit whose NET EFFECT is “an end-user capability that shipped before no longer ships.” Adding, replacing-with-operator-approved-equivalent, or fixing a capability is NOT a removal. When uncertain whether an edit constitutes a removal, treat it AS a removal and ask (per § . . . no-guessing + § . . . – removal of an existing user-facing capability is high-blast-radius and MUST be operator-confirmed, never autonomously decided). The tracked DROP path: ask → operator approves → mark the item Obsolete (→ Fixed.md) with Obsolete-Details^{11 4 17} reason feature-removed + an operator-approval citation (§ . . .) → then remove; the removal never precedes the operator's yes.

Classification: universal (§ . . .) – a platform-neutral discipline reusable by ANY project that ships a set of user-facing capabilities; the consuming project supplies

its concrete capability-manifest paths per § . . .
Composes § . . . / § . . . /
§ . . . / § . . . / § . . . /
§ . . . / § Propagation gate CM-
COVENANT-114-122-PROPAGATION (literal 11.4.122) + recommended
gate CM-NO-SILENT-COMPONENT-REMOVAL + paired § . . . meta-test
mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md
§ Non-compliance is a release blocker. No
escape hatch – no --remove-without-asking , --silent-removal , --
autonomous-removal-OK , --dedup-removal-exempt , --it-was-broken-
anyway flag.

§ . . . – Rock-solid-proof-or-deep-
research mandate (User mandate,
– –)

Forensic anchor – verbatim user mandate (– –):

“Every single reported issue MUST BE fully and %
validated with rock solid proofs! Nothing can be
considered fixed or completed without hard evidence! No
false results or bluff(s) of any kind is allowed! If we
are not sure on how to achieve full testing, validation
and verification of something we MUST ALWAYS perform deep
web research for all possible data (articles,
documentation, guides, and other resources) and
opensourced codebases which we can use to solve our
problems and perform testing with validation and
verification which produces rock-solid evidence(s) and
leaves no space for false results or any kind of bluff!”

Forensic case study (FACT). In the . . . -dev remediation
the validation method for two feature classes was, at first,
genuinely unclear: relocating a FLAG_SECURE secure surface to

a secondary display (pixel capture returns black) and asserting on-screen content in non-introspectable streaming-app UIs (blank accessibility hierarchy). Rather than declaring them “untestable” or accepting a metadata-only PASS, the cycle performed deep web research (docs/research/testing_frameworks_20260603/) that yielded the CV/OCR/liveness/sink-probe oracle stack (now \$. . + \$. . + \$. .) – making rock-solid evidence possible where it had appeared impossible. “Unclear how to validate” is a research trigger, NEVER a bluff licence.

Every single reported issue, every fix, and every claimed completion MUST be fully and % validated with rock-solid CAPTURED proof per \$. . / \$. . / \$. . before it may be marked fixed / implemented / completed (\$. . closure vocabulary). Nothing may be considered fixed or complete without hard captured evidence – metadata-only / configuration-only / absence-of-error / grep-without-runtime PASS are all forbidden (\$. . / \$. .); no false results, no bluff of any kind, at any layer.

The research-or-don’t-bluff rule (the operative addition): when the agent is UNSURE how to fully test / validate / verify something – when no obvious evidence-producing method exists OR the candidate method would yield only metadata/config/absence-of-error evidence – it MUST ALWAYS first perform deep web research per \$. . + \$. . (official docs, articles, guides, vendor references, standards, issue trackers, reusable open-source codebases) to DISCOVER or BUILD a validation method that produces rock-solid evidence and leaves no space for a false result. Declaring something “untestable” / “not automatable” / accepting a metadata-only PASS WITHOUT first exhausting this deep-research path is itself a \$. . violation – same severity class as a PASS-bluff. The research output

(cited source URLs + the evidence-producing method, OR the literal “NO external solution found – original work” per § 87(2)(b)) is the captured proof the path was exhausted. Only after that research genuinely fails may the item be classified `PENDING_FORENSICS: / Operator-blocked` (§ 87(2)(b)) / `structurally-impossible` won’t-fix (§ 87(2)(b)) – with the cited research as the evidence the classification is earned, never a convenience.

Classification: universal (§ 1.1) – a platform-neutral discipline reusable by ANY project; the consuming project supplies its concrete capture mechanisms + research corpora per § 1.2. Composes § 1.3 / § 1.4 / § 1.5 / § 1.6 / § 1.7 / § 1.8 / § 1.9

Propagation gate CM-COVENANT-114-123-PROPAGATION (literal 11.4.123) + recommended gate CM-ROCK-SOLID-PROOF-OR-RESEARCH + paired \$. meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md

\$. . . Non-compliance is a release blocker. No escape hatch — no `--metadata-pass-suffices`, `--skip-proof`, `--untestable-without-research`, `--config-only-closure-OK`, `--bluff-when-unsure` flag.

```
$ . . - Dead/unwired-code
investigate-before-remove mandate (User mandate,
- - )
```

Forensic anchor – verbatim user mandate (–):

“Before removing any seemingly-dead (zero-importer / unwired) codebase, we MUST investigate via git history where/how it was originally used and how it became dead. Removal is permitted ONLY when we have captured PROOF it is genuinely no longer needed – and that removal MUST be its own separate commit with a proper descriptive message. If there is no such proof, the code MUST be investigated for where/how it should be wired in properly, and any missing or unwired tests MUST be added. We MUST ALWAYS be extra careful with any codebase removal.”

“Zero importers / never called / unwired ⇒ dead ⇒ delete” is a GUESS (\$. .), never a finding – a “no references” result proves only *current* non-reference, not genuinely-unneeded. Before removing ANY seemingly-dead element (zero-importer / never-called / unwired function / method / type / file / module / package / asset / config / build target) the agent MUST FIRST investigate via git history (`git log --follow` , `git log -S / -G` pickaxe across all history, blame on the deleted call-site) and capture as FACT: () WHERE/HOW it was originally wired in, () WHEN/HOW it became dead – call-site deleted deliberately / by mistake (regression) / never-completed / refactored-unreachable, () whether “no references” is real OR a hidden reference the static tool cannot see (reflection / dynamic dispatch / build-tags / codegen / DI / plugin registry / FFI / config-driven wiring). The investigation output (cited commits + determination) is the captured evidence... **Removal is conditional:** permitted ONLY with captured PROOF the element is genuinely no longer needed; that removal MUST be its OWN SEPARATE COMMIT (independently reviewable + revertible, composes \$. . quiescence + \$. . multi-pass) with a descriptive message citing the git-history evidence – plus \$. . operator-confirmation when the element is an

end-user capability; the § . . . tracked.path marks it
 Obsolete,,(→Fixed.md) . **No proof ⇒ do NOT delete:** investigate
 WHERE/HOW to wire it in properly (restore a mistakenly-
 deleted call-site per § . . . ; finish never-completed
 wiring) AND add any missing / unwired tests (§ . . . /
 §_{11 4 4 . 11 4 101} / §_{11 4} . . . – the missing test is part
 of why it drifted into apparent-deadness). **Extra-caution**
default: when uncertain whether removal-proof is sufficient,
 default to NOT removing (investigate + wire + test) per
 § . . . + § . . . + § . . . ; “probably
 dead” is never sufficient – the bar is captured proof.
 Classification: universal (§ . . .) – the consuming
 project supplies its static-analysis / importer-graph tooling
 + hidden-reference mechanisms per § Composes
 § . . . / § . . . / § . . . /
 § . . . / § . . . / § . . . /
 § . . . / § . . . / § . . . /
 § . . . / § Propagation gate CM-
 COVENANT-114-124-PROPAGATION (literal 11.4.124) + recommended
 gate CM-DEAD-CODE-INVESTIGATE-BEFORE-REMOVE (a net-deletion
 commit must be removal-only + cite the git-history
 investigation OR be part of a tracked Obsolete item) + paired
 § . . . meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md
 § Non-compliance is a release blocker. No
 escape hatch – no --zero-importers-means-dead , --delete-unwired-
 on-sight , --skip-git-history-investigation ,
 --remove-without-proof , --bundle-removal-with-other-work flag.

2026 06 04

§ . . . – **Code-review-agent gate**
 before pre-build + main build (mandatory multi-
 layer review) (User mandate,
 – –)

Forensic anchor – verbatim user mandate (– –):

“After all fixes/changes/implementations are done, BEFORE running pre-build tests and the main build, dispatch code-review agent(s) that analyze all work done + all existing data/facts + the existing codebase + current git history to determine quality, safety, and whether the fixes/changes will REALLY work; they MUST validate and verify that every test covering the fixes/changes genuinely validates the work with NO chance of false results or bluff of any kind. Any finding MUST be fixed, polished, improved, and covered with additional tests before the build proceeds. Multiple strong layers of checks.”

After all fixes / changes / implementations in a batch are done, and BEFORE running the pre-build test sweep AND the main (artifact) build (for ANY project), the agent MUST dispatch one or more dedicated code-review agent(s) (subagent-driven by default per § . . . /§ . . .) performing a multi-layer review that: () analyzes ALL work done in the batch (every fix/change + its source diff + stated intent); () analyzes ALL existing data + facts (captured evidence per § . . . /§ . . . / § . . . , tracker entries, prior findings, the § . . . runtime-signature registry); () analyzes the existing codebase (blast radius per § . . . , cross-feature interaction, contract integrity of every dependency); () analyzes current git history (what each change touched, how it composes with concurrent/recent work, whether it reproduces a known-broken pattern per § . . . / § . . .); () determines quality + safety + will-it-REALLY-work (robust + not error-prone – no solve-A-create-B; no host/data/security regression; genuinely delivers the end-user-visible behaviour per § . . . /§ . . .); () validates + verifies the tests covering the work – every covering test genuinely exercises the work-under-test and catches its

negation, with ZERO chance of a false result or bluff (a test that PASSES on broken-for-the-user work, a metadata-only/config-only/absence-of-error/grep-without-runtime assertion, or a gate whose paired § . mutation does not make it FAIL is a finding). Any finding (defect / error-prone change / safety risk / will-not-really-work / bluff-or-false-result-capable test / missing-coverage gap) MUST be fixed, polished, improved, and covered with additional tests (four-layer per § . . (b), TDD-RED-first per § . . / § . . .) BEFORE the pre-build sweep + main build proceed; the review iterates (re-review after each remediation) until no blocking findings remain. The review is itself anti-bluff (its conclusions are captured evidence per § . . . / § . . ; a rubber-stamp review of a defective batch = PASS-bluff). It is one of MULTIPLE STRONG LAYERS – complementing, never replacing, the § pre-build sweep, § . . multi-pass (author-side self-review; § . . adds the structurally-separated reviewer seam per § . . .), § . . four-layer fix-verification, § . . build-readiness verdict, and the post-build / runtime-on-clean-target / user-visible layers. Composes § . . / § . . / § . . / § . . . / § . . . / § . . . / § . . . / § . . . / § . . . / § . . . / § . . . / § . . . / § Classification: universal (§ . . .). Propagation gate CM-COVENANT-114-125-PROPAGATION (literal 11.4.125) + recommended gate CM-CODE-REVIEW-GATE-BEFORE-BUILD (build starts only with a fresh code-review-completed marker for the current batch, produced after the last fix + before the pre-build sweep + main build) + paired § . mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md
§ . . . Non-compliance is a release blocker. No
escape hatch – no --skip-code-review , --build-without-review ,
--no-review-gate , --review-optional , --trust-the-author flag.

§ . . . – Default autonomous-loop
working mode from first prompt (User mandate,
– –)

Forensic anchor – verbatim user mandate (– –):

“Make sure that you continue work in endless fully autonomous loop, do not stop until new fully validated and verified version (tag) is created and published (all submodules and main repo) or IN A CASE OF some other main stream work until it is fully completed with all side work streams and nothing else is left in our working queue! THIS MUST BE ALWAYS the default working mode without us asking you! We tend to achieve ABSOLUTE EFFICIENCY, with this and all other projects which will incorporate this MANDATORY RULE / CONSTRAINT!!! This way of (your) working will be ALWAYS applied / followed / executed / fully respected, as soon as we assign / send first request (prompt) in the session! This stops only if we explicitly say so or nothing is left to be done in current working scope (release that will come / upcoming version)!!! Any mimicking (imitation) of this behavior / rules / mandatory constraints, false results or any kind of bluff(s) is ABSOLUTELY FORBIDDEN!!!”

The endless fully-autonomous loop is the **DEFAULT working mode**, engaged automatically the moment the operator sends the **FIRST** request / prompt of a session – the operator **MUST NOT** have to ask for it, request it, restate it, or re-enable it per session. § . . . framed the endless-loop covenant as an explicit-instruction opt-in (“continue in endless loop

fully autonomously” or a semantically-equivalent phrasing);

\$ is the **capstone** that promotes the same covenant to always-on: from the first prompt onward, every agent operates in the \$ loop discipline as the standing default, with \$ zero-idle, \$ maximum-idle-use, \$ autonomous-decision-over-blocking, and \$ continuous-parallel-stream all engaged by default – no per-session activation handshake. The continuation contract: the loop continues until ONE of two terminal conditions holds – (A) **Release scope** – a new, fully-validated-and-verified version (tag) is created AND published across all owned submodules AND the main repo to all configured remotes (per \$ multi-upstream push + \$ full-suite-retest-before-tag + \$ absolute-no-force-push merge-onto-latest-main); OR (B) **Non-release main-stream scope** – the main-stream goal is fully completed AND every side work stream is done AND the working queue holds nothing left for the current scope. Until (A) or (B) holds, the agent MUST keep working (claim the next priority item, dispatch the next parallel stream, progress every non-blocked item per \$ / \$ / \$ / \$). The loop STOPS ONLY on: () the operator explicitly saying so (STOP / pause / end); () nothing left to do in the current working scope – the upcoming release / current main-stream goal – with the queue genuinely empty per the (A)/(B) terminal conditions; () a \$ host-session-safety demand (the loop yields to host safety unconditionally). Idle-while-blocked parks one work unit, it does not stop the loop – the agent keeps progressing every non-blocked item in parallel per \$ + \$ + \$. Goal – ABSOLUTE EFFICIENCY (no operator-side restart overhead, no idle gaps, no stop-and-wait round-trips); applies to this project AND every project that incorporates this Constitution. Anti-bluff: mimicking / imitating this loop behaviour, narrating

continuation without performing it, fabricating progress, or emitting false / bluff results of ANY kind is ABSOLUTELY FORBIDDEN – this composes the entire § 11.4.126 anti-bluff covenant family (§ 11.4.126.1 / § 11.4.126.2 / § 11.4.126.3 / § 11.4.126.4 / § 11.4.126.5 / § 11.4.126.6 / § 11.4.126.7 / § 11.4.126.8 / § 11.4.126.9 / § 11.4.126.10); the agent MUST genuinely perform the continuous work and capture positive evidence for every closure, and a report claiming the loop ran while no real work / no captured evidence was produced is a § 11.4.126.11 PASS-bluff at the operating-mode layer. Classification: universal (§ 11.4.126.12). Composes with § 11.4.126.13 (the endless-loop covenant – § 11.4.126.14 promotes it from opt-in to always-on default) / § 11.4.126.15 / § 11.4.126.16 / § 11.4.126.17 / § 11.4.126.18 / § 11.4.126.19 / § 11.4.126.20 / § 11.4.126.21 / § 11.4.126.22 / § 11.4.126.23 / § 11.4.126.24 / § 11.4.126.25 / § 11.4.126.26 / § 11.4.126.27 / § 11.4.126.28 / § 11.4.126.29 / § 11.4.126.30 / § 11.4.126.31 / § 11.4.126.32 / § 11.4.126.33 / § 11.4.126.34 / § 11.4.126.35 / § 11.4.126.36 / § 11.4.126.37 / § 11.4.126.38 / § 11.4.126.39 / § 11.4.126.40 / § 11.4.126.41 / § 11.4.126.42 / § 11.4.126.43 / § 11.4.126.44 / § 11.4.126.45 / § 11.4.126.46 / § 11.4.126.47 / § 11.4.126.48 / § 11.4.126.49 / § 11.4.126.50 / § 11.4.126.51 / § 11.4.126.52 / § 11.4.126.53 / § 11.4.126.54 / § 11.4.126.55 / § 11.4.126.56 / § 11.4.126.57 / § 11.4.126.58 / § 11.4.126.59 / § 11.4.126.60 / § 11.4.126.61 / § 11.4.126.62 / § 11.4.126.63 / § 11.4.126.64 / § 11.4.126.65 / § 11.4.126.66 / § 11.4.126.67 / § 11.4.126.68 / § 11.4.126.69 / § 11.4.126.70 / § 11.4.126.71 / § 11.4.126.72 / § 11.4.126.73 / § 11.4.126.74 / § 11.4.126.75 / § 11.4.126.76 / § 11.4.126.77 / § 11.4.126.78 / § 11.4.126.79 / § 11.4.126.80 / § 11.4.126.81 / § 11.4.126.82 / § 11.4.126.83 / § 11.4.126.84 / § 11.4.126.85 / § 11.4.126.86 / § 11.4.126.87 / § 11.4.126.88 / § 11.4.126.89 / § 11.4.126.90 / § 11.4.126.91 / § 11.4.126.92 / § 11.4.126.93 / § 11.4.126.94 / § 11.4.126.95 / § 11.4.126.96 / § 11.4.126.97 / § 11.4.126.98 / § 11.4.126.99 / § 11.4.126.100). Propagation gate CM-COVENANT-114-126-PROPAGATION (literal 11.4.126 across the consumer fleet) + paired § 11.4.126.101 meta-test mutation (strip the literal → propagation gate FAILs; gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md

\$. . . Non-compliance is a release blocker. No
11 4 127
escape hatch — no --ask-before-continuing , --single-turn-only ,
--not-default-loop , --mimic-OK flag.

```
$ . . - Session-handoff resumption-
prompt mandate (User mandate,
                - - )
```

Forensic anchor – verbatim user mandate (–):
 “make sure that in situations like this now when new session is needed you ALWAYS prepare such sentence – which will be valid for particular moment and the phase of the project and enough for work to continue.”

When the agent determines a fresh session is needed (context-window limits, performance degradation) OR the operator asks whether a new session is needed / requests a handoff, the agent MUST ALWAYS prepare + proactively provide a ready-to-paste **resumption prompt valid for that EXACT moment and project phase** – self-contained enough that pasting it into a fresh session resumes work with ZERO loss. Two variants on demand: a SHORT first-sentence (“Read <handoff docs> , then continue <terminal goal> ...”) AND a FULL detailed block. The prompt MUST: () point to the live handoff doc(s) – .remember/remember.md if present + docs/CONTINUATION.md per \$. – read FIRST + git fetch --all ; () state current PHASE + immediate NEXT action + terminal goal; () embed exact live-state anchors (build IDs / artifact MD , device/target serials, commit HEAD, in-flight PIDs + log paths, captured-evidence paths); () restate binding constraints (anti-bluff \$. , no-force-push \$. , exact version/naming, hardware/target gotchas); () be MOMENT-VALID, NEVER a generic template. Handoff doc(s) MUST be current BEFORE the prompt is given (\$.). A missing / stale / generic prompt is a \$. . violation. Composes \$. / \$. / \$. / \$. / \$. . Classification: universal (\$.). Propagation gate CM-COVENANT-114-127-PROPAGATION (literal 11.4.127) + paired \$. meta-test mutation.

Canonical authority: constitution submodule Constitution.md \$. . Non-compliance is a release blocker. No escape hatch – no --skip-handoff-prompt , --generic-prompt-OK , --no-resumption-sentence , --handoff-without-state flag.

\$. . – Always-on device-recording mandate (User mandate, – –)

Forensic anchor – direct user mandate (– –):
we MUST ALWAYS live-record all available data from all devices we use for testing (or known to be under manual testing), EXTRA carefully so it never harms the device / its performance / causes side effects; raw recordings are NOT processed without need (token-conscious) and are ALWAYS git-ignored + code-intelligence-excluded; only curated evidence is committed, and only at release prep.

For EVERY test/debug device the project uses + every device under known manual testing, across EVERY reachable transport (USB / wireless ADB / SSH / serial / network introspection API), the project MUST ALWAYS live-record all analysable data: activities, all logs, performance metrics (CPU/memory/I/O/thermal/load), every sink-side report per \$. . , and any other live-changeable parameter. () **Extra-careful, side-effect-free** – non-invasive read-only probes only, bounded sampling, bounded write-volume, an observer-effect budget; a recorder that perturbs the device-under-test is a \$. . violation, NOT evidence. () **Background + parallel + subagent-driven** per \$. . + \$. . – never blocks the main stream. () **Token-conscious – record-now, analyse-later** – raw data NOT processed without need; the only standing analyse-trigger is release-tag prep (\$. . / \$. .) OR explicit operator ask. () **Raw is git-ignored (with a \$. . regen-mechanism declaration) AND code-intelligence-excluded (\$. . / \$. .)** – only CURATED evidence is committed, and only at release prep under docs/qa/<run-id>/ (\$. .). () **Deterministic layout** <recording-root>/YYYY-MM-DD/<combined main+submodules state hash>/<DEVICE>_<SERIAL>/recording_NNN/<files> . () **Anti-bluff** – a recorder claimed running but with no growing corpus is a

\$. bluff; every curated finding traces to a real raw-corpus path; recorder health is itself captured evidence per \$. . /\$. . .

Composes \$. . / \$. . 11 4 / \$. . /
 \$. . / \$. . / \$. . /
 \$. . / \$. . / \$. . /
 \$. . / \$. . / \$. . /
 \$. . . Classification: universal (\$. .).
 Propagation gate CM-COVENANT-114-128-PROPAGATION (literal 11.4.128) + recommended gate CM-DEVICE-RECORDING-ALWAYS-ON + paired \$. mutation.

Canonical authority: constitution submodule Constitution.md
 \$. . . Non-compliance is a release blocker. No escape hatch – no --skip-recording , --record-without-layout , --commit-raw-corpus , --index-raw-corpus , --analyse-corpus-always , --invasive-probe-OK flag.

2026 06 06

\$. . – Huge-blocker release protocol (User mandate, – –)

Forensic anchor – direct user mandate (– –):
 when a huge blocker is discovered during release validation we MUST stop all testing, fix ALL discovered issues, process all recorded data from the last session, land rock-solid fixes, author NEW validation+verification tests of ALL supported test types, rebuild, reflash, and RESTART the full validation+verification of every fix/change from the last release tag to now – on both devices in parallel, recorded, with real physical captured proofs and no bluff.

On discovery of a HUGE BLOCKER (release-blocking-severity defect: core user-facing capability broken, regression invalidating the in-flight cycle, or blast radius reaching the batch's other fixes) during release validation, execute in order with NO spot-check shortcut: () **STOP all testing** on every device (the \$. . test-interrupt STOP at

release granularity – continuing past a huge blocker is the
 § 11.4.129 PASS-bluff). () **Fix ALL discovered issues** – not
 just the blocker; root-cause each per § 11.4.129 +
 isolate regressions against the last known-good tag per
 § 11.4.129 () **Process all recorded data from the last
 session** – analyse the § 11.4.129 raw-corpus slice (this
 IS the § 11.4.129 () release-prep analyse-trigger).
 () **Land rock-solid fixes** per § 11.4.129 +
 § 11.4.129 / § 11.4.129 + § 11.4.129 () **Author
 NEW validation+verification tests of ALL supported test types**
 per § 11.4.129 + § 11.4.129 , each anti-bluff + paired
 § 11.4.129 mutation. () **Rebuild (full, not module-only) +
 reflash to a CLEAN target** per § 11.4.129 () **RESTART
 the full validation+verification from the last release tag to
 now** per § 11.4.129 – RESTART, never resume – on both/all
 owned devices IN PARALLEL per § 11.4.129 /
 § 11.4.129 , every run RECORDED per § 11.4.129 ,
 real physical captured proofs per § 11.4.129 / § 11.4.129 /
 § 11.4.129 , no bluff. This anchor BINDS the existing
 release anchors for the huge-blocker case (adds STOP→fix-
 all→process-recordings→new-tests-all-
 types→rebuild→reflash→full-restart + the restart-not-resume
 rule), citing them rather than duplicating.

Composes § 11.4.129 / § 11.4.129 / § 11.4.129 /
 § 11.4.129 / § 11.4.129 / § 11.4.129 /
 § 11.4.129 / § 11.4.129 / § 11.4.129 /
 § 11.4.129 / § 11.4.129 / § 11.4.129 /
 § 11.4.129 / § 11.4.129 . Classification:
 universal (§ 11.4.129). Propagation gate CM-
 COVENANT-114-129-PROPAGATION (literal 11.4.129) + recommended
 gate CM-HUGE-BLOCKER-FULL-RESTART + paired § 11.4.129 mutation.

Canonical authority: constitution submodule Constitution.md

\$. . . Non-compliance is a release blocker. No escape hatch – no --resume-after-blocker , --spot-validate-after-fix , --skip-recording-analysis , --skip-new-tests , --module-only-after-blocker , --single-device-restart flag.

\$. . . – **Post-remediation validate-the-fix-FIRST-after-redeploy** (User mandate, – –)

Forensic anchor – direct user mandate (– –): when a blocker discovered during release validation is fixed and a new artifact (rebuild / new flashing image / redeploy) is produced + the target reflashed, we **MUST** first re-test the **SPECIFIC** last-failing features + validate the just-incorporated fixes **BEFORE** the broader / full validation.

When a blocker / critical failure found during release validation is **FIXED** and a new artifact is produced + the target reflashed / redistributed / updated, the agent **MUST:**

- () **re-test the SPECIFIC last-failing features FIRST** (targeted guard tests for exactly the defects this fix addressed) **BEFORE** any broader / full-suite validation;
- () **validate the just-incorporated fixes with real captured evidence** – the \$. . . RED test flips GREEN at RED_MODE=0 on the new artifact AND the \$. . . runtime-signature verifies on the CLEAN target the redeploy produced (metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden per \$. . . / \$. . . ; proof per \$. . . / \$. . . / \$. . . / \$. . .);
- () **only after the targeted fix is CONFIRMED working** proceed to the \$. . . full retest from the last tag to now.

Rationale: a first fix attempt may not work / may be incomplete / may regress again under the new artifact – confirming the targeted fix **FIRST** catches a fix-did-not-take case immediately instead of hours later at the **END** of a full

cycle (then restarting per § 11.4.124); cheap-confirmation-first is § 11.4.125 applied to the post-blocker reflash. This is the § 11.4.126 recent-work-validation gate specialised for the post-blocker-reflash case + the targeted-confirmation phase that GATES

§ 11.4.127's step-full-restart. Honest boundary (§ 11.4.128): "the fix probably took" ≠ "the fix took" – the RED→GREEN.flip + runtime-signature on the new artifact is the proof; a still-FAILing targeted re-test re-enters the

§ 11.4.129 / § 11.4.130 isolate→RED→fix loop, never proceeds to the full cycle on a still-broken fix. Composes

§ 11.4.131 / § 11.4.132 / § 11.4.133 /
 § 11.4.134 / § 11.4.135 / § 11.4.136 /
 § 11.4.137 / § 11.4.138 / § 11.4.139

Classification: universal (§ 11.4.140). Propagation gate

CM-COVENANT-114-130-PROPAGATION (literal 11.4.130) +

recommended gate CM-FIX-FIRST-AFTER-REDEPLOY + paired § 11.4.141 mutation.

Canonical authority: constitution submodule Constitution.md

§ 11.4.142. Non-compliance is a release blocker. No escape hatch – no --skip-targeted-retest, --full-cycle-first, --assume-fix-took, --validate-fix-at-end, --skip-red-green-flip-on-new-artifact flag.

§ 11.4.143 – Standing session-resumption file mandate (User mandate, – –)

Forensic anchor – verbatim user mandate (– –):

"Make this markdown a standard file which will be written EVERY TIME when we need fresh session out of the box! It MUST BE always up to date and in sync so whenever new session is created all we have to do is just point to it!"

Every project MUST maintain a SINGLE canonical, always-current session-resumption file at a fixed, project-declared standard path (declared once per § 11.4.144, never moved

without a `$ . .` operator decision). This file is the OUT-OF-THE-BOX entry point for any fresh session: creating a new session requires ONLY pointing the new agent at this one file. `$ . .` promotes `$ . .` (PREPARE a resumption prompt on demand) into a **STANDING**, version-controlled **ARTIFACT** – ALWAYS present, ALWAYS in sync. (A) **Existence + fixed path** – exists at the declared path at all times, encoded as a literal path in the project-layer instantiation (`$ . .`), never silently moved. (B) **Always written + always synced** – (re)written whenever a fresh session is needed OR the live state materially changes (new HEAD, build/artifact id, phase, device/target state, in-flight job, blocking decision) – the `$ . .` trigger set; a stale resumption file is a `$ . .` violation of the same severity class as a `$ . .` stale-CONTINUATION violation. (C) **Content (composes `$ . .`)** – both **SHORT + FULL** variants; points to `.remember/remember.md` + `docs/CONTINUATION.md` read **FIRST** + `git fetch`; embeds exact live-state anchors (HEAD, build/artifact ids + checksums, device serials, in-flight PIDs + log paths, captured-evidence paths); states **PHASE** + immediate **NEXT** + terminal goal; restates binding constraints (anti-bluff `$ . .`, no-force-push `$ . .`, exact version/naming, hardware gotchas); **MOMENT-VALID**, never a generic template (`$ . .`). (D) **Export + freshness** – `$ . .` scope (synchronized `.html` / `.pdf` siblings) + `$ . .` revision header.. (E) **Out-of-the-box resumption** – a fresh session, given ONLY this file's path, fully resumes with zero additional context. Composes `$ . .` / `$ . .` / `$ . .` / `$ . .` / `$ . .` / `$ . .` / `$ . .` .

Classification: universal (`$ . .`). Propagation gate `CM-COVENANT-114-131-PROPAGATION` (literal `11.4.131`) + recommended gate `CM-SESSION-RESUMPTION-FILE-PRESENT` + paired `$ . .` meta-test mutation.

Canonical authority: constitution submodule Constitution.md
§ . . . Non-compliance is a release blocker. No
escape hatch – no --skip-resumption-file , --ephemeral-prompt-
only , --stale-resumption-OK , --generic-template-OK flag.

§ . . . – Risk-ordered validation
priority mandate (User mandate,
– –)

Forensic anchor – verbatim user mandate (– –):
“We MUST ALWAYS first test and validate features,
functionalities and fixes/changes that have been worked most
recently, the ones which were most problematic, which have
the most chance to crash or break again, the ones which have
been re-opened the most times! Then, after we validate and
verify all this with real (physical) proofs and hard
evidence, with no false results and bluffs of any kind, we
continue with all other existing tests in the test suites!
This IS MANDATORY.”

Tests / validations / verifications MUST run in **RISK-
DESCENDING order** – the highest-risk set FIRST, and ONLY AFTER
that set is fully GREEN with real (physical) captured
evidence does the remainder of the suite run. Risk ranking is
computed from a CLOSED set of factors, highest-risk first:
(a) **most-recently-worked** features / fixes / changes; (b)
historically most-problematic (longest defect history, most
prior fixes/failures); (c) **highest crash/break/regress
likelihood** (greatest blast radius / complexity / dependency
surface); (d) **most-reopened** per § . . . reopens-count
(a high reopen count is the strongest empirical fragility
signal). Each item in the highest-risk set MUST pass with
real (physical) captured evidence per § . . . /
§ . . . / § . . . – no metadata-only / config-
only / absence-of-error / grep-without-runtime PASS
(\$. . . / \$. . .), no false results, no bluff
(\$. . .). ONLY AFTER the entire highest-risk set is

GREEN with captured proof does the rest of the suite run; running the suite in arbitrary order, or running lower-risk tests before the highest-risk set is GREEN, is a violation.

\$. . REFINES/
STRENGTHENS \$. . (generalises “validate the just-fixed items first” to the full risk-ordered set) +
\$. . (adds explicit risk-ordering within the recent/high-risk set) + \$. . (applies the implementation-layer priority discipline to VALIDATION ordering). Classification: universal (\$. .) – the consuming project supplies its recency / problematic-history / reopen-count sources (e.g. \$. . workable-items DB reopens_count + last_modified) per \$. .

Composes . .

\$. . /. /. /. /. /. /. /. /. /. /. /. /.

Propagation gate CM-COVENANT-114-132-PROPAGATION (literal 11.4.132) + recommended gate CM-RISK-ORDERED-VALIDATION-PRIORITY + paired \$. meta-test mutation.

```
Canonical authority: constitution submodule Constitution.md
$ . . . . Non-compliance is a release blocker. No
  1.1 4 1.3.3
escape hatch — no --skip-risk-ordering , --any-order-OK , --
suite-order-fixed flag.
```

§ 2.2.2 – Target-System + hardware
safety mandate (User mandate,
– –)

Forensic anchor – verbatim user mandate (–):
 “Make sure that all changes we do to the System are ALWAYS
 safe for the System itself and for the hardware the system
 runs on! This is MANDATORY.”

Every change to the TARGET system (firmware, kernel, init/boot scripts, drivers, sysfs/devfreq/voltage/clock/thermal/regulator register writes, partition/bootloader/U-Boot, HAL, framework, prebuilts, device config) MUST ALWAYS be safe for BOTH (a) the target System itself – MUST NOT brick, boot-

loop, corrupt data, or render the device unrecoverable – AND
 (b) the hardware it runs on – MUST NOT exceed safe
 electrical/thermal/voltage/clock limits or damage panels/
 storage/radios/regulators. Concrete obligations: ()
 reversible-first – verify irreversible high-blast-radius
 changes (bootloader/U-Boot MD , partition layout) against
 known-good values + capture a pre-op backup (§ .) BEFORE
 applying; () NO unverified hardware-control writes – never
 write an unverified value to a voltage/clock/regulator/
 thermal-throttle/current-limit sysfs node or register that
 could exceed datasheet limits, the safe range established as
 FACT (§ . .), never guessed; () thermal/perf changes
 (forcing a performance governor, pinning the top OPP,
 disabling thermal management) MUST respect the device’s
 cooling design, validated by captured thermal evidence; ()
 flashing MUST use the sanctioned tool + a freshly-built
 integrity-verified image – never an ad-hoc partition write or
 stale/unverified artifact; () unprovable-safety ⇒ blocked –
 a change whose target/hardware safety cannot be established
 from captured evidence is treated as UNSAFE and blocked
 (§ . . + § . . reversible-first +
 § . . rock-solid-proof). DISTINCT from §
 host-session safety: § protects the DEVELOPER’s HOST +
 session; § protects the TARGET device + its
 hardware – both apply, neither weakens the other.
 Classification: universal (§ . .) – the consuming
 project supplies its concrete hardware-control surfaces,
 datasheet-safe ranges, known-good bootloader/image hashes,
 and sanctioned flashing tool per § Composes
 § / § . . / § . . /
 § . . / § . . . Propagation gate CM-
 COVENANT-114-133-PROPAGATION (literal 11.4.133) + recommended
 gate CM-TARGET-HARDWARE-SAFETY + paired § . meta-test
 mutation.

Canonical authority: constitution submodule Constitution.md
§ . . . Non-compliance is a release blocker. No
escape hatch – no --unsafe-hardware-write , --skip-system-safety ,
--brick-risk-accepted flag.

§ . . . – Code-review iterate-until-GO
+ rock-solid-evidence mandate (User mandate,
– –)

Forensic anchor – verbatim user mandate (– –):
“For any fixes/changes given back to us for re-work by the
code-review process, once we fix/improve everything per the
code-review’s requests, we MUST RE-RUN code-review AGAIN
until we get a GO from it with NO new issues reported or
warnings of any kind! All results produced by this whole
process MUST ALWAYS give us rock-solid PHYSICAL evidence that
the fixed/improved codebase really works now as expected,
with no false results and no bluff(s) of any kind.”

When the § . . . code-review returns ANY finding –
BLOCKING, nit, or warning – and the author fixes/improves the
batch per that review, the code review MUST BE RE-RUN, and
MUST KEEP being re-run after each remediation round, until it
returns a clean GO with ZERO new issues AND ZERO warnings of
any kind. A single pass that “addressed the findings” is NOT
sufficient: the corrected batch MUST pass a FRESH adversarial
review (a re-review can surface NEW findings introduced by
the very fixes that closed the prior ones – the § . . .
fix-A-creates-B failure mode). The loop terminates ONLY on a
clean GO (no new findings, no warnings); a residual warning
is itself a finding that re-arms the loop. Every round’s
verdict AND every fix’s validation MUST carry rock-solid
PHYSICAL captured evidence per § . . . /
§ . . . / § . . . (captured audio / video /
sysfs / dumphsys / sink-side / runtime-signature) proving the
fixed/improved codebase REALLY works as expected – never
metadata-only / configuration-only / absence-of-error / grep-

without-runtime; no false results, no bluff at any round; a reported GO unbacked by captured physical evidence is itself a § . PASS-bluff at the review-loop layer.

§ . . REFINES / STRENGTHENS § . .
(iterate “until no blocking findings remain”): it makes the loop EXPLICIT (re-run after every remediation round, not once), raises termination to ZERO findings AND ZERO warnings (not merely zero-blocking), and BINDS rock-solid physical evidence to every round. Classification: universal

(§ . .). Composes § . . / § . . /
§ . . / § . . / § . . /
§ . . / § . . / § . . /
§ . . / § . . . Propagation gate CM-COVENANT-114-134-PROPAGATION (literal 11.4.134) + recommended gate CM-CODE-REVIEW-ITERATE-UNTIL-GO + paired § . meta-test mutation (gate-code = separate work item).

Canonical authority: constitution submodule Constitution.md

§ . . . Non-compliance is a release blocker. No escape hatch – no --skip-rereview, --single-review-pass, --warnings-ok, --evidence-optional flag.

- § . . . – Standing regression-guard suite + every-fixed-defect-gets-a-permanent-regression-test (User man... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .
§ . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .
- § . . . – Real-content end-to-end playback-test mandate (User mandate, – –). Refines/strengthens... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .
§ . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .

- § 11 4 3 11 4 5 11 4 6 11 4 13 – Subtitle/caption content-correctness oracle + secure-display-proxy-honesty mandate (User mandat... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:

§ 11 4 137 . . § . . § . . § . .

§ . . § . . § . .

§ 11 4 138 . . § . . § . .

§ . . § . . § . .

§ . . § . .

§ . .
- § 11 4 1 11 4 11 4 102 – Operator-escape => mandatory bluff-audit + permanent guard (User mandate, .). When t... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:

§ 11 4 139 . . § . . § . .

§ . . § . . § 2026 01 11 84 . .

§ . . § . . § . .

§ . . § . . § . .

11 4 17 11 4 43
- § 11 4 101 11 4 120 11 4 120 – Fresh-process clean-artifact runtime-signature mandate (User mandate, 11 4 140 – –). Refines § ... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .

§ 06 09 . . § 07 01 . . § . .

§ . . § . . > § . . –

Universal action-prefix system (ACTION_NAME ::) (User mandate, > – – ; GRAMMAR_ADDENDUM – – ; arrow form – –). When a > user prompt's FIRST > non-blank line starts with a recognised action prefix, you MUST: (.) look the > action token up in the action registry > constitution/actions/registry.yaml (or \$HELIX_ACTION_REGISTRY); > () if it is a registered action, REPLACE the prefix with that action's > expansion text and apply its rules; () execute the remainder of the prompt > under the expanded instruction.

Five EQUIVALENT forms – same action, same > expansion, same execution: () ACTION_NAME :: <rest> (bare ::), > () PREFIX::ACTION_NAME :: <rest> (namespaced ::), () / ACTION_NAME <rest> > (bare slash), () /PREFIX::ACTION_NAME

`<rest>` (namespaced slash), `> ( ) ACTION_NAME ---> <rest>`
 (bare arrow) $\equiv$ `PREFIX::ACTION_NAME ---> <rest>` `>` (namespaced arrow). Thus `> BACKGROUND :: x` $\equiv$ `DEFAULT::BACKGROUND :: x` $\equiv$ `/BACKGROUND x` $\equiv$ `> /DEFAULT::BACKGROUND x` $\equiv$ `BACKGROUND ---> x` $\equiv$ `DEFAULT::BACKGROUND ---> x`. `> PREFIX` is an action NAMESPACE; the reserved default `>` namespace is **DEFAULT**, and an action runs WITH or WITHOUT the prefix. `>` Grammar (all hold): anchored at the FIRST non-blank line only (mid-prose tokens `>` never match); the action token AND the namespace are UPPERCASE-only `> [A-Z][A-Z0-9_]*` (lowercase never matches); the namespace separator `::` `>` inside the token carries NO surrounding spaces (`PREFIX::ACTION_NAME`), DISTINCT `>` from the action-body separator `" :: "` (one ASCII space on each side of `::` — `>` avoids C++ `Foo::Bar`, YAML `key: value`, URLs) in forms `/`, the arrow-body `>` separator `" ---> "` (one ASCII space on each side) in form `/`, and the `>` slash-body separator (one space) in forms `/`; stacked prefixes (`A :: B :: rest`) apply `>` outer-to-inner, left-to-right (expand `A`, re-scan, expand `B`, then the `>` residual is the task); a leading `\` escapes the prefix for the `::`, arrow, `>` and slash forms (`\BACKGROUND :: x`, `\BACKGROUND ---> x`, `\BACKGROUND x` — `>` treat literally, strip the backslash, NO expansion) so action names can be `>` discussed. **Conflict rule `>` (slash form):** `/ACTION_NAME` (form `/`) is honored as the action ONLY when `> ACTION_NAME` (case-folded) does not collide with a built-in/host slash command `>` (registry `slash_bare: auto` + `slash_conflicts: [..]`); form `> ( / PREFIX::ACTION_NAME )` is ALWAYS unambiguous and always honored. An unknown `>` token that matches the grammar shape (any of the `/` forms) but is NOT registered `>` is NEVER silently expanded or silently dropped — ask which registered action `>` was meant (`$ . . / $ . .`) or treat it as a literal prompt, NEVER invent `>` an expansion (`$ . .`); any prompt not satisfying the grammar is an ordinary `>` prompt and the

system is a no-op. The registered action **BACKGROUND** expands > to: “The following prompt that we will provide *MUST BE* executed in background > in parallel with all main work streams using the subagents-driven development > approach! All work done *MUST PRODUCE* rock solid evidence covered with hard > physical proof(s) that all done is working as expected and as specified without > any false results and without any bluff!” (composes \$. . . / \$. . . > subagent-driven, \$. . . / \$. . . parallel streams, \$. . . background > execution, \$. . . / \$. . . / \$. . . captured physical evidence, \$. . . > anti-bluff). A second registered action **REMINDER** re-surfaces > previously-scheduled, CRITICAL, status-UNCERTAIN work: FIRST verify the ACTUAL > current status from captured evidence (never assume done or not-done – > \$. . . no-guessing), THEN act on the delta – report the captured proof if > genuinely complete, resume from the exact point if partial, action it NOW if > not started, or surface the block per \$. . . / \$. . . – always producing a > status verdict (composes \$. . . / \$. . . / \$. . . / \$. . . / \$. . . / > \$. . . / \$. . . / \$. . .). The system is UNIVERSAL (every CLI agent reads this block via its > context carrier per \$. . .), extensible (new action = new registry row), > decoupled + reusable (\$. . .), and loads out-of-the-box. Classification: > universal (\$. . .). **Canonical authority:** constitution submodule > Constitution.md \$ Non-compliance is a release > blocker. No escape hatch – no --skip-action-prefix , --ignore-prefix , > --no-registry , --invent-expansion-OK , --single-layer-only flag.

- \$. . . – Token-efficiency mandate (research-derived + operator mandate, – –). Every project wor... [full → CLAUDE_ANCHORS_FULL.md +

11 4 7 11 4 7 11 4 35 11 4 50
11 4 78 11 4 79 11 4 96
11 4 102 11 4 123 11 4 125

Constitution.md] · refs: \$. . \$. .
\$. . \$. . \$. . \$. .
11 4 142
\$. . \$. . \$. .
\$. . \$. . \$. .
2026
\$. .

11 4 1 11 4 4

- \$11 4 5 11 4 11 4 17 11 4 25 – Universal code-review mandate – every change reviewed, always, no exception (User mandate,

11 4 70 11 4 ... [full → CLAUDE_ANCHORS_FULL.md +
Constitution.md] · refs: \$. . \$. .
\$11 4 134 11 4 142 \$. . \$. . \$. .
\$. . \$. . \$. .
11 4 143
\$. . \$. . \$. .
\$. . \$. . \$. .
2026 06
\$. . \$. .

11 4 3 11 4 6

- \$11 4 10 11 4 11 4 35 – Real-user-journey mandate for video-streaming-app full-automation tests (User mandate,

11 4 112 11 4 117 11 4 ... [full → CLAUDE_ANCHORS_FULL.md +
Constitution.md] · refs: \$. . \$. .
\$. . \$. . \$. .
11 4 144
\$. . \$. . \$. .
\$. . \$. . \$. .
2026 04 13 **
\$. . \$. .

11 4 5 11 4 14

- \$11 4 17 11 4 11 4 35 – Tracked/recorded-device availability-following mandate (User mandate, – –).

Direct open... [full → CLAUDE_ANCHORS_FULL.md +
Constitution.md] · refs: \$. . \$. .
11 4 145
\$. . \$. . \$. .
\$. . \$. . \$. .
2026 06 10 **
\$. .

11 4 1 11 4 5

- \$11 4 6 11 4 11 4 10 11 4 40 – Independent multi-angle impact-research per change (User mandate, – –). For

EVERY fix... [full → CLAUDE_ANCHORS_FULL.md +
Constitution.md] · refs: \$. . \$. .
\$. . \$. . \$. . \$. .

§ 11 4 144 . . § . . § . .
§ . . § . . § . .
§ . . § . .
11 4 1 11 4 3 11 4 4 11 4 5

- § 11 4 5 • 11 4 • 11 4 17 11 4 23 – Reproduce-first test + same-test-confirms-fix + mandatory extend-to-all-cases workflow (User ma... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] •

refs: § 4 115 • 4 106 § . . § . . § . .
§ 11 4 123 • 11 4 • 130 11 4 135 § . . § . . § . .
§ 11 4 131 • 11 4 • 139 11 4 146 § . . § . .
§ . . § . . § . .
§ 11 4 147 § . . § . .
§ . . § . . § . .
2326 06 10 § . . § . .
11 4 5 11 4 6

- § 11 4 17 • 11 4 • 11 4 59 – Crashed-agent respawn-until-complete + no-work-loss registry mandate (User mandate,

11 4 94 11 4 12 11 4 101)... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs: § . . § . .
§ 11 4 125 • 11 4 • 126 11 4 143 § . . § . .
§ 11 4 144 • 11 4 • 147 § . . § . .
§ . . § . . § . .
§ 11 4 148 § . . § . .
§ . . § . . § . .
§ . . § . .
11 4 5 11 4 6

- § 11 4 10 • 11 4 • 11 4 16 – Workable-item integrity (status+type+id) + comprehensive structured description + bidirectional... [full → CLAUDE_ANCHORS_FULL.md +

Constitution.md] • refs: § . . § . .
§ 11 4 91 • 11 4 • 93 11 4 95 § . . § . .
§ 11 4 104 • 11 4 • 106 11 4 108 § . . § . .
§ 11 4 115 • 11 4 • 123 11 4 146 § . . § . .
§ . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .

- §_{11 4 10 ● 11 4 ● 17 11 4 27} – Per-workable-item testing-diary mandate (User mandate, – –). Every workable item MUST... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .

§_{11 4 108 ● 11 4 ● 122 11 4 140} § . . § . .

§_{11 4 141 . .} § . . § . .

§ . . § . . § . .

§_{11 4 151} . . § . . § . .

§ . . § . . § . .

§ . .

11 4 4 11 4 6 11 4 7 11 4 8
- §_{11 4 17 ● 11 4 ● 0 11 4 33} – Mandatory deep multi-angle web research per change/issue, before declaring fixed or structural... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:

§_{11 4 89 ● 11 4 ● 13 11 4} § . . § . . § . .

§_{11 4 95 ● 11 4 ● 17 11 4 99} § . . § . .

§_{11 4 101 ● 11 4 ● 102 11 4 103} § . . § . .

§_{11 4 104 ● 11 4 ● 112 11 4 116} § . . § . .

§_{11 4 123 ● 11 4 ● 125 11 4 126} § . . § . .

§_{11 4 142 ● 11 4 ● 145 11 4 150} § . . § . .

§ . . § . . § . .

§_{11 4 151} . . § . . § . .

§ . . § . . § . .

§ . . § . . § . .

2026 06 12 **

11 4 6 11 4 17
- §_{11 4 28 ● 11 4 ● 6 11 4 30} – Project-prefixed release-tag/version-naming mandate (User mandate, – –). Verbatim oper... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .

§_{11 4 182} . . § . . § . .

§ . . § . . § . .

§ . . § . . § . .

11 4 5 11 4 6 11 4 9 11 4 10
- §_{11 4 17 ● 11 4 ● 0 11 4 43} – Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage m... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:

§ . . § . . § . . § . .

§ . . § . . § . .

§ 11 4 153 . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .
11 4 2 11 4 3 11 4 4 11 4 5

- § 11 4 6 . 11 4 11 4 35 11 4 20 — Comprehensive per-feature Status + Status_Summary document set with mandatory video-recording c... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] ·

refs: § 11 4 74 . 11 4 86 . § . . § . . § . .
§ 11 4 102 . 11 4 106 11 4 107 . . § . . § . .
§ 11 4 108 . 11 4 110 11 4 134 § . . § . .
§ 11 4 141 . 11 4 153 11 4 155 § . . § . .
§ . . § . . § . .
§ 11 4 154 . . § . . § . .
§ . . § . . § . .
§ 2326 06 . . § . . § . .
11 4 2 11 4 3

- § 11 4 6 . 11 4 11 4 17 11 4 83 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (User mandate,

11 4 122 11 4 137 11 4 138 — [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .
§ 11 4 151 . . § . . § . . § . .
§ . . § . . § . .
§ 2326 06 15 . . § . . § . .

- § 11 4 6 . 11 4 11 4 19 11 4 20 — Project-name-prefixed feature/QA recording filenames (User mandate,

11 4 107 11 4 122 11 4 128 —). Verbatim: “A... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:
§ 11 4 151 . . § . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .
§ . .

- § 11 4 75 11 4 109 11 4 156 – All CI/CD automation (GitHub Actions / GitLab pipelines / equivalents) MUST be disabled (User m... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . . § . . § . . § . . § . .
- § 11 4 26 11 4 10 11 4 65 – GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (User mandate, ... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . . § . . § . . § . . § . . § . . § . .
- § 11 4 35 11 4 10 11 4 83 – Intensive all-feature/flow/edge-case video-recording + read-the-screen content-verification man... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . . § . . § . . § . . § . . § . . § . .
- § 11 4 10 11 4 10 11 4 29 – Mandatory window-specific video recording + vision validation mandate (User mandate, ... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . . § . . § . . § . . § . . § . . § . .
- § . . – Vision-verified recording + HelixQA bridge mandate (User mandate, - -). Compact summar... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .

11 4 161
\$. . \$. . \$. .
2026 06 21 **
\$. . \$. . \$. .
\$. . \$. .
11 4 5 11 4 10

- \$_{11 4 17} •_{11 4 4} •_{4 11 4 76} – Rootless container runtime mandate (User mandate, – –). Compact summary: every project... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs: \$. . \$. .
2026 06 21 **
\$. . \$. . \$. .
\$. . \$. .
11 4 5 11 4 17

- \$_{11 4 27} •_{11 4 4} •_{4 11 4 48} – OpenDesign UI design system mandate (User mandate, – –). Compact summary: every projec... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs: \$. . \$. .
11 4 163
\$. . \$. . \$. .
\$. . \$. . \$. .
2026 06 21 **
\$. .

- \$_{11 4 5} •_{11 4 4} •_{4 11 4 107} – Universal Media Validation & Verification Mandate (User mandate, – –). Compact summary... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs: \$. . \$. .
11 4 164
\$. . \$. . \$. .
2026 06 21 **
\$. . \$. . \$. .
\$. .
11 4 5 11 4 17

- \$_{11 4 28} •_{11 4 4} •_{4 11 4 67} – Universal Constitution Auto-Propagation & Hook System (User mandate, – –). Compact sum... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs: \$. . \$. .
11 4 165
\$. . \$. . \$. .
2026 06 21 **
\$. . \$. . \$. .
\$. .
11 4 10 11 4 17

- \$_{11 4 40} •_{11 4 4} •_{4 11 4 65} – Universal Independent Verification Agent Mandate (User mandate, – –). Compact summary:... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] • refs: \$. . \$. .
\$. . \$. . \$. .

11 4 106
§ . . § . . § . .
230 06 22 84
§ . . § . . § . .
§ . . § . . § . .
11 4 106

- § . . – REPEALED (operator decision,
11 4 107 – –). The Universal Semgrep static-analysis mandate is re... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . .
2005 06 23 84
11 4 6 11 4 17

- § 11 4 29 ● 11 4 10 11 4 40 – Big-work-item feature work-stream lifecycle (User mandate, – –). Compact summary: ever... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .

§ 11 4 142 ● 11 4 145 11 4 147 § . . § . .
§ 11 4 151 ● 11 4 107 § . . § . .
§ . . § . . § . .
11 4 161
§ . . § . . § . .
§ . . § . . § . .
§ . . § . .
11 4 5 11 4 6 11 4 17 11 4 65

- § 11 4 69 ● 11 4 10 11 4 107 – Exported-document independent content + textual + full-visual validation mandate (User mandate,... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:

§ 11 4 161 . . § . . § . . § . .
§ . . § . . § . .
11 4 161
§ . . § . . § . .
§ . . § . . § . .
220
§ . .
11 4 2 11 4 4 11 4 5

- § 11 4 10 ● 11 4 107 11 4 25 – Mandatory comprehensive test-type coverage with anti-bluff captured evidence (User mandate, ... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md]

· refs: § 11 4 107 11 4 161 . . § . . § . .
§ 11 4 161 . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .
§ . . § . . § . .
§ . .

- § 11 4 3 11 4 11 4 12 11 4 107 — Device-independent host-side rendered-UI .visual-proof mandate (User mandate, 11 4 171 — —). Com... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:

§ 11 4 171 . . § . . § . . § . .

§ . . § . . § . .

2026 06 29 **

§ . .

11 4 15 11 4 16
- § 11 4 17 11 4 11 4 65 — Mandatory comprehensive human-readable workable-item .descriptions (User mandate, 11 4 171 — —). ... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .

11 4 172

§ . . § . . § . .

§ . . § . . § . .

2026

§ . .

11 4 6 11 4 17
- § 11 4 40 11 4 11 4 65 — Mandatory production-readiness planning with .realistic .timeline projection (User mandate, -... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .

11 4 172

2026 06 29 **

§ . . § . . § . .

§ . . § . . § . .

11 4 6 11 4 17
- § 11 4 24 11 4 11 4 36 — Containerized + distributed build mandate (User mandate, — —). EVERY build of EVERY co... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs: § . . § . .

§ . . § . . § . .

11 4 174

§ . . § . . § . .

§ . . § . . § . .

2026 06 29 **

§ . . § . .
- § . . — Shared-host process-ownership verification mandate (User mandate, — —). On any shared... [full → CLAUDE_ANCHORS_FULL.md + Constitution.md] · refs:

-
- Figure 1 displays a sequence of 9 diagrams, labeled 1 through 9, arranged in a 3x3 grid. Each diagram shows a set of points (black dots) and a set of lines (gray lines) connecting them. The diagrams illustrate a progression of a system, likely related to the evolution of a network or a complex structure. The diagrams are arranged in a 3x3 grid, with the first row containing diagrams 1, 2, and 3; the second row containing diagrams 4, 5, and 6; and the third row containing diagrams 7, 8, and 9. The diagrams show a progression from a simple grid of points to a more complex structure with additional lines and points.

reserve explicit host headroom (cores+RAM), scale -j against BOTH the memory model AND the process rlimit (privileged nproc-raise for full -j; EAGAIN-safe -j otherwise), never break/bottleneck/wedge. NO fixed low -j for the containerized path. The \$. % ceiling + bare-metal -j cap REMAIN, SCOPED to user.slice-resident work₁₂ - \$. 3 12 12 10 does NOT weaken \$. , it scopes it. Classification: universal (\$. .). Gate CM-MAXRES-DYNAMIC-BUILD + CM-COVENANT- - -PROPAGATION + paired \$. mutation. [full → Constitution.md] · refs:

11 2 177
 \$. \$. \$. \$. \$.
 \$. . \$. . \$. . \$.
 2026 07 04
 \$. . \$.

\$. . - Developer-tooling project-decoupling + invocation-directory operation (User mandate, - -). Compact summary: no project-specific script / hook / alias / binary may be wired (copied / symlinked / PATH-exported / aliased) into a global or shared developer-tooling PATH; shared tooling MUST be project-agnostic and operate on the INVOCATION directory (cwd / explicit path / config arg), NEVER a hardcoded project path; a project-specific helper lives in + runs from its own repo, a genuinely-reusable helper is promoted to the shared toolkit ONLY after being stripped of every project-specific assumption (paths, device serials, package names, region endpoints) + taking its target from the invocation context; a project script reachable on the global/shared PATH is a decoupling violation of equal severity to importing project-specific code into a shared submodule (\$. .); forensic FACT - - - a project cwd-hook symlinked into the global Claude-Toolkit PATH re-coupled every alias to one hardcoded checkout; honest boundary \$. . - decoupling guarantees project-agnostic, not correct (still needs the tool's own tests). Classification: universal (\$. .). Composes \$. . /. /. /. /. . Propagation gate CM-

COVENANT-114-177-PROPAGATION (literal 11.4.177) + recommended gate CM-TOOLING-PROJECT-DECOUPLED + paired \$. mutation.

Canonical authority: constitution submodule Constitution.md

\$. . . Non-compliance is a release blocker. No escape hatch – no --global-symlink-ok , --hardcode-project-path , --wire-into-shared-path flag.

\$. . . – Track-qualified identity for parallel work streams (session-name collision ban) (User mandate,

– –). Compact summary: parallel work streams sharing hardware / a git backing store / a tmux-session namespace / a lock namespace / any single-owner resource MUST be addressed by a TRACK-QUALIFIED identity

(<project>__<track>__<role>), NEVER a bare directory basename; multiple checkouts/clones/worktrees sharing basename

atmosphere collapse into one session/lock/log key + cross-wire tmux targets, lock files, device claims, log dirs

(observed collision –); every registry

row / session name / lock path / log dir / device claim for a ≥ -concurrent-claimant resource MUST carry the track-

qualified key, a bare-basename identity for such a resource

is a violation; honest boundary \$. . . – a unique

identity prevents cross-wiring, it does not itself arbitrate

ownership (that is \$. . . + \$. . . (the

multi-track work-division + exactly-once-claim + device-lock arbitration anchor)). Classification: universal

(\$. . .). Composes

\$. . . /. /. . . / . + \$. . . (the

multi-track work-division + exactly-once-claim + device-lock

arbitration anchor). Propagation gate CM-COVENANT-114-178-

PROPAGATION (literal 11.4.178) + recommended gate CM-TRACK-

QUALIFIED-IDENTITY + paired \$. mutation. **Canonical**

authority: constitution submodule Constitution.md

\$. . . Non-compliance is a release blocker. No

escape hatch – no --bare-basename-ok , --skip-track-qualifier ,

--shared-session-name flag.

§ 11.4.179. – Corruption-isolated parallel git streams (own-.git independent clones, not shared-common-dir worktrees) (User mandate, 11.4.179). Compact summary: when corruption-isolation is a requirement, parallel git work streams MUST each be an isolated repo with its OWN .git (own object store, own index, own lock namespace), NOT .git worktree checkouts sharing one common .git; a shared common-dir is a single point of failure – one stale index.lock / .commit_all.lock / .push_all.lock freezes EVERY stream at once (observed 11.4.179 -h all-track freeze 11.4.179 -h all-track freeze 11.4.179 -h all-track freeze) + one corrupted object store loses every stream; each stream's git rev-parse --git-common-dir MUST resolve INSIDE its own tree, a common-dir resolving to a shared parent is a violation; where CoW/reflink disk-feasibility is the constraint use the § 11.4.179. reflink-clone-with-own-.git path (btrfs/XFS/ZFS/APFS reflink shares extents on disk while keeping a per-stream .git, so isolation + disk-feasibility both hold); honest boundary § 11.4.179. – own-.git contains lock/corruption blast radius, it does NOT remove per-repo stale-lock reaping (§ 11.4.179) nor ff-only no-force integration (§ 11.4.179). Classification: universal (§ 11.4.179). Composes § 11.4.179 / . / . / § 11.4.179 + § 11.4.179 (the multi-track work-division + exactly-once-claim + device-lock arbitration anchor). Propagation gate CM-COVENANT-114-179-PROPAGATION (literal 11.4.179) + recommended gate CM-GIT-STREAMS-OWN-COMMON-DIR + paired § 11.4.179 mutation. **Canonical authority:** constitution submodule Constitution.md § 11.4.179. Non-compliance is a release blocker. No escape hatch – no --shared-common-git, --worktree-when-isolation-required, --skip-git-isolation flag.

§ 11.4.180. – Commit/push (single-writer) wrappers MUST auto-reap provably-stale locks before acquiring (User mandate, 11.4.180). Compact summary: every commit / push / single-writer wrapper MUST, before acquiring its own lock, auto-reap any PROVABLY-stale git lock – one

whose recorded holder PID is DEAD (`kill -0` fails), OR (no PID recorded) older than a defined threshold AND with no live wrapper/git process holding that repo; the reap covers the wrapper's own lock plus the git-internal existence-based locks (`index.lock` , `HEAD.lock` , `refs/**/*.lock`) git does NOT auto-release on holder death; it MUST NEVER remove a lock whose holder is ALIVE (removing a live-held lock corrupts a concurrent writer – § 11.4.179); each reap decision (REAPED / KEPT + reason) is logged as captured evidence (§ 11.4.180 – liveness PROVEN via `kill -0` , never assumed); a wrapper that blocks indefinitely on a dead-holder lock (observed `-h freeze` § 11.4.184 – § 11.4.186) is a violation; honest boundary § 11.4.179 – reaping clears dead-holder deadlock, it does NOT substitute for the own-`.git` isolation of § 11.4.180 . Classification: universal (§ 11.4.180). Composes § 11.4.180 / § 11.4.181 / § 11.4.182 / § 11.4.183 / § 11.4.184 . Propagation gate CM-COVENANT-114-180-PROPAGATION (literal 11.4.180) + recommended gate CM-WRAPPER-STALE-LOCK-AUTO-REAP + paired § 11.4.185 mutation. **Canonical authority:** constitution submodule Constitution.md § 11.4.186 . Non-compliance is a release blocker. No escape hatch – no `--skip-stale-lock-reap` , `--force-remove-live-lock` , `--block-on-dead-holder` flag.